

Introducción

Me es grato presentar un tema imprescindible para los profesionales de la Informática, como es lo relacionado al diseño e implantación de la base de datos.

La importancia de tratar este tema radica en asegurar el desarrollo de un software eficiente orientado a usar el mínimo de recursos informáticos así como conseguir la máxima performance del sistema.

La Metodología presentada se ha generalizado a plantear tres pasos básicos:

1.- Modelo Arquitectónico

2.- Modelo Externo

3.- Modelo Interno

La obra cubre temas desde la Arquitectura del Sistema, pasando por el Diseño Externo que está orientado hacia el usuario, Diseño Interno tanto Lógico como Físico. Finalizando con los temas de Optimización así como la Arquitectura de un Motor de Base de Datos.

El capítulo del Diseño Interno Físico se presenta con todo detalle, porque de sus características depende el nivel de performance que el sistema exhibirá.

En cuanto a los temas de Optimización, se trata de resumir todo lo anterior de una manera práctica y directa, sirviendo de guía para profesionales con poca experiencia en este terreno.

En el ultimo capítulo se presenta la Arquitectura del Motor de una Base de Datos, a modo de conocer y entender el porqué de sus componentes y la mejor forma de utilizarlo en la implementación de una base de datos física.

Diseño de Base de Datos

SOLUCION DEL SISTEMA DETALLADA Y DOCUMENTADA SON LOS “PLANOS DEL FUTURO EDIFICIO”

Objetivos:

- **Debe Funcionar.**- Que se consigan los requerimientos del usuario.
- **Entendible.**- Al ser desarrollado por los programadores, debe tener claridad.
- **Confiable.**- Debe estar preparado para las fallas de usuarios y entorno en general.
- **Flexible.**- Debe adecuarse a nuevos entornos y sistemas futuros.
- **Fácil de desarrollar.**- No debe ser complicado en su construcción.
- **Eficientes.**- Utilizar el mínimo de recursos con el mejor rendimiento.
 - Buena performance
 - Bajos costos

El Diseño de Sistemas es la fase del desarrollo del software, en la cual se plantean las soluciones técnicas respetando los criterios de la optimización.

Como toda etapa debe generar un documento entregable, donde se presentan de manera general y detallada los componentes a ser desarrollados.

Metodología General

Para el tema de Diseño de Sistemas Informáticos, se debe de considerar todos los ángulos del software como:

Diseño Arquitectónico

Diseño Externo

Diseño Interno

Concepto: Independientemente del enfoque a utilizar, sea este estructurado, objetos, etc.; siempre se deberá precisar:

Los Alcances Funcionales, mediante módulos (arquitectura), de manera que se plantea la solución al requerimiento de usuario.

La razón de ser del Sistema Informático, es decir proporcionar información a través de los reportes o consultas (Modelo Externo)

Finalmente la solución técnica para su desarrollo e implantación (Modelo Interno)

Secuencia: La secuencia natural es la planteada, porque después de definir la solución total, se procede a detallar los módulos con el enfoque de resultado final (el qué) para luego encontrar la manera (el cómo)

Secuencia Metodológica

1. Diseño arquitectónico, Módulos
2. Diseño Externo
 - a. Prototipos: Pantallas/reportes
 - b. Impacto :Costo/acceso
3. Diseño Interno
 - a. Base de Datos
 - i. Lógica
 - ii. Física
 - iii. Rutinas
4. Arquitectura del Motor de Base de Datos (metodología para optimización)

La Secuencia Metodológica, se presenta de manera natural y lógica, se recomienda no alterarla para evitar problemas de demora por revisiones o ajustes adicionales al no haberse contemplado detalles que la secuencia lo exige.

A nivel del Diseño Interno el tema de Base de Datos debe verse tanto a nivel Lógico como Físico.

El tema de las rutinas es parte del catalogo de Estructuras Lógicas, que se presentan con todas sus características para conocer sus bondades y tener claro en qué momento y lugar aplicarlo.

Estas rutinas pueden ser ***On-Line*** y ***Batch*** en su situación harto extrema.

Así los sistemas pueden ser clasificados en:

- **On-Line**, (Tiempo-Real). - Critico.
- **Batch**. - Critico.
- **On-Line, Batch**: No Critico. Sistemas no especializados de uso general.

1.Arquitectura

Entiéndase por Arquitectura como el arte de juntar un grupo de elementos para producir una realidad nueva y estéticamente aceptable.

Cada elemento representa y cumple una función determinada, sin embargo el conjunto de ellos da como resultado al sistema.

El sistema resultado es mucho más que la simple agregación de funciones parciales por la naturaleza sinérgica del sistema.

A cada uno de los componentes del sistema se les denomina Módulos. Luego para definir una Arquitectura se deberá definir lo siguiente:

1. Módulos
2. Estructura entre Módulos

Ejemplo: Arquitectura de general de Sistemas para una Empresa.



Figura 1.1

1.1. Requerimientos para diseño

Antes de terminar el análisis, un tema ineludible es indicar las **características mínimas** para que la fase de diseño las tome en cuenta. A este entregable se le conoce como “**Requerimientos para Diseño**” que en pocas palabras nos indica **cómo debe ser el diseño**.

Es importante aclarar que es lo que podría pasar en caso se dejara de lado este documento. En principio si se dejara totalmente libre al diseñador este podría desviarse de la razón de ser del sistema desnaturalizando al mismo. Esto puede suceder dado que el análisis se centra principalmente en **Qué debe hacer** el sistema, más que en el **Cómo debe hacerse**.

Este paso se le conoce como “**el paso de lo ideal a lo real**”, es cuando se plantea la solución técnica para su materialización.

Muchas veces la realidad nos obliga a alejarnos del marco ideal teórico en beneficio de conseguir mejores resultados en el tiempo de respuesta. En teoría dado un problema se pueden plantear infinitas soluciones, pero si se plantea un perfil de la solución este gran número podrá ser acotado a un número finito y limitado de soluciones factibles.

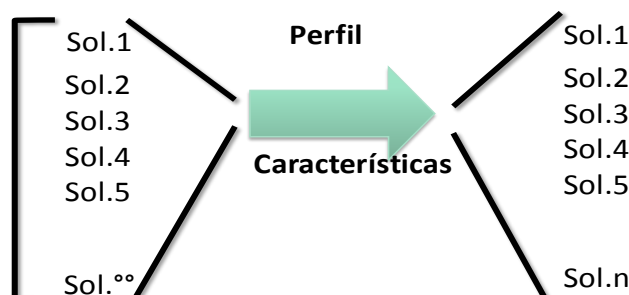


Figura 1.2

Perfil del Sistema: Para plantear las características que definen el perfil del sistema se deben considerar los siguientes parámetros:

1. Modalidad del Sistema
2. Magnitud del Sistema
3. Tiempo de Respuesta
4. Seguridad
5. Auditoria
6. Reversa de Operaciones

A.-Modalidad del Sistema. - Todo sistema normalmente presenta las modalidades de **On-Line** y **Batch**. Siendo ambas antagónicas por naturaleza, generalmente una de ellas prevalece sobre la otra.

On-line. -Ejemplos típicos: Servicios personales de Cajero Automático, Ventanilla de Bancos, etc.

La característica básica es la de procesamiento unitario o de bajo volumen de los procesos. Existen eventos que de manera natural requieren que se materialicen a nivel Informático en el mismo instante de su ocurrencia, a estos eventos se les conoce como eventos de tiempo real. Por tal motivo no tienen una frecuencia definida, dado que depende de la ocurrencia real de la naturaleza.

Por la necesidad de realizarse “en vivo” el sistema debe estar preparado para enfrentar cualquier contingencia ya sea en el plano informático así como en el humano. De allí que estos eventos presentan una complejidad en su planificación, dejando un mínimo margen a la improvisación.

Por su naturaleza estos procesos se caracterizan por:

- Tratar un evento a la vez.
- Se realiza en Tiempo Real (alta complejidad y seguridad)
- Son interactivos. Participan por lo menos dos interlocutores bajo el esquema de Cliente-Servidor.

Batch. - Ejemplos de estos sistemas tenemos: Facturación de servicios, Luz, Agua, Teléfono, etc. La característica básica es la de procesamiento masivo o de alto volumen en los procesos.

Las empresas tienen un ciclo de vida, ya sea diario, semanal, mensual, etc.

Al final de cada ciclo de deben de realizar algunos de los proceso de cambio de ciclo; de manera que se determina un ritmo empresarial el cual puede ser rápido o lento dependiendo de cada una en particular. Cuando existen eventos que involucran a toda la empresa. Las áreas de la empresa deberán sincronizar sus labores a fin conseguir los objetivos planteados.

Normalmente esta sincronización se realiza mediante fechas de cortes, donde todas las áreas deberán respetarla. De esta manera en estos cortes o punto de quiebre se generan gran cantidad de información porque han sido acumuladas durante todo un periodo.

Incluso entre empresas distintas pero que también operan entre sí, se determina los puntos de corte o quiebre para intercambiar información.

De allí que estos procesos presentan las siguientes características:

- Tratan un gran volumen de datos.
- Las transacciones corresponden a un intervalo de tiempo por tanto son diferidas.
- Los procesos no son interactivos. Sólo se realizan en el computador central o servidor.

B.- Magnitud del Sistema.- Este parámetro se encarga de medir el alcance del sistema y por tal motivo su impacto.

La magnitudes pueden ser Local o Corporativo.

Local.- Normalmente se trata de sistemas de apoyo administrativo como Recursos Humanos, Logística y Finanzas.

Se implementa con una Red de Área Local.

El impacto es limitado a los usuarios sin afectar a los clientes. Luego se dice que su nivel de criticidad es relativa. Solo se considera crítica para informar a la Alta Gerencia.

Corporativo.- Al nivel Corporativo involucra a todas las áreas operativas de la empresa incluyendo sucursales y agencias, los sistemas de esta categoría tratan las operaciones del negocio como Ventas, Cobranzas, Producción, etc.. Su impacto es alto porque llega directamente a los clientes, de allí que su nivel de criticidad es alto.

Normalmente se implante en una red de banda ancha, bajo el esquema de cliente / Servidor.

C.- Tiempo de Respuesta.- Los sistemas por su naturaleza pueden tener o no un Tiempo de Respuesta Crítico.

Cuando existe el manejo de Colas se consideran tiempos críticos.

Tiempo de Respuesta Crítico.- Ante la presencia de colas en la naturaleza de un sistema este, debe tipificarse como crítico respecto a su tiempo de respuesta.. En estos casos todo el diseño debe estar orientado a la reducción de tiempo de los proceso. El diseño comprende lo siguiente:

- **Diseño Externo.**- Tratar de minimizar el número menús, y mediante atajos o búsquedas rápidas se pueda llegar al dato para su consulta o actualización.
- **Diseño Interno.**- Disponer de los parámetros de la base de datos para que se encuentre preparada para operar en tiempo real o crear la redundancia que permita usar los atajos del diseño externo.

Para módulos de modalidad **batch**, estos deben respetar una ventana de tiempo definida, si este tiempo no se respeta, se dice que nos encontramos en un tiempo crítico.

Se deberá replantear el diseño del módulo involucrando tareas operativas.

Tiempo de Respuesta No Crítico.- Existen sistemas que a diferencia de los anteriormente descritos no manejan colas o que en el caso de módulos **batch** su ventana de tiempo no es crítica. Por ejemplo sistemas gerenciales para la toma de decisión donde se tratan de tareas de naturaleza analítica y reflexiva. Por tal motivo su diseño será diferente tanto en el nivel externo como en el interno.

D.- Seguridad.- Modulo encargado de restringir el ingreso de personal no autorizado al sistema. Esta restricción se da en diferentes niveles desde cero acceso hasta el máximo nivel de acceso.

Estos diferentes niveles obedecen a diferentes necesidades de acceso a datos. Cada uno de estos requerimientos de información define un perfil de usuario.

Todo perfil de usuario resume el acceso a un tipo de dato y grado de actualización desde consultas y reportes hasta las operaciones de alta, baja y cambio.

E.-Auditoría.- Este módulo es el encargado de supervisar el normal desarrollo operativo del sistema. Esta supervisión es transparente para el usuario, vale decir este no se percata que todos sus actos están siendo controlados de manera remota y dejando huella en cada evento. Estos hechos quedan registrado en el archivo bitácora, desde el cual se emitirán una serie de reportes de supervisión y control conocido como “Diario de Operaciones”.

F.- Reversa de Operaciones.- Existen operaciones que después de haber terminado y actualizado la base de datos, estas deben ser revertidas es decir, que la base de datos quede en el mismo estado hasta antes de su operación(también conocida como marcha atrás). Por tal motivo se debe de contar con toda la información a modo de “foto instantánea” antes de la actualización para dejarla en el mismo estado después de la reversa.

A continuación presentamos los elementos de la Arquitectura, como los Módulos, la Estrategia y Técnicas recomendadas.

1.2. Módulos

Cada uno de los módulos de la **figura 1.1** corresponde a un sistema dentro de la empresa. A continuación se presenta el esquema de una arquitectura de un sistema particular. En este caso se ha hecho un esfuerzo por tener un esquema general que sea válido para todo sistema.

Este esquema es posible, bajo la premisa que todos los sistemas en general presentan las mismas funcionalidades y características al margen de la naturaleza de datos que se manejen. Así en la figura 1.3 se muestran módulos como: Seguridad, Data-Entry, Consultas, etc.

Diseño Arquitectónico

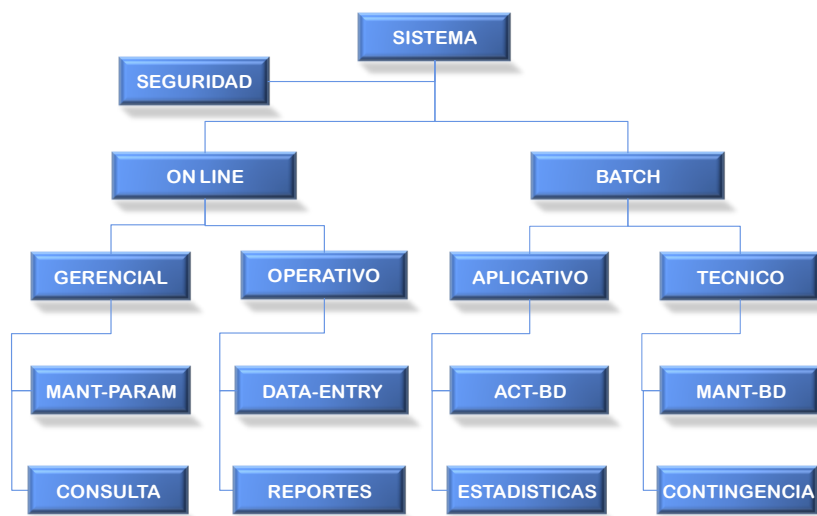


Figura 1.3

MÓDULOS TÍPICOS

SEGURIDAD.- Tiene por función restringir acceso a los datos. Esta restricción es de acuerdo al perfil asignado al usuario.

ON-LINE.- Interactúa con el usuario a modo de un dialogo.

GERENCIAL.- Dirigido hacia usuarios que toman decisiones.

MANTENIMIENTO PARÁMETROS.- Los parámetros son variables que se mantienen fijas para definir escenarios del negocio.

CONSULTA GERENCIAL.- Muestra la base de datos para la toma de decisiones. Información estadística, histórica. Deben mostrar tendencia que puedan proyectar extrapolando ratios.

OPERATIVO.- Dirigido hacia usuarios que operan el negocio día a día.

INGRESO DATOS.- Registro de las transacciones.

REPORTES OPERATIVOS.- Pequeños tipo comprobante de pago.

BATCH.- Procesos internos de grandes volúmenes de datos.

APLICATIVO.- Trata datos por requerimiento del usuario

ACTUALIZACION BD.- Liquidaciones, estados de cuenta, pago de remuneraciones.

GENERACIÓN DE ESTADÍSTICAS / HISTORICOS.- Generación de información para la toma de decisiones. Al estar trabajada agiliza la consulta.

TÉCNICO.- Procesos rutinarios con el fin de tener la base de datos en óptimas condiciones.

MANTENIMIENTO BD.- Con determinada frecuencia deberá reorganizarse la base de datos en cuanto a su estructura física, de-fragmentando y re-asignando áreas de disco que optimice recursos y tiempos.

CONTINGENCIA BD.- Procesos que solo se ejecutaran cuando se requiera reconstruir una tabla maestra que se haya destruido por error.

1.1.1. Concepto

Se entiende por módulo a un conjunto de elementos y que a su vez este es un elemento de un módulo mayor.

Por ejemplo:

RUTINA	= COMANDOS
PROGRAMA	= RUTINAS
PROCESO	= PROGRAMAS
SUB-SISTEMA	= PROCESOS
SISTEMA	= SUB-SISTEMAS

Los elementos en conjunto cumplen una función en el sistema. En principio puede ser una macro función, la cual puede ser descompuesta en funciones más especializadas y estas a su vez en mayor detalle.

1.1.2. Características

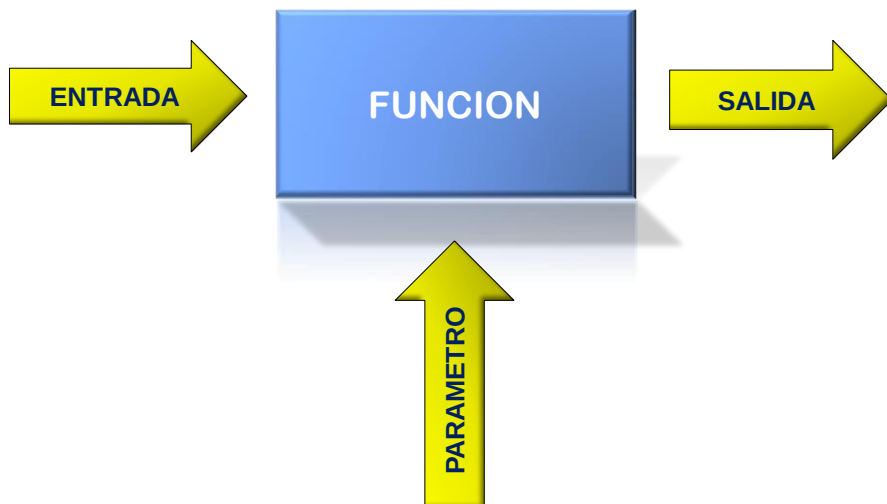
Todo modulo posee:

Función, es la razón de ser del módulo, se deriva de su concepto.

Entrada, trama o estructura de datos requeridos para la ejecución de la función.

Salida, resultado u objetivo del módulo.

Parámetro, variable que permanece fija, para definir escenarios posibles del comportamiento del módulo.



1.1.3. Elementos:

Los elementos del módulo pueden ser:

- Tablas,
- Archivos,
- Programas,
- Área de Datos
- Tramas
- Reportes y Pantallas.

1.1.4. Cohesión:

Estos elementos pueden tener diferentes grados de conectividad o cohesión.

La fuerza que une los elementos de un módulo es la COHESIÓN entre ellos.

Los diferentes grados de Cohesión son los siguientes:

1.- Funcional, es la manera natural de juntarse los elementos. De manera que si un elemento está ausente, el módulo no realizar la función para la cual fue diseñado

2.- Secuencial, cuando existen comando que se ejecutan uno detrás de otro. Si posteriormente estos comandos ya no deben ser secuenciales, el módulo debería ser alterado.

3.- Comunicacional, cuando existe un área de datos que es compartida por más de un programa. Si en el futuro uno de los programas requiere modificar la trama, este cambio alteraría a los demás programas.

4.- Temporal, cuando existen diferentes elementos que se procesan de manera simultánea. Si luego esta temporalidad ya no se cumple el módulo sería alterado.

Los casos 2,3 y 4 no se recomiendan, por las razones expuestas en cada caso.

1.1.5. Estrategia de Definición

Considerando que la Arquitectura está compuesta por un conjunto de elementos donde cada uno de ellos realiza *una función en el sistema, luego para cada función se debe considerar un modulo.*



En base a la modalidad de proceso podemos establecer cierta estrategia:

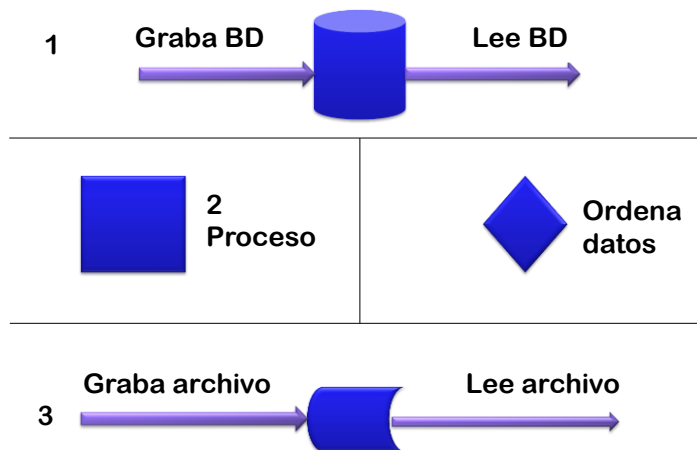
Modalidad On-Line

- **Por tipo de usuario:**
 - Gerencial
 - Operativo

Modalidad batch.- Orientado hacia procesos internos.

- **Por tipo de proceso:**
 - Liquidaciones
 - Actualizaciones
 - Generación de reportes
 - Recuperación de Base de datos.

1.1.6. Nomenclatura de procesos:



Para poder diagramar los procesos, se utiliza un conjunto estándar de elementos que simplifican su construcción y aseguran su lectura por parte del personal de sistemas.

Por ejemplo:

1.- BASE DE DATOS

GRABA-BD: En el diagrama se observa una flecha ingresando a un cilindro.

LEE-BD: En el diagrama se observa una flecha saliendo de un cilindro.

2.- PROCESOS

PROGRAMA: En el diagrama se representa por un cuadrado.

ORDENA-DATOS: Un archivo secuencial ingresa datos desordenados y genera a la salida un archivo con los mismos datos pero ordenados, de acuerdo a ciertos campos o columnas.

3.- ARCHIVOS PLANOS

GRABA-ARCHIVO.- En el diagrama se observa una flecha ingresando.

LEE-ARCHIVO.- En el diagrama se observa una flecha saliendo.

1.2. Técnicas

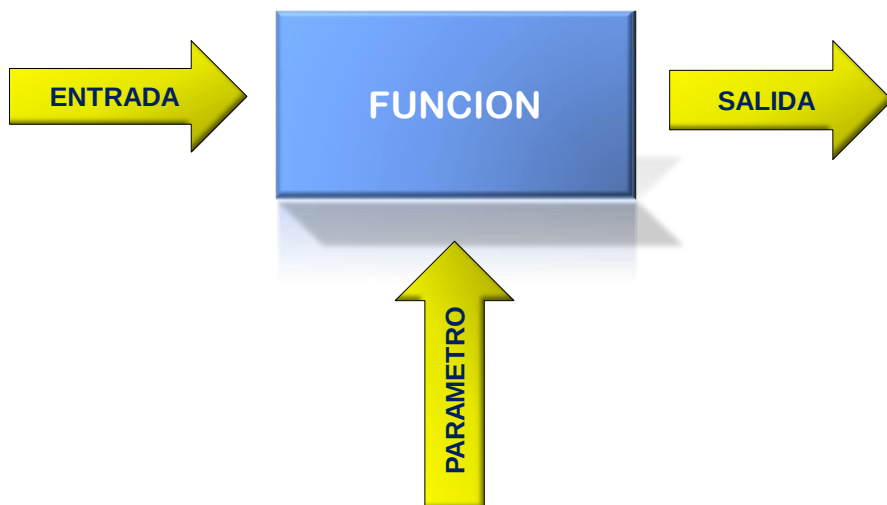
Para diseñar los módulos se presentan tres técnicas, a ser usadas de manera complementaria y en la secuencia establecida.

1.2.1. Caja Negra

Concepto.- Esta técnica permite definir la Función básica que deberá cumplir el modulo.

Ventaja.- El nombre de esta técnica nos dice que solo deberíamos preocuparnos por concentrarnos en definir la función, su entrada, la salida y parámetros que requerirá el modulo. Dicho de manera sencilla definir el “QUE” y no el “COMO”.

Recomendaciones.- Conviene tener muy claro los conceptos y alcances de la función de manera que se pueda proyectar con toda su potencialidad y riqueza de detalles necesarios para su posterior desarrollo e implantación.



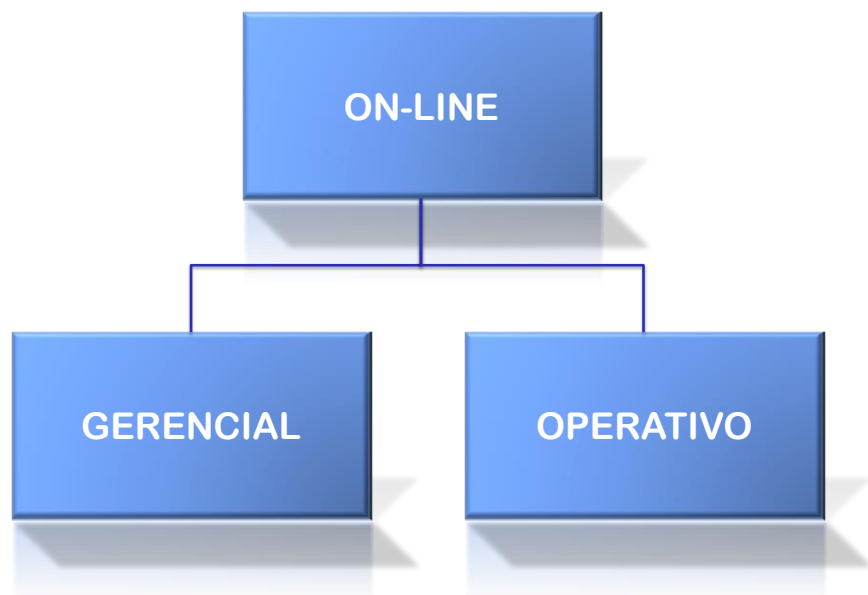
óTop Down

Después de definir los alcances de los módulos, se deben de estructurar entre ellos. Vale decir establecer la relación entre ellos, con lo cual se crea el esquema general del sistema. Un esquema jerarquizado significa que partiendo de un módulo o función general, se baja de nivel presentando los detalles en función de módulos más específicos. Se baja de nivel las veces que sea necesario hasta que el módulo se considere elemental, de allí el nombre de la técnica de “Arriba –Abajo”. Cada vez que se baja de nivel el control lo cede el módulo llamador hacia el módulo llamado. Al final el llamado retornara el control.

Es una manera de diferir la solución del problema o descomponerlo en pequeños y fácilmente solucionables. Se inicia con un problema y se transforma en “n” pequeños problemas. Para el tema de depuración de software esta técnica es muy utilizada al tener la capacidad de aislar los módulos para las pruebas.

La secuencia lógica se impone a la secuencia física.

Al final de este proceso se consigue una Arquitectura modular pero con jerarquía.



1.2.2. Estructuras Lógicas

Después de definir los módulos y la estructura que los relaciona, se debe trabajar al interior del modulo (trabajar en el “COMO”).

En este punto se abre la caja negra, porque se debe precisar los componentes del modulo, se requiere contar con un catalogo de estructuras o modelos básicos, que agrupándolos en determinada secuencia resuelven la función del módulo.

De esta manera estos elementos permiten realizar procesos especializados cubriendo una amplia gama de realidades y de fácil adecuación a casos particulares.

Se muestra a continuación un conjunto de esquemas que resuelven problemas típicos del software.

ESTRUCTURAS LÓGICAS **Estructuras Lógicas**

1. Acceso aleatorio (vía index, vía PTR)
2. Acceso secuencial/acceso dinámico
3. Punto de control proceso(CHCKPOINT)
4. Creación de Tablas(carga BD)
5. Actualización masiva aleatoria
6. Actualización masiva secuencial
7. Validación de datos
8. Clave primaria, secundaria, foránea
9. Cadenas de eventos
10. Listas invertidas
11. Enfoque de redes
12. Acumular movimientos y generar histórico.

1.3. Acoplamiento de Módulos

EL ACOPLAMIENTO ENTRE MÓDULOS ES LA FORMA DE INTERACTUAR ENTRE ELLOS MEDIANTE INTERFASES

Principio de Independencia.- Todo cambio al interior de un módulo no debe afectar a los demás módulos acoplados.

Cuando se acoplan módulos, las interfaces utilizadas entre ellos pueden ser:

Datos.- Es el tipo de interface más natural que debería usarse entre módulos.

Parámetros.- Son variables que se mantienen fijas al interior del módulo. No es recomendable este tipo de interface por que obliga al analista a conocer la naturaleza interna de un módulo, sobre todo cuando este es ajeno.

Áreas de Datos.- Cuando dos módulos están acoplados mediante una área común de datos, tiene la desventaja que en cualquier momento uno de los módulos puede requerir un dato adicional, lo cual obligaría a alterar al otro módulo involucrado.

1.4. Síntesis del Capítulo

El proceso de Diseño de Módulos es un proceso sistemático que maneja Modelos probados y certificados (estructuras Lógicas) para facilitar el desarrollo y mantenimiento del software.

Recomendaciones:

Diseño Modular

Cohesión Funcional

Acoplamiento a través de datos.

ARQUITECTURA (SÍNTESIS)

MÓDULOS	• INTERACCIÓN • JERARQUIA	
ESTRATEGIA	• FUNCION • OBJETO	} MÓDULO
TÉCNICAS	• TOP-DOWN • CAJA NEGRA • ESTRUCTURAS LÓGICAS	

(CATALOGO DE PROBLEMAS / SOLUCIONES)

ARQUITECTURA

Procesos Típicos

Modalidad de Proceso  Tendencia

BATCH

Facturación de servicios Luz, Agua, Teléfono, etc.
Grandes lotes de información a procesar
Cálculo de muchas variables
Frecuencia Periódica

ON-LINE

Servicios personales Cajeros Automáticos y otros
Tratamiento unitario a procesar a la vez
Cálculo Simple
Frecuencia Aleatoria

ARQUITECTURA: PROCESOS

PROCESO: Tratamiento o transformación de datos

PROCESO = PROGRAMAS + DATOS

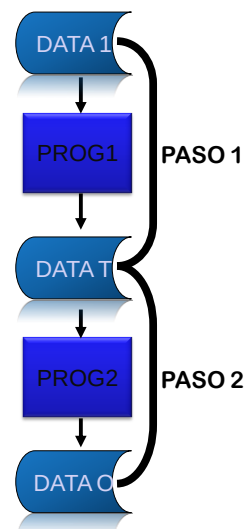
PROGRAMA = Tareas sincronizadas OBJ.

MODELO EN CASCADA

Este modelo esta compuesto por “n” pasos secuenciales, cada paso formado por un programa y archivos de entrada, salida y acceso a base de datos.

Muchas veces se utilizan archivos temporales como interfaces entre dos procesos.

La cantidad de pasos puede ser grande, se debe tener cuidado al reprocesar a partir de un punto para no repetir desde el inicio.



ARQUITECTURA: PROCESOS

PROCESO: Tratamiento de datos

PROCESO = PROGRAMAS + DATOS

PROGRAMA = Tareas sincronizadas OBJ

MODELO N ENTRADAS N SALIDAS



Este modelo es OBJ. típico para módulos que liquidan deudas o pagos, como la facturación de servicios o pagos de remuneraciones. Requiere muchas entradas y genera muchas salidas.

Se presentan dos ejemplos típicos para procesos que tratan grandes volúmenes de datos conocidos como batch. Esta presentación es a nivel introductoria, que debe ser ampliada y detallada en capítulos posteriores.

Sin embargo queda claro que a modo de “estructuras lógicas” están disponibles a ser usadas para caso en particular.

Los ejemplos además ilustran la utilización de los símbolos típicos en estos diagramas, como diagramas de Proceso, Archivos planos o Base de Datos.

La Arquitectura del Sistema resume la solución integral propuesta, que debe satisfacer tanto a los usuarios como a los técnicos encargados del desarrollo del software. La importancia reside en que se constituye en el Plan Maestro de la construcción de la solución.

En base a este plan se pueden planificar:

1.- Tareas

2.- Recursos

3.- Tiempos y Estimar Costos.

2. Diseño externo

El segundo documento entregable que compone el diseño del sistema es el llamado Diseño Externo que contiene todas las presentaciones de los datos mediante diagramas de reportes o de las ventanas en las pantallas.

La importancia de este documento es vital, en la medida que debe satisfacer al usuario, quien es el que aceptara o rechazara la solución propuesta.

Se recomienda que el analista realice este diseño, luego que lo someta a la decisión del usuario, para que este aporte sus inquietudes y sugerencias de cambio.

Esta estrategia tiene la ventaja de definir la base del diseño a cargo del personal técnico y luego los ajustes más de forma que de fondo por parte del usuario. De esta manera ambos se encontraran satisfechos.

2.1. Objetivos y componentes

Diseño Externo: Objetivos

Propósito: La razón de ser de los modelos externos es la mostrar o visualizar el 100% de la data almacenada en la base de datos.

Valor: El valor agregado es contar con información rápida y oportuna, que permita tomar decisiones.

Objetivo: Satisfacer al usuario su necesidad de información. Sea usuario operativo, supervisor o gerencial.

Meta: Obtener el diseño de pantallas que satisfaga 100% a los usuarios.

Los datos almacenados en la base de datos deben ser mostrados de manera que sean considerados información y no simples datos. Este es el valor agregado que la interface debe conseguir además de ser rápida, amigable y oportuna.

Todo dato contenido en la base de datos debe ser mostrado en algunas de las interfaces a desarrollarse, porque de lo contrario sería un dato que sobra en la base de datos. Las interfaces deben ser diseñadas de acuerdo al modulo al que pertenecen y al tipo de usuario a cual se dirigen.

USUARIO	PROCESO	
	MANTENIMIENTO	CONSULTAS
GERENCIAL	MOD-01	MOD-02
OPERATIVO	MOD-03	MOD-04

Diseño Externo: Componentes

Interface: Conjunto de mecanismos que la persona usa para interactuar con objetos.

Facilidades: Interacción con objetos
Refleja realidad del usuario.

Herramientas: CUA(Common User Access)
Componentes básicos CUA

Menús : Ruta hacia los objetos

Ventanas : Ver objetos

Controles : Manipulan objetos (métodos)

2.2. Proto-tipos

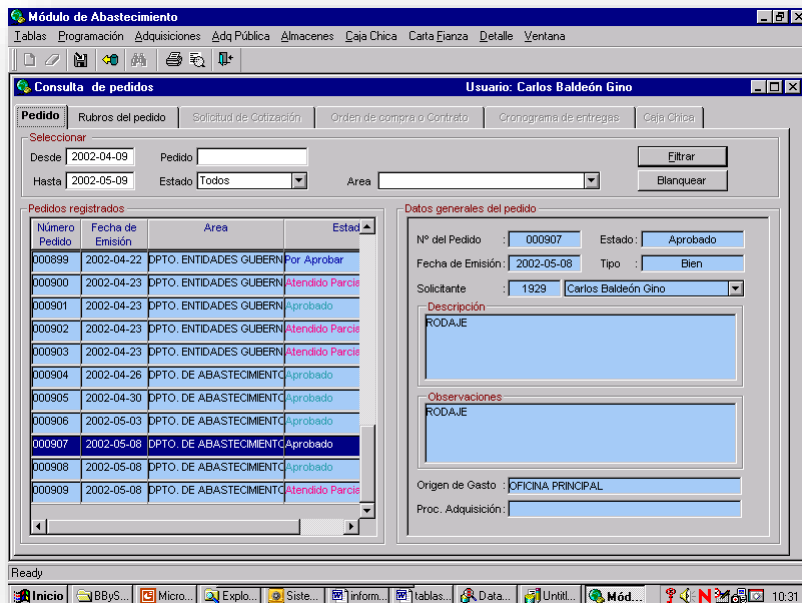


Fig. 2.2 Típica ventana, se muestra una relación de documentos con datos genéricos.

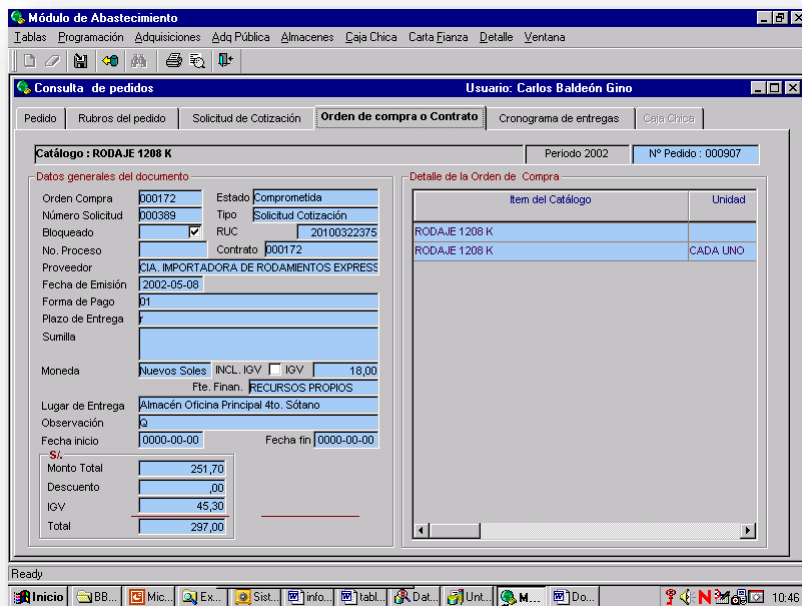


Fig. 2.3 En el grafico se aprecian datos de una orden de Compra.

Diseño Externo: Características

Diseño Detalle de Interacción (Características)

- Consistencia:
 - Información generada a partir de los datos
 - Secuencia de presentación
- Orientar al usuario con opciones por defecto
- Opción de arrepentimiento: Deshacer lo hecho
- Arte y belleza: Diseño gráfico e intuitivo(natural)
- Uniformizar mensajes y teclas de atención
- Contar con la aprobación del usuario
- Modelos típicos: Presentación de Detalle, Masiva y Resumen.

Algunas de las características a tener en cuenta son:

1.- **Consistencia**.- Esta característica es fundamental del modelo externo por su objetivo que es **crear información** a partir de un conjunto de datos. No basta con simplemente con presentar datos sino que se genere sinergia al juntarlos creando información. En este estado es útil al usuario.

Cuando hacemos uso de una ventana para acceder a determinado dato, la **secuencia** de las **pantallas previas** deberán considerarse como una ruta natural y óptima para arribar hacia la pantalla final objetivo que posee la información buscada.

2.- **Orientar al usuario con opciones por defecto**.- En la acción de navegación sobre la base de datos nos encontraremos con ventanas en las que deberán darse algún comando o control para seguir avanzando.

Por ejemplo si nos encontramos en una pantalla de búsqueda alfabética de clientes, lo natural será por **defecto** presentar la primera vista de la pantalla con la relación de nombres a partir desde el primero (que comience su nombre con la letra "A").

3.- **Opción de arrepentimiento. Deshacer lo hecho**.- Operando el sistema, el usuario puede tener la necesidad de modificar algunos objetos de la base de datos, después que se realice los cambios, el sistema deberá advertir que está lista para realizar los cambios de manera definitiva en la base de datos. Presionando alguna tecla función en particular podría **confirmar** y con otra **reversar** todos los cambios realizados.

4.- **Arte y belleza**.- Una buena presentación con un buen formato siempre es recomendable. Además tanto la presentación como la secuencia de ellas deberán ser lo más natural posible, debe entenderse la naturalidad para que nada sea forzado ni antiestético.

5.- **Uniformidad de mensajes y teclas de atención**.- Los mensajes permiten la interacción con el usuario. Estos deben ser informativos y amigables de manera que se tome como un asistente.

En todos los módulos a igual error debe darse el mismo mensaje, de esta manera el usuario conociendo la operatividad de un módulo, podrá fácilmente adecuarse a cualquier otro.

6.-Contar con aprobación del usuario.- A pesar que el personal técnico este en capacidad para realizar el diseño del modelo externo, este deberá siempre ser aprobado por el usuario. Considerando que el usuario es el que dará la conformidad del sistema, la satisfacción y conformidad es clave por parte del usuario.

7.- Modelos Típicos: Presentación Detalle, Masiva y Resumen.- Todas las interfaces de los módulos se clasifican en tres grupos:

a) Masiva.- Después de una pantalla tipo menú, la que generalmente sigue es la que presenta un esquema masivo para realizar una búsqueda y selección.

b) Detalle.- Una vez que se selecciona se debe mostrar el detalle del objeto escogido como se muestra en la siguiente figura.

Módulo de Abastecimiento

Tablas Programación Adquisiciones Adq Pública Almacenes Caja Chica Carta Fianza Detalle Ventana

Consulta de pedidos Usuario: Carlos Baldeón Gino

Pedido Rubros del pedido Solicitud de Cotización Orden de compra o Contrato Cronograma de entregas Caja Chica

Selección

Desde 2002-04-09 Pedido Filtrar
Hasta 2002-05-09 Estado Todos Area Blanquear

Pedidos registrados

Número Pedido	Fecha de Emisión	Area	Estado
000899	2002-04-22	DPTO. ENTIDADES GUBERN	Por Aprobar
000900	2002-04-23	DPTO. ENTIDADES GUBERN	Atendido Parcial
000901	2002-04-23	DPTO. ENTIDADES GUBERN	Aprobado
000902	2002-04-23	DPTO. ENTIDADES GUBERN	Atendido Parcial
000903	2002-04-23	DPTO. ENTIDADES GUBERN	Atendido Parcial
000904	2002-04-26	DPTO. DE ABASTECIMIENTO	Aprobado
000905	2002-04-30	DPTO. DE ABASTECIMIENTO	Aprobado
000906	2002-05-03	DPTO. DE ABASTECIMIENTO	Aprobado
000907	2002-05-08	DPTO. DE ABASTECIMIENTO	Aprobado
000908	2002-05-08	DPTO. DE ABASTECIMIENTO	Aprobado
000909	2002-05-08	DPTO. DE ABASTECIMIENTO	Atendido Parcial

Datos generales del pedido

Nº del Pedido : 000907 Estado: Aprobado
Fecha de Emisión: 2002-05-08 Tipo: Bien
Solicitante : 1929 Carlos Baldeón Gino

Descripción

RODAJE

Observaciones

RODAJE

Origen de Gasto : OFICINA PRINCIPAL
Proc. Adquisición:

Ready

Inicio BByS... Micro... Explo... Siste... inform... tablas... Data... Unitt... M6d... 10:31

Módulo de Abastecimiento

Tablas Programación Adquisiciones Adq Pública Almacenes Caja Chica Carta Fianza Detalle Ventana

Consulta de pedidos Usuario: Carlos Baldeón Gino

Pedido Rubros del pedido Solicitud de Cotización **Orden de compra o Contrato** Cronograma de entregas Caja Chica

Catálogo : RODAJE 1208 K Período 2002 N° Pedido : 000907

Datos generales del documento

Orden Compra	000172	Estado	Comprometida
Número Solicitud	000389	Tipo	Solicitud Cotización
Bloqueado	☒	RUC	20100322375
No. Proceso		Contrato	000172
Proveedor	CIA. IMPORTADORA DE RODAMIENTOS EXPRESS		
Fecha de Emisión	2002-05-08		
Forma de Pago	01		
Plazo de Entrega			
Sumilla			
Moneda	Nuevos Soles	INCL. IGV ☐	IGV 18,00
	Fte. Finan. RECURSOS PROPIOS		
Lugar de Entrega	Almacén Oficina Principal 4to. Sótano		
Observación	Q		
Fecha inicio	0000-00-00	Fecha fin	0000-00-00
S/L			
Monto Total	251,70		
Descuento	,00		
IGV	45,30		
Total	297,00		

Detalle de la Orden de Compra

Item del Catálogo	Unidad
RODAJE 1208 K	
RODAJE 1208 K	CADA UNO

Ready

Inicio BB... Mic... Ex... Sist... info... tabl... Dat... Unt... M... Do...

10:46

c) Resumen.- Finalmente se puede considerar interfaces que resumen los datos más importantes de un módulo.

Se sugiere respetar esta secuencia, por ser natural y practica.

Nueva Actividad - Microsoft Internet Explorer

File Edit View Favorites Tools Help

Back Forward Stop Search Favorites History Print

Address http://msoft/sgi/Cont.htm Go

Sistema de Gestión de Iniciativas

Detalle de Iniciativa

Desestimar
Reasignar
Tomar
Responsabilidad
Cambiar
Prioridad
Fijar Fecha de
Término
Estimar

Título:	Responsable:	Delegado:	F. Inicio:	F. Fin:
Implantando el BSC	Peter Schreier		9/4/2001	9/5/2001
Objetivo Estratégico:	Indicadores:	Meta:		
Implantación del Balanced Scorecard en AT&T Latin America-Perú	Porcentaje de Equipo Directivo entrenado y capacitado	100% del Equipo Directivo apto para manejar la herramienta de Gestión a Junio del 2001		
Estado:	B En Proceso	Revisión Anterior:	9/4/2001	
Avance (%):	0	Próxima Revisión:	4/9/2001	
Nueva fecha Fin:	4/9/2001	Presupuesto (\$/):	166,000.00	
Indicadores de Gestión				
Atraso(%)	Sobrecosto (%)	Replanificación (%)		
10%	-20%	2%		
Observaciones:				

Ingreso Actividad Ver Actividad

Done Local intranet

Start Nueva Activi... My Documents msoft/SGI - M... Jasc Paint Sho... ES 05:52 p.m.

Para el tema de las búsquedas, deben tomar en cuenta la definición de índices alternos al índice de la clave principal. Típicamente los atributos candidatos a claves alternas son los nombres o algún atributo de búsqueda familiar para los clientes.

2.3. Costo de acceso

Diseño Externo: Costo de Acceso

Costo de una vista

Se considera Costo de una vista, al número de accesos al disco y recursos utilizados para conseguir el dato solicitado.

Los recursos utilizados son:

- Espacio en disco
- Memoria intermedia (buffers).

Los buffers están limitados al hardware (memoria). Cuando se agota este espacio deberá de liberarse para conseguir la nueva información.

Antes de realizar acceso a disco siempre verifican si están en el buffer.

El hecho de acceder a una o muchas tablas de la base de datos, demanda una serie de procesos y recursos físicos que se traduce en un tiempo de respuesta que determina la performance o rendimiento del software.

El proceso consiste en los siguientes pasos:

- 1.- Realizar la búsqueda de la clave en el índice (cargado en el buffer). Previamente se ha verificado que el dato solicitado no se encuentra en los buffers de memoria.
- 2.- Con la dirección física de los datos solicitados se les ubica en el disco.
- 3.- Se transmite hacia los buffers de memoria.

De los tres pasos indicados, el paso 2 generalmente se constituye en un cuello de botella, porque al ubicar a cada registro requerido es necesario se manipular físicamente al disco lo cual hace lento al sistema ser este un elemento mecánico y no electrónico. Luego en la medida que aumenta la cantidad de tablas aumentara el tiempo de respuesta.

2.4. Consulta crítica

Diseño Externo: Consulta Crítica

Costo de una vista.-Una vista de la base de datos se da mediante una consulta.

La vista puede visualizar una o más tablas

En la medida que una vista requiera el acceso a más tablas su costo aumentará.

Se deben determinar cuáles son las consultas de mayor costo.

Consulta Críticas: Son las que teniendo un alto costo son muy solicitadas.



Como se observa en la figura, los datos requeridos para esta consulta son muchos y variados siendo necesario el acceso a muchas tablas. Por otro lado esta consulta es muy demandada por los usuarios generándose varias instancias de la consulta original.

Transacciones Críticas

¿Por qué alguna transacción debería ser considerada crítica?

- Ser parte del frente de atención al cliente.
- Ser de las más usadas en el sistema de información
- Tener un tiempo de respuesta inaceptable por el usuario
- Ser un proceso batch para una actualización masiva de Base de Datos, donde el tiempo disponible para todo el proceso es menor que el requerido

Además de las consultas críticas, las transacciones igualmente pueden serlo. Sobre todo aquellas que:

- 1.- **Dan frente al Cliente.**- Cualquier error le llegaría directamente a los clientes, generando alerta y desconfianza en la comunidad de clientes alejándolos del sistema y por lo tanto del negocio.
- 2.- **Son muy solicitadas.**- Si la cantidad de usuarios que demandan una misma transacción podría hacer colapsar al software.
- 3.- Tiempo de respuesta inaceptable.-

On-Line.- Lentitud frente al usuario.

Batch.- Procesos de grandes lotes que requieren más tiempo para su proceso que el asignado por el usuario. Esta situación altera el normal desenvolvimiento del negocio y, podría estar retrasando el inicio de las operaciones diarias. Se presenta en capítulos posteriores, una propuesta de solución para estos casos.

3. Diseño interno

Después de tener especificado el conjunto de interfaces de cada modulo, a continuación se requiere detallar técnicamente la base de datos y la rutinas, con el objetivo de hacer posible los esquemas planteados en el modelo externo de cada modulo.

3.1. Diseño de base de datos

El elemento fundamental en el software es la estructura de la base de datos. El cual será abordado desde el aspecto tanto lógico como físico. En todo este proceso se debe tener presente los parámetros de la optimización:

- 1.- Economía de recursos.
- 2.- Mínimo tiempo de respuesta.

3.1.1. Diseño lógico de la base de datos

El modelo lógico de la base de datos es muy similar al modelo conceptual de datos de la fase del análisis. Donde se definen para cada componente sean entidades o relaciones:

- 1.- Atributos.
- 2.- Formato.
- 3.- Claves primarias y alternas.

3.1.1.1. Concepto

Esta fase es la encargada de generar todas las especificaciones técnicas al detalle de los módulos de la arquitectura. Las especificaciones contemplan el enfoque lógico (en los siguientes capítulo se tocaran el aspecto físico). Se basa en el modelo conceptual de datos, pero adecuándolo para que respete las normas de optimización ya descritas.

3.1.1.2. Principio de la eficiencia

Aplicación de Eficiencia

Este concepto nos indica que nuestro diseño interno debe estar orientado hacia la obtención de un **gran rendimiento al más bajo costo posible**.

En términos informáticos:

- Los costos se plantean a nivel de base datos como la economía de espacio en discos, es decir minimizar la redundancia de datos.
- La performance se mide por el tiempo de respuesta del sistema. En la medida que el sistema responda adecuadamente en un mínimo de tiempo, será mejor la performance.

Estos conceptos, son antagónicos, por lo que se tendrá que buscar un punto intermedio que satisfaga a ambos.

De aquí se pueden sugerir algunos consejos prácticos como:

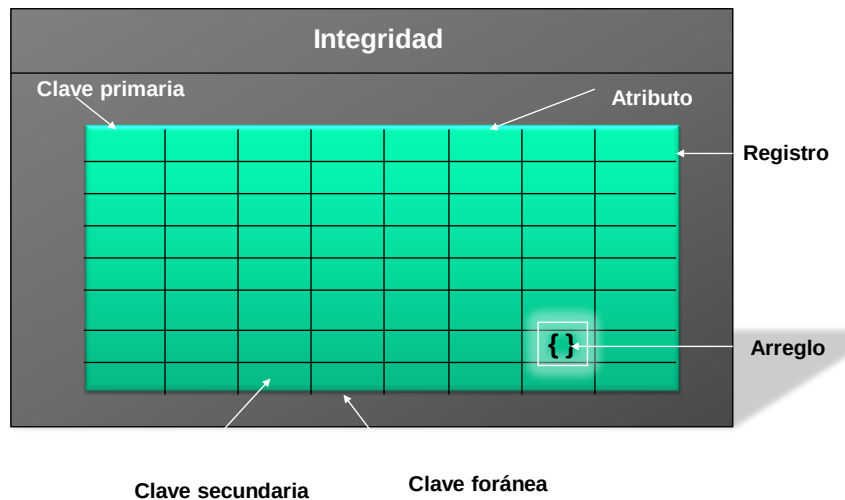
1.- Minimice el número de archivos físicos.

Normalmente por cada archivo físico se generan software para realizar operaciones básicas como alta, baja, cambio y consulta.

2.- Minimice el número de accesos a discos para conseguir datos. Es decir encontrar mejores rutas de acceso hacia los datos.

3.1.1.3. Diseño lógico de archivos

ENFOQUE LOGICO



```

*****
* COBOL DECLARATION FOR TABLE ATOC010.C1TFMVDV *
*****
01 DCLC1TFMVDV.
  10 TFM-COD-CLI-CD      PIC S9(9) USAGE COMP.
  10 TFM-COD-CTA-CD      PIC S9(9) USAGE COMP.
  10 TFM-COD-TIP-DOC-CD  PIC X(2).
  10 TFM-NUM-DOC-NU      PIC X(30).
  10 TFM-COD-EMP-EMI-CD  PIC S9(9) USAGE COMP.
  10 TFM-COD-EMP-EXT-CD  PIC S9(9) USAGE COMP.
  10 TFM-AGR-COD-FTC-CD  PIC S9(9) USAGE COMP.
  10 TFM-COD-TIP-MTO-CD  PIC S9(4) USAGE COMP.
  10 TFM-MTO-TOT-IM      PIC S9(14)V9(4) USAGE COMP-3.
  10 TFM-ULT-TFE-MTO-FF  PIC X(10).
  10 TFM-IND-MTO-EXI-IN  PIC X(1).
  10 TFM-COD-USR-REG-CD  PIC X(16).
  10 TFM-FEC-REG-FF      PIC X(10).
  10 TFM-COD-USR-ATC-CD  PIC X(16).
  10 TFM-TIM-ULT-MOD-TS  PIC X(26).
*****
* THE NUMBER OF COLUMNS DESCRIBED BY THIS DECLARATION IS 15 *
*****

```

Se muestra un ejemplo real de una tabla de una base de datos, se puede observar a nivel 01 el nombre lógico de la tabla y a nivel de 10 los nombres de los atributos. Para cada atributo se le especifica su formato de almacenamiento.

Claves Primarias y Secundarias

A nivel lógico se deben especificar las estructuras de los archivos.

Estas estructuras están compuestas por atributos también llamados campos elementales.

Los atributos pueden ser numéricos, alfanuméricos o cualquier otro formato que presente.

Normalmente los atributos únicos de las entidades se convertirán **Claves Primarias**.

Cuando se requiere acceder por medio de otros atributos que no sea la clave primaria toman el nombre de claves **Alternas o Secundarias**.

Las estructuras se implementan con los niveles 01, 10, etc.

El significado de estos niveles es relativo entre dos niveles. El nivel de mayor valor está subordinado al de menor nivel. Para el ejemplo anterior todos los atributos tienen valor 10 luego están subordinados a la tabla con nivel 01.

Los atributos pueden tener diferentes formatos como CHAR (carácter), binarios, empacados, etc.

La función de las claves primarias básicamente es servir de **identificador único**, es decir al contener el atributo clave un valor que no se repite para otras instancias permite una identificación sin ambigüedades ni confusiones, normalmente el valor utilizado es un número consecutivo. Pero para realizar una búsqueda generalmente este atributo no es un elemento práctico.

De allí que para realizar búsquedas lo natural es utilizar por ejemplo nombre de la persona o entidad correspondiente, o cualquier otro atributo que sea fácilmente recordable y familiar para el usuario.

3.1.1.4. Listas invertidas generan claves foráneas

Claves Foráneas

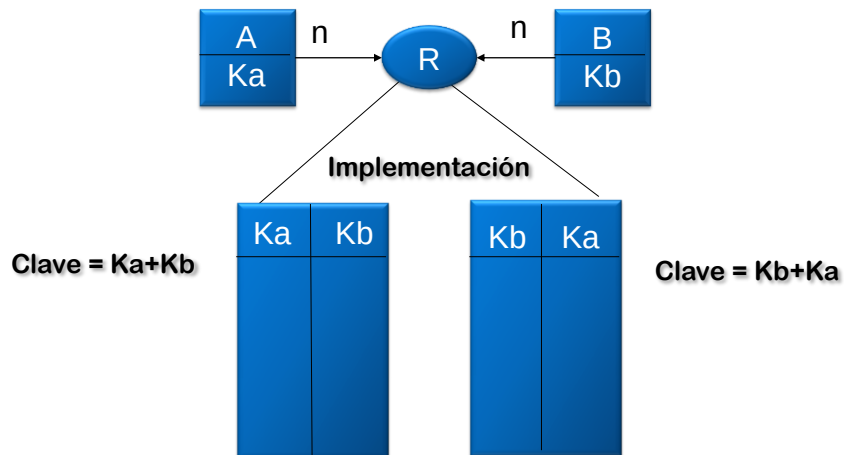
Al igual que se ha indicado la importancia de definir a nivel lógico las claves primarias y alternas. También son importantes las claves conocidas como claves foráneas.

Como su nombre lo indica es un atributo que siendo almacenado en una entidad, sin embargo también es almacenado en otra pero en esta última tiene la categoría de clave.

El hecho de almacenarse en otra entidad diferente a la que pertenece es para jugar un papel de apuntador o direccionamiento.

Aplicación de Listas Invertidas

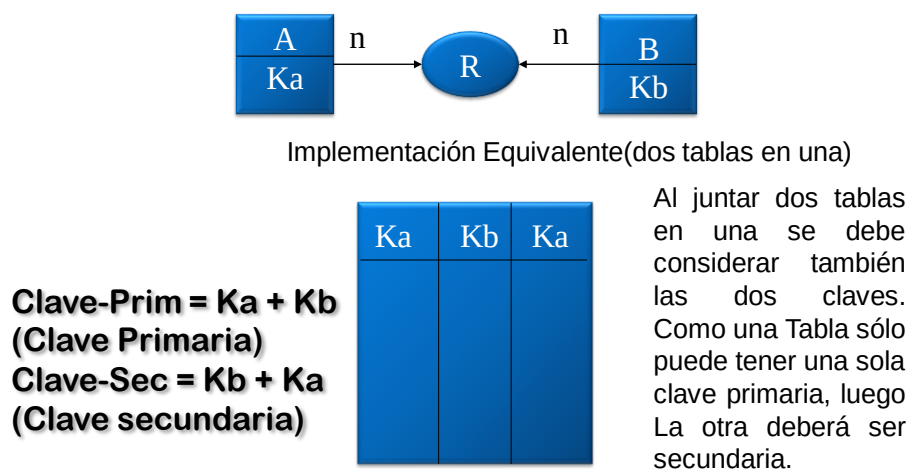
Permite implementar físicamente relaciones entre Entidades con cardinalidad $N \times N$



Este es un ejemplo de una típica relación $n \times n$ originada en el análisis y su implementación generan dos tablas físicas, como se muestra en la figura.

Aplicación de Listas Invertidas

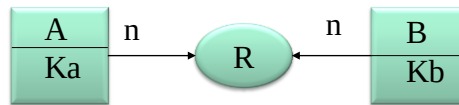
Permite implementar físicamente relaciones entre Entidades con cardinalidad $N \times N$



En este caso las claves primarias y secundarias están formadas por la unión de dos atributos, de esta manera los valores de las instancias son únicas.
Ambas formadas por los mismos atributos en diferente orden.

Ej.: Proveedor – Producto, Producto – Paciente.

Aplicación de Listas Invertidas



Implementación Equivalente

Clave-Prim = $Ka + Kb$

Clave-Sec = $Kb + Ka$

Ka	Kb	Ka
Ka1	Kb1	Ka1
Ka1	Kb2	Ka1
Ka1	Kb3	Ka1
Ka1	Kb4	Ka1

Se recomienda para implementar la seguridad de la información
Por ejemplo Clientes con sus Cuentas

Aplicación de Listas Invertidas

Clave-Prim = $Ka + Kb$

Ka	Kb
Ka	Kb1
Ka	Kb2
Ka	Kb3

**CRECE
VERTICALMENTE
(requiere n accesos)**

Su equivalente con enfoque al modelo de Redes sería utilizando un conjunto de punteros. Para una relación de 1 a "n"

Clave-Prim = Ka



**CRECE
HORIZONTALMENTE
(requiere un solo acceso)**

Se muestra un esquema equivalente para almacenar los atributos de una relación de 1 a n. El primer esquema es típicamente relacional mientras la segunda es bajo el enfoque de redes (se implementa con archivos de longitud variable). Al residir en un solo registro requiere de solo un acceso (salvo el caso de registros esparcidos).

Aplicación de Listas Invertidas

Su equivalente con enfoque a redes sería utilizando un conjunto de punteros.



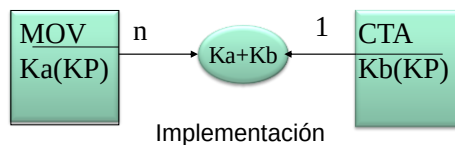
**CRECE
HORIZONTALMENTE
(requiere un solo acceso)**

Es una manera de eliminar la relación y crear la co-relación Mediante claves foráneas. Esta situación se da de **N a 1**
Ejemplo “n” movimientos de una Cuenta. Los movimientos poseen el atributo Cuenta.

Ejemplo:

Supongamos que existen N Movimientos x Cuenta Bancaria

Luego:



Clave = Ka+Kb

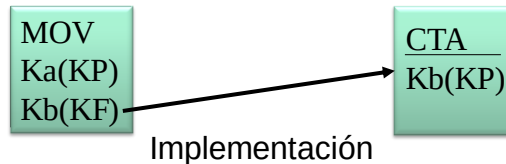
R	
Ka	Kb
Ka1	Kb
Ka2	Kb
Ka3	Kb
Ka4	Kb
Ka5	Kb

Aplicación de Listas Invertidas

Ejemplo:

Supongamos que existen N Movimientos x Cuenta Bancaria

Luego:



R = No se requiere tabla física

La importancia de esta modelo se basa en la eliminación de una tabla física mas, con lo cual se compacta la base de datos y elimina software de mantenimiento asociado a la tabla.

MOVIMIENTO

Movimiento	NROMOV	NROCTA
MOV01	010	M1
MOV02	010	M2
MOV03	010	M3
MOV04	010	M4
MOV05	010	M5
MOV06	010	M6
MOV07	010	M7
MOV08	010	M8
MOV09	010	M9
MOV10	010	M10
MOV11	010	M11
MOV12	010	M12
MOV13	010	M13
MOV14	010	M14
MOV15	010	M15
MOV16	010	M16

CUENTAS

NROCTA	SALDO
010	S1



Esta transformación del modelo Original donde se elimina la Tabla Relación permite minimizar el total de tablas de la BD Física.

Este es el primer caso donde se muestra la diferencia entre el Modelo Conceptual y el Modelo Lógico. En el primero siempre está presente una Relación, en cambio desaparece en el Modelo Lógico al ser reemplazado por las claves foráneas.

Luego para el modelo lógico las entidades absorben a las claves foráneas presentes en una relación.

Clave-primaria
Atributo 1



Atributo n
Clave-foránea

3.1.1.5. CONSULTA CRÍTICA- REDUNDANCIA DE DATOS

Consulta Crítica

Caso: Consulta de Saldos.

Las entidades bancarias, requieren para su operativa diaria una consulta en línea, que muestre la situación general de los clientes.

Esta consulta deberá mostrar en una pantalla:

- a.- Datos del cliente más
- b.- Relación de todas sus cuentas, visualizando para cada una de ellas su CODIGO y SALDO.

Esto significa que para cada cliente se debe realizar “n” accesos para obtener datos del cliente así como de cada una de sus cuentas. Si además se sabe que es la consulta más utilizada por todos los ejecutivos de cuentas, estamos frente a una típica **Consulta Crítica**.

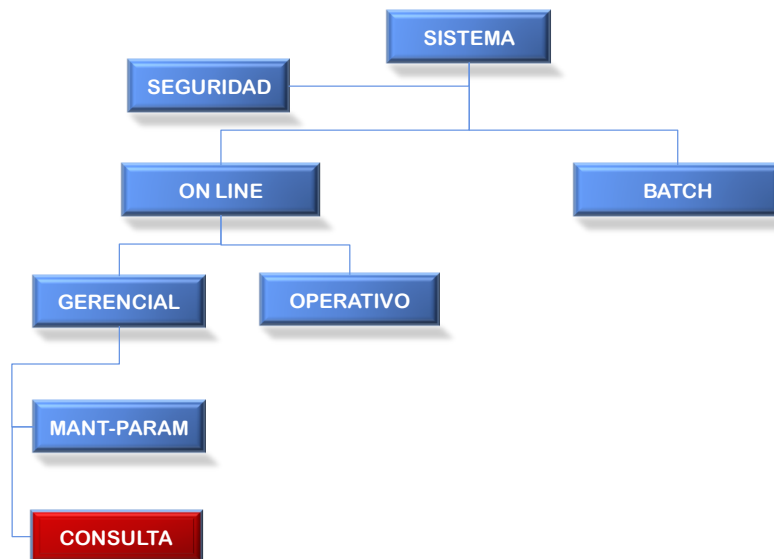
Solución.- Para solucionar todo caso de diseño se debe seguir los pasos hasta ahora tratados como:

- a) Arquitectura.- Se muestra módulo donde se ubica el caso a tratar.
- b) Modelo Externo.- Las interfaces involucradas con el caso en mención.
- c) Modelo Interno

Base de Datos.-

Enfoque lógico: Mostrar el conjunto de atributos que deben ser contenidos por cada una de las tablas o archivos que resuelven el caso. Cada atributo debe quedar completamente definido con su formato de almacenamiento y precisión o longitud en bytes.

Diseño Arquitectónico: Consulta Crítica



Lo primero que se recomienda es ubicar dentro de la arquitectura el módulo al cual pertenece. La consulta crítica hereda las características del módulo Gerencial-Consulta.

Diseño Externo – Consulta Crítica

24 Enero 2007 , 18:42:46 T.Cambio (Referencial) **Compra** S/.3.1550 **Venta** S/.3.2150

REYNA MONTEVERDE/TINO

Acceso y Débito Maestro (Titular) 60402***

Cuentas y Productos

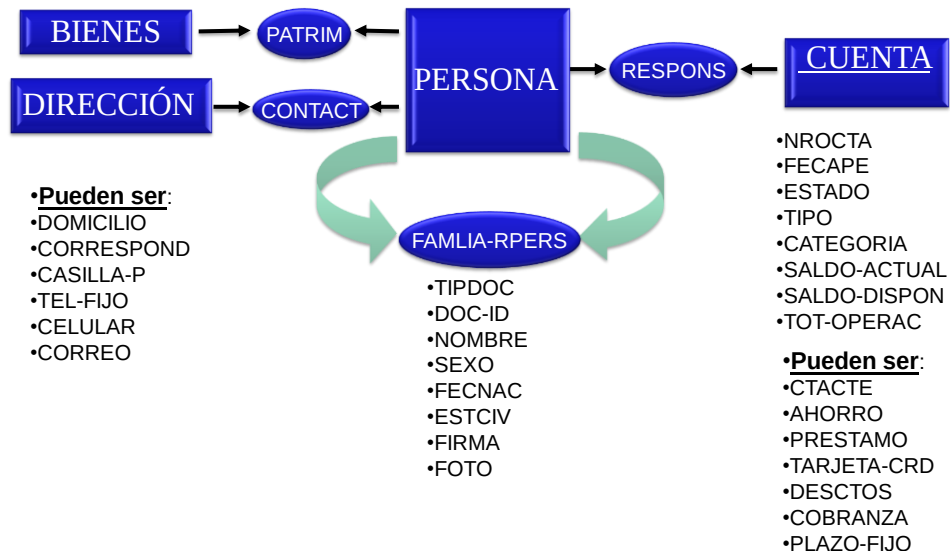
Ahorros y Cuentas Corrientes				Actualiza Saldos
Tipo	Número	Moneda	Saldo disponible	Detalle
* Cta.Cte.	000-2575299	Soles	999999.00	Elija opción
* Cta.Cte.	000-1038047	Dólares	999999.00	Elija opción
* Ahorros	000-7034689	Soles	999999.00	Elija opción
Ahorros	001-1104270	Soles	999999.00	Elija opción
* Ahorros	000-7104200	Dólares	999999.00	Elija opción
* CTS	170-9010644	Dólares	999999.00	Elija opción

Los saldos que se muestran son saldos disponibles, a menos que se indique lo contrario.

Las cuentas marcadas con (*) se encuentran afiliadas a su tarjeta.
Usted sólo podrá efectuar transacciones bancarias entre cuentas afiliadas.

El cliente está de acuerdo con los productos mostrados. En caso requiera que no se muestre algún producto a partir de la fecha en Scotia en Línea, deberá indicarlo por medio de la opción **Restringir Cuentas**.

CONSULTA CRÍTICA



El Modelo Conceptual es una entrada para este proceso. Es nuestro punto de partida, nos indica los datos y la estructura que entre ellos existe. Así se puede simular la navegación necesaria para obtener la información requerida.

Consulta Crítica Simplificada

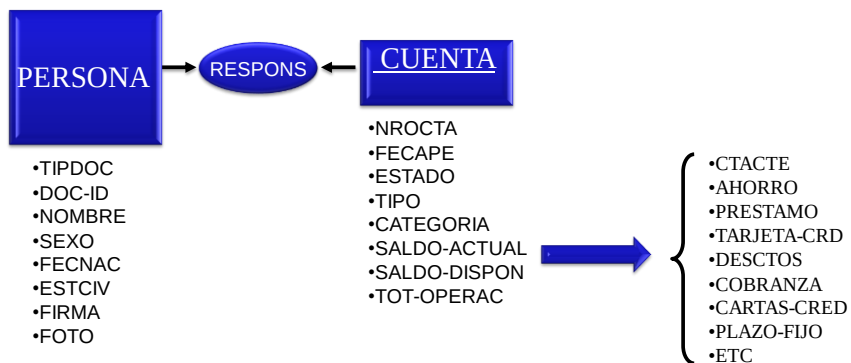
Una manera de solucionar una típica consulta crítica es creando tablas **suplementarias** a las originales (redundancia) de manera que se disponga de esa consulta previamente elaborada, sin sacrificar la información original.

Al estar la consulta previamente elaborada, evita su construcción en Tiempo Real minimizando así el tiempo de respuesta.

La redundancia debe ser controlada a fin de economizar el espacio en disco.

Por ejemplo en el caso propuesto toda la información sobre el cliente se puede resumir en un solo archivo.

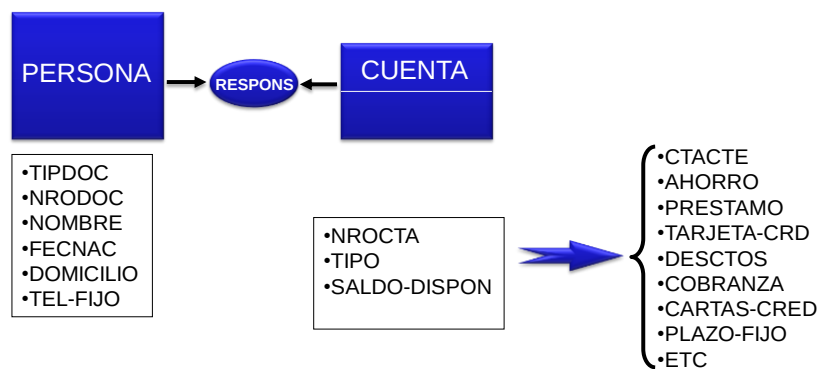
CONSULTA CRÍTICA- Simplificada



Para el caso de los clientes.

Se procede de manera similar con las Cuentas.

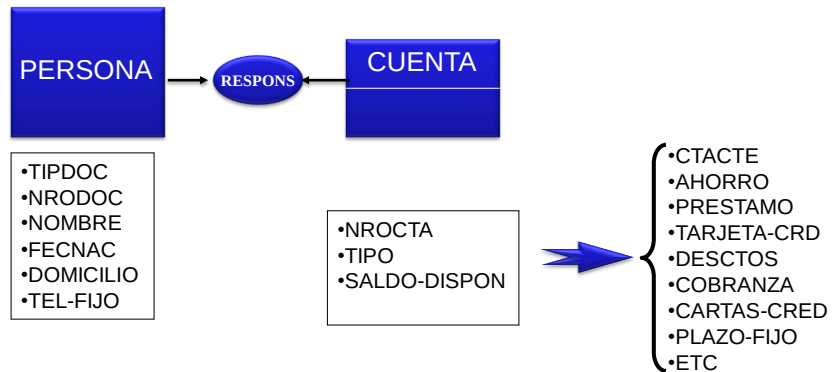
CONSULTA CRÍTICA



Para el caso de las cuentas se puede simplificar de la siguiente manera.

CONSULTA CRÍTICA

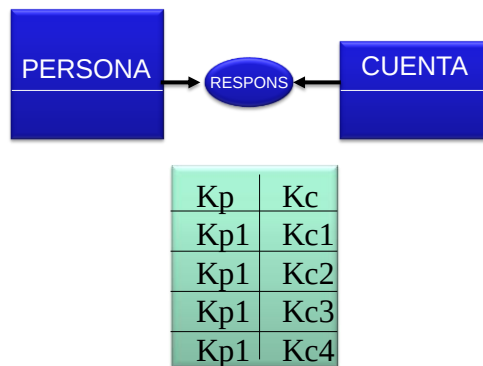
SIMPLIFICADA



Luego: Se accede a PERSONA y mediante la relación RESPONS se llegará hacia CUENTA. Lo cual demanda “n” accesos para acceder a RESPONS y “n” accesos para CUENTA. La solución óptima sería si se compacta en un solo archivo y un solo registro.

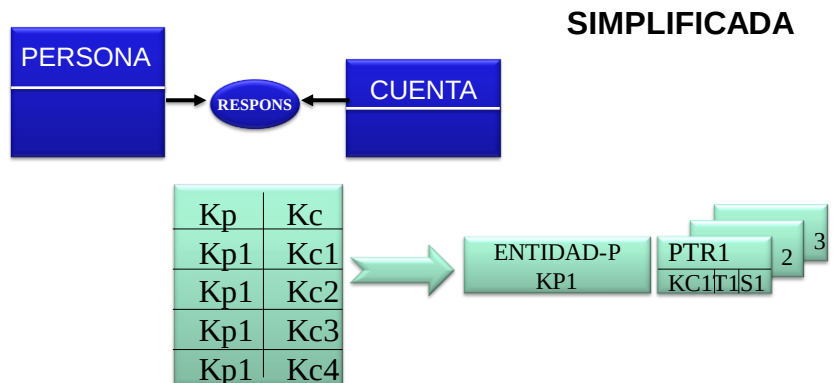
CONSULTA CRÍTICA

SIMPLIFICADA



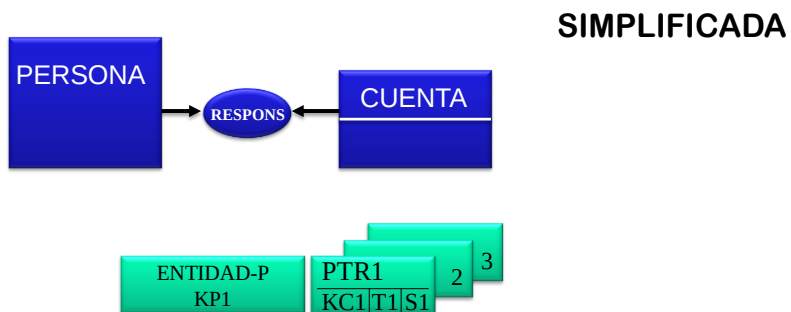
Para eliminar la relación esta deberá ser asimilada por el archivo principal a modo de atributo múltiple (arreglo) el cual contendrá en cada elemento la clave de la Cuenta destino.

CONSULTA CRÍTICA



Para eliminar la relación esta deberá ser asimilada por el archivo principal a modo de atributo múltiple (arreglo) el cual contendrá en cada elemento la clave de la Cuenta destino mas los datos de la cuenta como TIPO y SALDO.

CONSULTA CRÍTICA

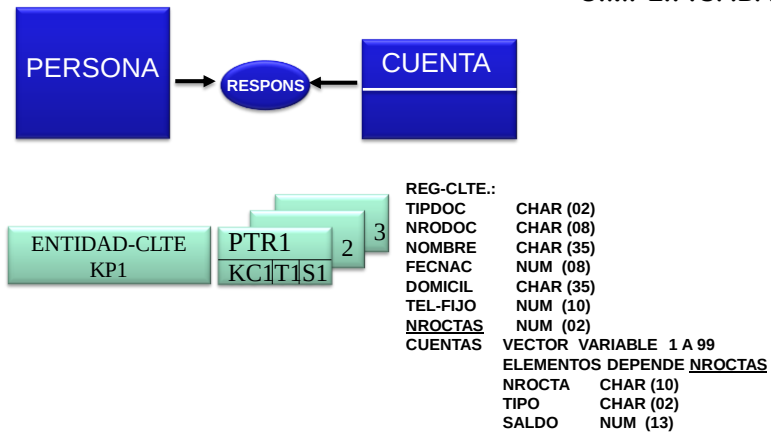


Finalmente el arreglo deberá tener un número variable de elementos para utilizar sólo el número exacto de elementos.

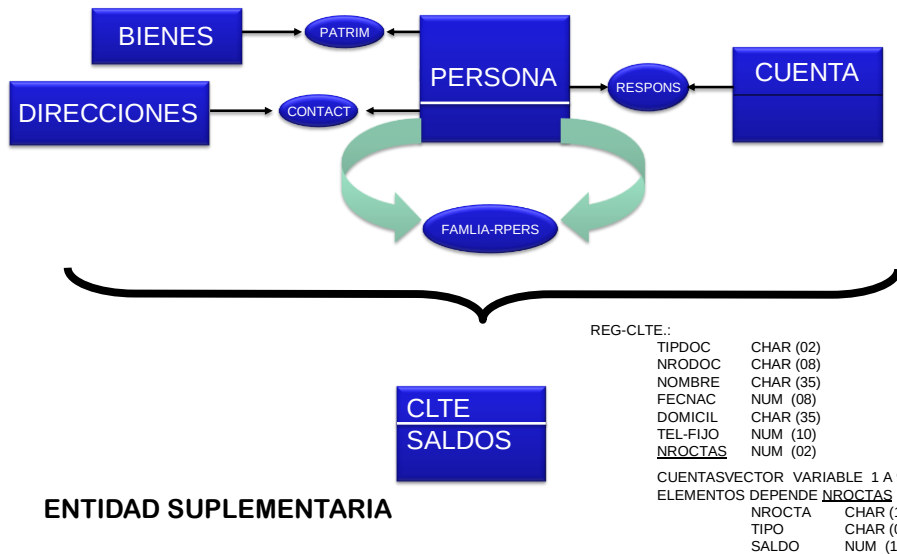
El **número de elementos** del vector debe estar dado por un parámetro contenido en el mismo registro.

CONSULTA CRÍTICA

SIMPLIFICADA



CONSULTA CRÍTICA



De esta manera se demuestra, como al crear redundancia en la base de datos mediante una tabla suplementaria, se logra minimizar el número de accesos a disco, asegurando una óptima performance.

Se recomienda esta estrategia de solución sólo después que se ha demostrado la existencia de transacciones o consultas críticas. Entiéndase como crítico el acceder a muchas tablas de la base de datos. En números podríamos decir a por encima de 10 tablas. Se debe tener cuidado con el hecho de acceder a pocas tablas pero a muchas instancias (objetos), si la situación de una tabla es de fragmentada podría requerir muchos accesos a pesar que los objetos a recuperar lógicamente se encuentren en secuencia.

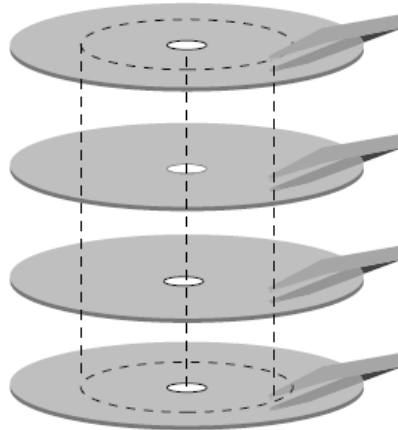
Las tablas redundantes generalmente son generadas en procesos nocturnos de lotes y se recomienda que sean solo de consultas.

3.1.2. Diseño Físico de la base de datos

3.1.2.1. Características y limitaciones físicas

- **Unidad de Control de discos.** Elemento de hardware que controla el tráfico de datos entre los buffers y los discos.
- **Descripción física de los discos.**
 - Paquete de N discos (N depende del modelo de discos).
 - Semejantes a los CDs.
 - Se usan ambas caras del disco.
 - La primera cara del disco almacena información de control
 - Las pistas son concéntricas y tienen la misma capacidad.





- Cálculo de la capacidad de área
- Sean:
 - **A=Numero de discos**
 - **B=Numero de pistas**
 - **C=Numero de bloques físicos por pista (por ejemplo de 4096 bytes)**
 - **Luego: Memoria=(2^a-1)*B*C*4096 bytes.**

Las pistas son concéntricas y tienen la misma capacidad

- Los discos giran a gran velocidad sobre un eje concéntrico.
- El movimiento del brazo de lectura es hacia el centro o hacia el borde externo del disco.
- Lo más lento del sistema es el movimiento del brazo, que en su extremo alberga el cabezal de lectura.



Si comparamos las velocidades de los componentes del hardware encontramos que mientras el ciclo de memoria se mide en nano-segundos el acceso a discos su medida es en milisegundos luego la diferencia de la magnitud entre ambos componentes es de un millón. Así se demuestra que el cuello de botella en el sistema informático está en el acceso a la base de datos que reside en los discos.



CILINDROS

- Conjunto de pistas paralelas
- El número de cilindros es igual al número de pistas
- Desde la misma posición del disco se accede a $(2A-1)$ pistas
- Minimiza el movimiento mecánico y optimiza performance
- Se recomienda como factor de crecimiento para archivos físicos grandes de grandes volúmenes de datos.
- Capacidad: $(2^a-1)*C*4096$ (ejemplo)

Unidades de Medida de discos

Sector	=0.5Kbytes
Bloque Físico	=8 Sectores
Pista	=C bloques Físicos =8*C sectores
Cilindro	=(2A-1)pistas =(2 ^a -1)*8*C sectores
Disco	= B cilindros
A: Discos por paquete	
B:Pistas por disco	
C:Bloques físicos	

DISEÑO INTERNO

(Base de datos física)

Determina la forma de almacenar, recuperar y transmitir la data.

INPUTS:

- El Modelo Conceptual está formado en físico para:
 - Minimizar el espacio utilizado en el disco
 - Minimizar el tiempo de respuesta del disco
- El Modelo Externo indica cuales consultas son críticas.

En esta fase se determina todos los componentes internos del sistema, vale decir todos aquellos que no son visibles a los usuarios.

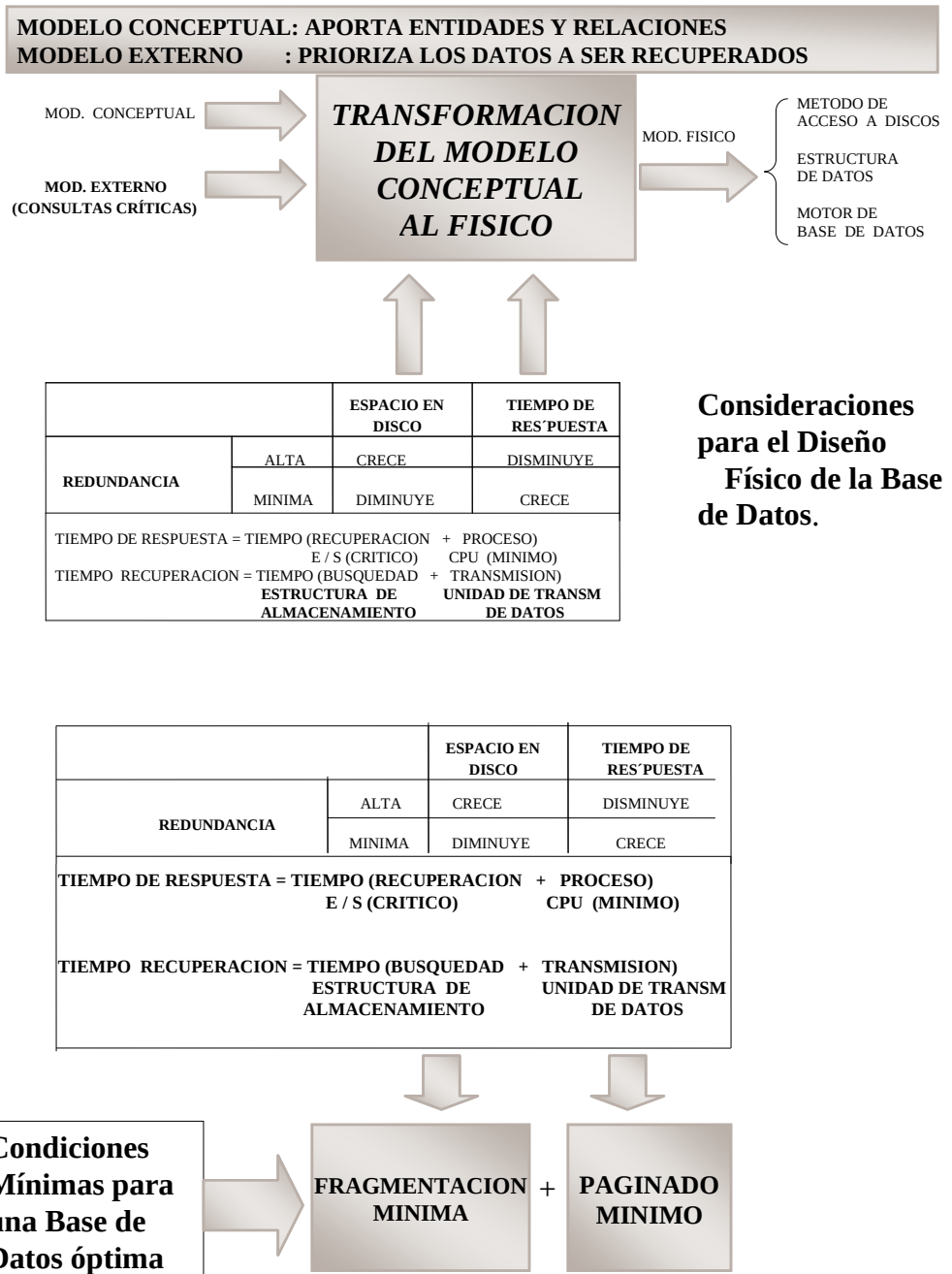
Estos componentes fijan la manera de Almacenar, Recuperar y Transmitir la data desde los discos hacia la memoria RAM.

El objetivo del diseño es optimizar los componentes de manera que se asegure el menor tiempo de respuesta así como el menor espacio en disco posible.

Para esta fase se requieren de insumo de entrada el Diccionario de Datos estructurado en el Modelo Conceptual, distinguiendo las entidades de las relaciones.

Las interfaces se consideran críticas cuando para su generación requieren acceder a muchas tablas, a estas se las identifican y se les da un tratamiento especial.

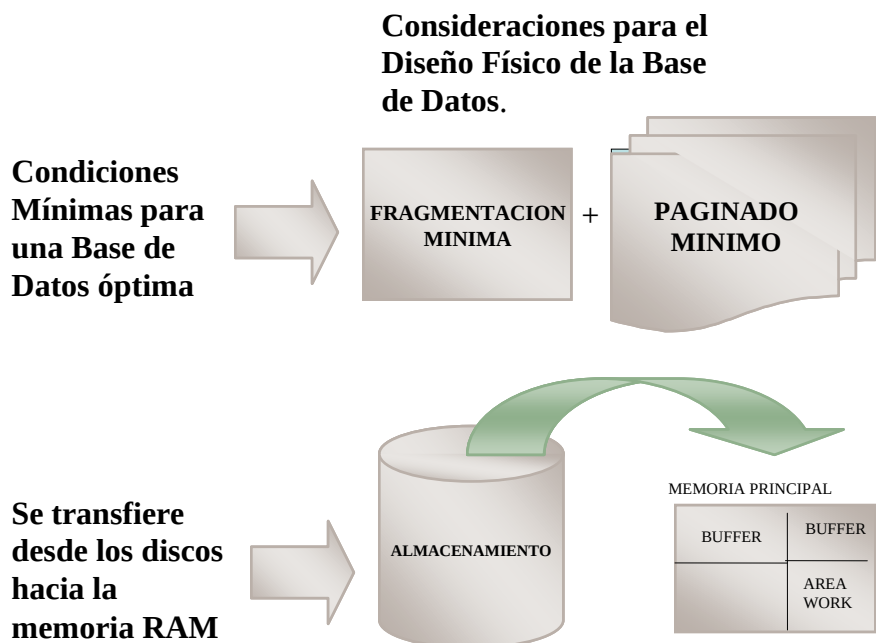
3.1.2.2. Esquema General



En este esquema se demuestra que el tiempo de respuesta depende fundamentalmente del Tiempo de Recuperación de los datos. A su vez el tiempo de recuperación depende de la operación de búsqueda y de transmisión de los discos a memoria.

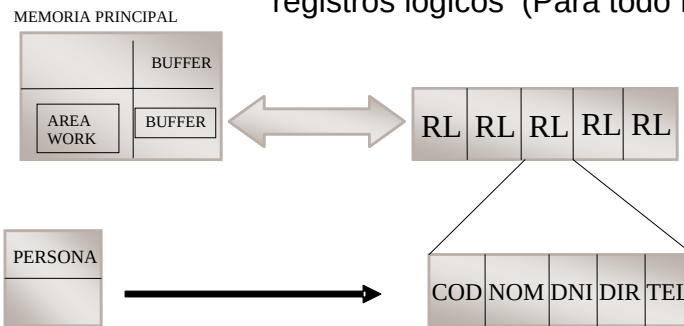
Paginación de Memoria

- Entre los discos y la memoria real, se transfieren datos, ya sea para lectura o grabación.
- Esta transferencia se realiza en bloques de registros lógicos llamados “páginas”.
- Estas páginas tienen una longitud de un número entero de sectores.
- Por tal motivo a las páginas se les conoce como el factor de transferencia de datos, entre la memoria virtual y la real.



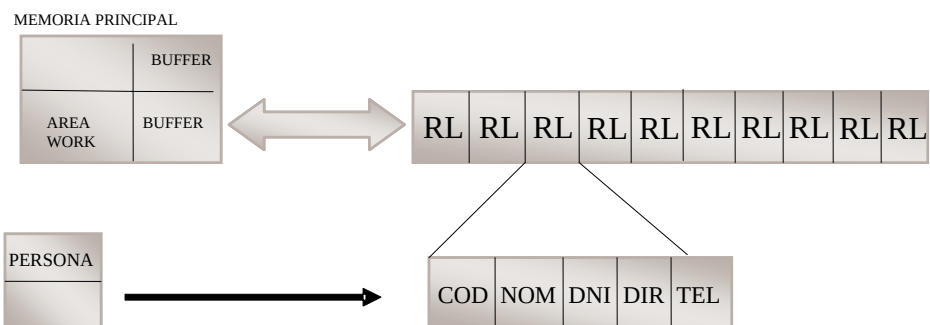
Paginado Mínimo

Transmitir desde los discos hacia la memoria principal de manera que la data fluya con la mayor rapidez posible:
Toda transmisión se realiza por bloques de registros lógicos (Para todo I/O).



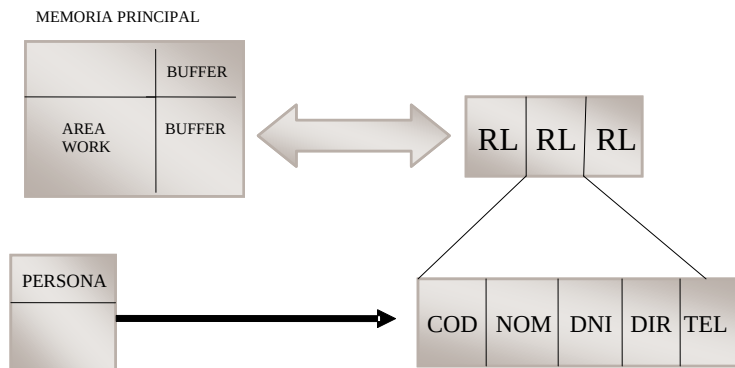
Si el registro solicitado ya se encuentra en memoria ya no será necesario

Luego para procesos por lotes se recomienda el tamaño de página máximo posible, de esta manera el paginado será mínimo.



Para procesos de acceso aleatorio el tamaño de página deberá ser pequeño, de manera que los Buffers se utilicen para más páginas y evitar el trabajo de liberación de buffers consiguiendo así el mínimo paginado.

Para procesos de acceso aleatorio el tamaño de página deberá ser pequeño, de manera que los Buffers se utilicen para más páginas evitando el proceso de liberación de Buffers consiguiendo así el mínimo paginado.



En los módulos transaccionales, se desarrollan procesos de multi-tarea donde con un solo programa se pueden atender n requerimientos de la misma naturaleza.

Por ejemplo en el caso de los bancos las operaciones típicas son las de depósitos y retiro de dinero.

Luego solo es necesario que se cargue el programa de depósitos para atender todas las operaciones de depósitos.

Cada una de estas operaciones para el sistema informático se conoce como tareas. Cada tarea para su ejecución requiere de diferentes recursos uno de ellos son los buffer.

Este se agota cuando es poblado por los registros lógicos traídos desde los discos. Supongamos que las páginas o bloques son de 2k, luego para un buffer de 2 mega bytes se podrán almacenar hasta 1,000 tareas de manera concurrente.

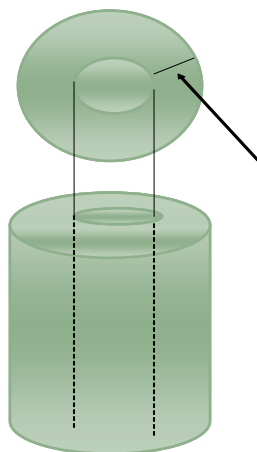
Así cuando se han llegado a ocupar totalmente los buffer y existen tareas pendientes por desarrollarse se creará una cola de tareas a la espera que se liberen estas áreas.

Se concluye que para estos módulos las páginas deberán ser pequeñas, para que el número de tareas concurrentes sea el máximo.

Datafiles y su fraccionamiento

- Los datafiles son áreas de disco destinadas al almacenamiento de datos.
- Estas áreas pueden estar contiguas o separadas.
- Se recomienda ser contiguas (minimiza movimiento del brazo del disco), pero por distribución de las áreas de disco muchas de ellas pueden no serlo.
- Un datafile con áreas físicas no contiguas se conoce como un datafile fraccionado.
- La consecuencia de este fraccionamiento será una pobre performance en su funcionamiento.

Fragmentación mínima



Distancia de recorrido por parte del sistema del disco.

El tiempo de acceso a datos depende de la distancia entre los diferentes cilindros.

Para cilindros contiguos la distancia es óptima.

Para cilindros alejados la performance se degrada.

3.1.2.3. Tipos de organización

Tipos de organización de los datafiles

- Organización secuencial.
- Organización indexada.
- Organización relativa .

Los datos se agrupan en bloques lógicos llamados filas o registros. Así una fila contiene la información referente a una entidad o una relación semántica.

El esquema más sencillo de almacenamiento es conocido como secuencial. Donde los registros se almacenan conforme llegan a la tabla o archivo. La única forma de extraer cierto dato particular de la tabla es leyendo en forma secuencial desde el inicio hasta encontrarlo. Teniendo como número mínimo de lecturas igual a 1 en caso se trate del primero de la tabla y, como número máximo de lecturas igual N (total de páginas), si se trata del último registro de la tabla. Se estima como el número esperado de lecturas es igual a $\frac{1}{2}N$ (cada acceso tiene igual probabilidad de acertar o no es decir 50%).

Para lograr mayor rapidez se crea un índice de esta manera se hacen mínimas las lecturas para encontrar el registro deseado. Los registros ordenados por clave se conocen como indexados. Finalmente se puede utilizar una tabla simple donde para recuperar una fila se requiere conocer su ubicación física al interior de la tabla conocida como organización Relativa.

3.1.2.4. Método de acceso a disco



Sin duda las preguntas típicas acerca del sistema de cómputo son las siguientes:

- ¿Porqué el sistema esta lento?
- ¿Puede ser que la base de datos está muy fragmentada?
- ¿Debería incrementar la frecuencia de re-organización de la base de datos?
- ¿Cuánto más de discos se requerirá?

Estas preguntas y muchas más podrán ser respondidas al término de este capítulo.

Este tema trata los elementos físicos básicos contenidos en todo software encargado de la gestión de la base de datos.

Temas como indexación, transferencia física de datos, archivos de longitud variable, etc.

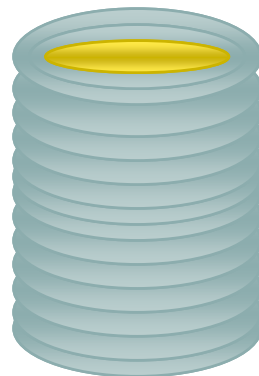
MÉTODO DE ACCESO A DISCOS

- Concepto
- Descripción física
- Intervalos de Control
- Aéreas de control
- Organización Secuencial
- Organización indexada
 - Espacio Libre
 - Índice alternante
 - Generación de Archivo
- Organización relativa

Un enfoque gráfico del almacenamiento en discos:

MÉTODO DE ACCESO A DISCOS

- Concepto: Software encargado de administrar y facilitar el uso de los discos para el tratamiento de archivos.
- Descripción:
 - Almacenamiento físico
 - Genera índices primarios y alternos
 - Maneja catálogos de los datafiles y sus índices
 - Lleva estadísticas de su performance.



INTERVALOS DE CONTROL(IC) (PÁGINAS) (BLOCKS)

Los términos Intervalos de Control, Páginas y Blocks son equivalentes.

Estas son áreas de espacio en disco donde se almacenan las filas o registros lógicos. Estos mismos bloques de registros son transmitidos hacia la memoria RAM en la zona llamada memoria intermedia (buffer).

Luego estos bloques determinan la forma de almacenar y transmitir a la vez.

Estos bloques físicos solo pueden tomar una longitud predefinida por la longitud de la unidad mínima de almacenamiento en disco que actualmente es de un sector (512 bytes). Dado que el sector es la unidad mínima de dirección, se deberá de definir el tamaño del intervalo de control como un número entero de sectores.

Diferente software de gestión de base de datos como el Oracle definen estas áreas como Block y le asigna los posibles tamaños como potencia de 2, así:

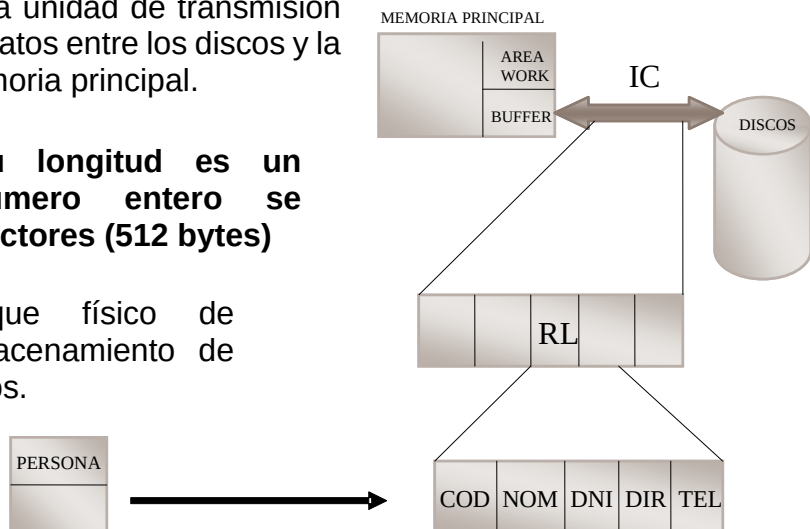
2KB (2048Bytes), 4 KB (4096Bytes), 8KB (8192Bytes), 16KB (16384Bytes), 32KB(32768Bytes).

INTERVALOS DE CONTROL(IC) (PAGINAS)

Es la unidad de transmisión de datos entre los discos y la memoria principal.

Su longitud es un número entero se sectores (512 bytes)

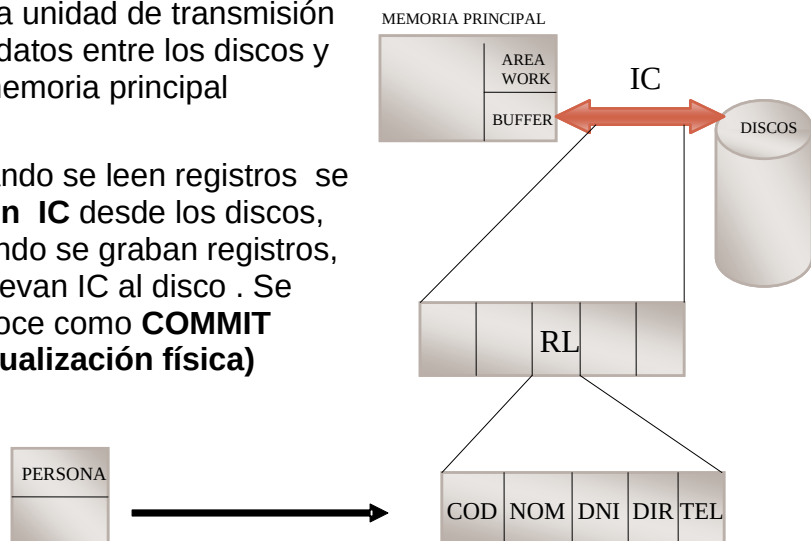
Bloque físico de almacenamiento de datos.



INTERVALOS DE CONTROL(IC) (PAGINAS)

Es la unidad de transmisión de datos entre los discos y la memoria principal

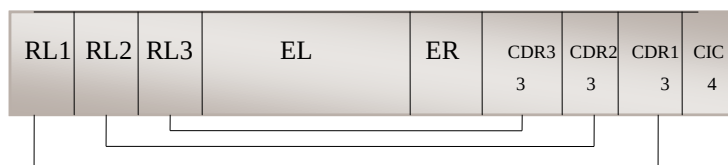
Cuando se leen registros se **traen IC** desde los discos, cuando se graban registros, se llevan IC al disco . Se conoce como **COMMIT (actualización física)**



INTERVALOS DE CONTROL(IC)

Los IC contienen registros lógicos e información de control

ESTRUCTURA DE UN IC:



DONDE:

RL = REGISTRO LOGICO

EL = ESPACIO LIBRE

ER =ESPACIO RESIDUAL

CDR = CAMPO DESCRIPTOR DEL REGISTRO(3 BYTES)

CIC = CAMPO DE IC (4 BYTES)

Los valores de 3 bytes para el CDR y 4 bytes para el CIC son típicos más no son los únicos.

El Intervalo de Control presenta una estructura que considera la capacidad para almacenar registros lógicos, datos de control, espacio libre y espacio residual.

El espacio libre se reserva para registros nuevos o crecimiento del tamaño de los registros que ya existen.

El espacio residual es el sobrante sin poder ser utilizado para nada útil.

Los campos descriptores de registros (CDR) contienen la información de cada registro lógico como por ejemplo su longitud, o la no existencia de uno de sus atributos es decir con valor nulo.

El campo CIC contiene información de control referente al Block de registros lógicos que los contiene.

DETERMINACIÓN DE LONGITUD ÓPTIMA

- COMO TODO DISEÑO SE DEBE TOMAR EN CUENTA LA PERFORMANCE Y LA ECONOMÍA.
- CRITERIO: ECONOMIA → MINIMIZAR ESPACIO RESIDUAL
- Ejemplo: Sea un Registro lógico de 300bytes, luego:

IC	REGLOG	INF.CNTRL	ER
512	1	4 + 3	205
1024	3	4 + 9	111
1536	5	4 + 15	17 ← MÍNIMO
2048	6	4 + 18	226

La formula que permite este cálculo es :
 $(IC - 4) / (RL + 3) = \#RL, R(\text{residuo})$

El cuadro nos indica como varía el espacio residual cuando varía el tamaño del IC.

Para el cálculo del espacio residual se debe descontar el área destinado para los registros lógicos, los CDR, y los bytes del CIC.

Considerando solo los múltiplos de 512 bytes (un sector) se determina una función:
 RESIDUO = F (IC), dejando fija la longitud del registro.

La fórmula $(IC - 4)$ nos da el IC ajustado y, $(RL + 3)$ registro lógico ajustado. De esta división se debe tomar solo la parte entera como cociente luego determinar el residuo.

CRITERIO: OPTIMIZAR PERFORMANCE

- Supuestos:
 - Se desea procesar todo un archivo (cuentas)
 - El archivo contiene 1'000,000 RLs
 - El RL = 200 bytes

IC	RL's x (i/o)	Tiempo
512	2	T hrs.
4096	20	T/10 hrs.

Tiempo
óptimo

Este es un caso típico de un proceso batch, por lo que posee un intervalo de control grande.

CRITERIO: OPTIMIZAR PERFORMANCE

PROCESO	CARACTERISTICA MÁS IMPORTANTE	LONG.
BATCH	Recuperación masiva del RL	Tiende a Max
ON LINE	Recuperación unitaria del RL	Tiende a min

- Se debe tomar en cuenta ambos criterios para asegurar economía y performance.

En este cuadro se determinan las tendencias de las longitudes de acuerdo a la modalidad de proceso.

CRITERIO: OPTIMIZAR AMBOS

Se debe tomar en cuenta ambos criterios para asegurar economía y performance

PROCESO	LONG.	LONG. SUGERIDA	ECONOMIA
BATCH	TIENDE A MAX	4096, 3584, 3072	MIN(ER)
ON LINE	TIENDE A MIN	512, 1024, 1536	MIN(ER)

En el cuadro se observa que la longitud sugerida está dada por una terna de valores, que pueden ser los menores o los mayores típicamente conocidos. De la terna se escogerá el IC con menor espacio residual. En esta decisión se habrán combinado tanto el criterio de economía así como el criterio de rendimiento o performance.

MÉTODO

MOD-PROC	IC-BLOCK
ONLINE	512
ONLINE	1024
ONLINE	1536
BATCH	3072
BATCH	3584
BATCH	4096

La fórmula que permite este cálculo es :
 $(IC - 4) / (RL + 3) = \#RL, r(\text{residuo})$

$(IC - 4) - \#RL(RL + 3) = r (\text{residuo})$

En base a la modalidad se toma el IC
 Correspondiente, donde:

$$0 \leq r < (RL + 3)$$

- 1.- Se calcula “r” para cada IC
- 2.- Se escoge el IC con mínimo “r”.

AREAS DE CONTROL (AC) (FACTOR DE EXPANSIÓN) (EXTENTS)

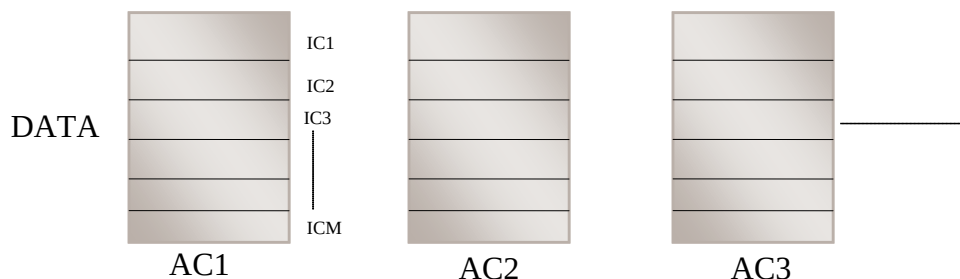
Los términos Área de Control, Extents y Factor de Expansión son similares. Están formados por un grupo de intervalos de control de esta manera son los grandes bloques que constituyen físicamente las tablas, además permiten la expansión física de las mismas.

Cuando una tabla es creada y cargada realiza un crecimiento inicial. Cuando es puesta en producción realiza un crecimiento tanto primario como secundario. Se conoce como primario cuando utiliza el espacio libre del IC. Cuando este espacio se agota se utilizara el espacio libre del AC.

Si el espacio asignado al crecimiento secundario es pequeño y rápidamente es agotado la tabla seguirá creciendo pero en bloques iguales a los de un área de control pero físicamente ubicados en cilindros del disco no necesariamente contiguos.

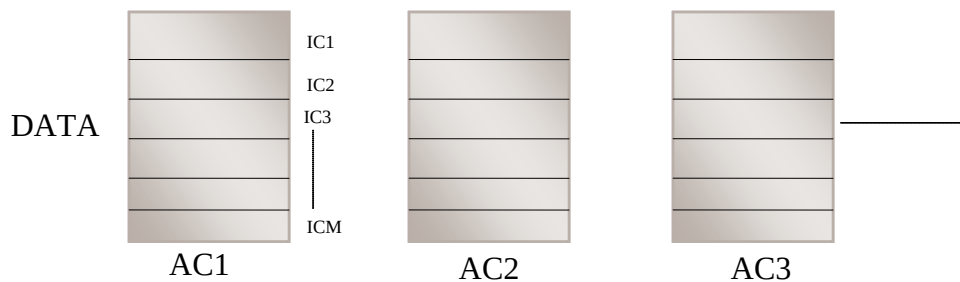
Después de repetirse este crecimiento no planeado configurará una tabla original con muchos bloques distribuidos en diferentes cilindros del disco este esquema es conocido como tabla fragmentada. Los gestores de base datos presentan un parámetro que indica el número máximo de crecimiento secundario, que funciona como tope máximo de fragmentación de la tabla.

AREA DE CONTROL(AC)



- AREA CONFORMADA POR UN CONJUNTO DE IC
- EL CONJUNTO DE AC CONFORMAN EL ARCHIVO FISICO(SEGMENTS)
- ES EL FACTOR DE CRECIMIENTO DEL ARCHIVO
- LA CAPACIDAD DE UN ARCHIVO PUEDE SER DADO EN TERMINOS DE REG.LOG. SIN EMBARGO SON CONVERTIDOS A AC'S.

AREA DE CONTROL(AC)



El crecimiento (extensión) de un archivo es limitado a un número determinado de AC's que depende del espacio disponible en el disco, o del parámetro definido en el método de acceso.

No es recomendable esta forma de crecimiento, porque Genera fragmentación en la base de datos.

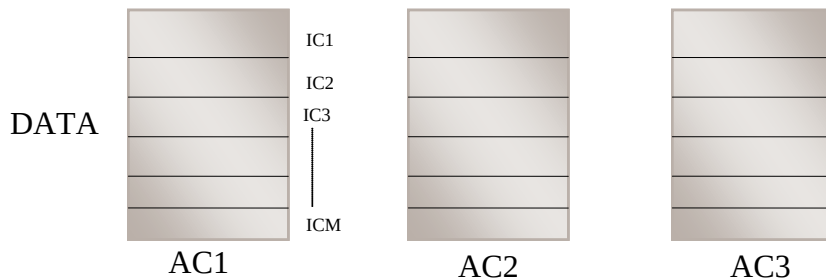
LÍMITES: 1 PISTA $\leq$ AC $\leq$ 1 CILÍNDRIO

Los software gestores de base datos dan la libertad a que estos bloques puedan definirse con una longitud que pueden ser "n" cilindros.

Sin embargo cuando ocurren las rupturas de AC y se desea insertar uno o muchos registros, el espacio en disco que requerirá este proceso podría ser suficiente con un cilindro. Cuando el sistema tenga que crecer lo deberá hacer en un bloque igual al AC y si la longitud definida para el AC es mayor a un cilindro, se deberá ubicar en el disco un tamaño similar, si existe un área igual a la longitud del AC se le asignará, en caso contrario el proceso será cancelado con el mensaje 'NO HAY ESPACIO DISPONIBLE'.

Se puede concluir que es recomendable definir como longitud máxima de un cilindro.

AREA DE CONTROL(AC)



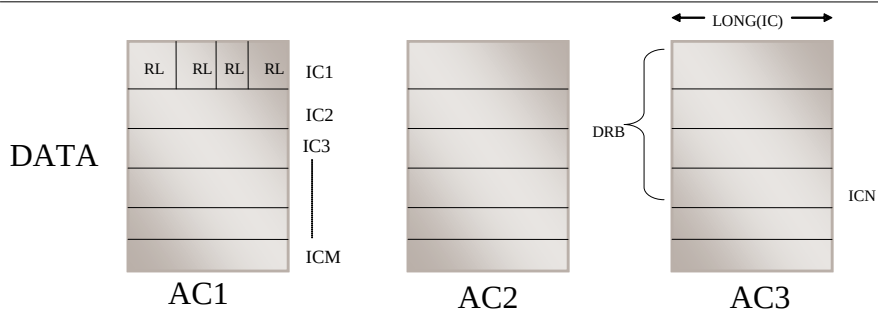
$AC = M \cdot N(RL)$ donde : $M = \text{Nro. de IC} \times AC / N = \text{Nro. RL} \times IC$

Determinación de la longitud de un AC.

Dado que la longitud máxima es un cilindro , se recomienda esta longitud para archivos grandes.

En caso contrario puede ser un número entero de pistas de un cilindro.

LOCALIZACIÓN DE LOS IC



LOCALIZACION DE LOS IC

Tanto para grabar como para leer la data se debe ubicar físicamente los IC's, esta dirección es relativa al inicio del archivo. Esta cantidad de bytes es conocida como DRB (desplazamiento relativo en bytes) es de 3 bytes.

Ejemplo $DRB(ICN) = (N-1) \cdot LONG(IC)$

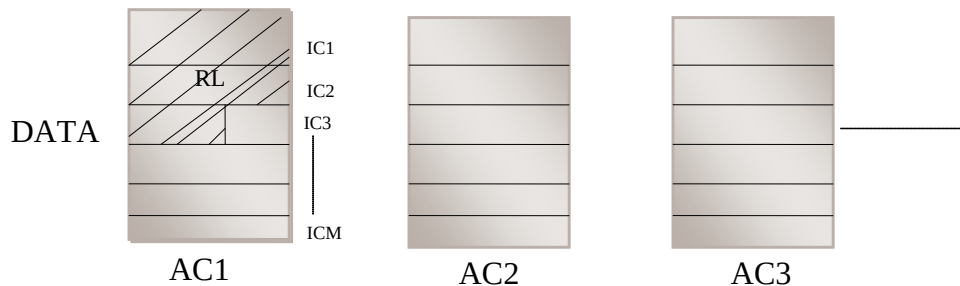
Los IC deben poder ser ubicados físicamente al interior de cada AC.

Las direcciones están compuestas por dos componentes: base + desplazamiento.

La dirección base está dada por la dirección del AC y el desplazamiento por la cantidad de bytes desde el inicio del AC hasta donde se encuentra el IC.

Dir-IC	= Dir-Base + Dir-Desplazamiento
Dir-Base	= Dirección del Cilindro.
Dir-Desplazamiento	= $(n - 1) \text{ Long-IC}$
Donde n	= Lugar que ocupa el IC al interior del AC.

REGISTROS ESPARCIDOS



REGISTROS ESPARCIDOS

Existen registros lógicos de longitud variable, que de manera excepcional pueden tener una longitud de registro que exceda a su IC, pero no podrá exceder as su AC



En el caso de los archivos de longitud variable, estos poseen un componente de longitud fija y otro variable este puede variar desde un tamaño mínimo hasta un valor máximo. Los valores máximos son de muy baja probabilidad que se den, pero en el caso que ocurra podrían llegar a tener una longitud que exceda el tamaño del IC, pudiendo llegar a ocupar más de uno, pero como tope nunca su longitud excederá su AC.

La longitud máxima de un IC es de 32,768 bytes.

3.1.2.5. Tipos de información

Naturaleza de los datos y su forma de Almacenar

Tipos de Información

- I. Información maestra (cuentas)
- II. Información de movimiento (eventos)
- III. Información tabulada (catálogo)
Información de parámetros (catalogo)
Información histórica/DWH

AC=Extensión

IC =Bloque

- Para cada uno de estos tipos de información se recomienda agruparlos en datafiles que posean las mismas características físicas, como su extensión o bloque de crecimiento y su bloque físico

Los componentes físicos para almacenar las tablas son conocidos como tablespace o simplemente espacio para tablas.

En estos elementos se definen los parámetros como la longitud del IC y del AC.

Los Eventos o Movimientos generan instancias si estas son muy numerosas se recomienda que su AC deberá ser de un cilindro.

Para módulos transaccionales su IC deberá de longitud pequeña.

Las tablas históricas deben estar sin cambios es decir “congeladas”, su crecimiento será llevado a cabo en procesos por lotes, durante la noche.

Las tablas maestras, mantienen los saldos. Se deben actualizar vía transaccional solo los saldos comprometidos con los eventos. Cuando se desee adicionar cuentas nuevas estas deberán ser insertadas en modalidad batch.

Estas diferentes características hacen que sea recomendable almacenar en tablespace las tablas que compartan las mismas características físicas.

ESTRUCTURA DE ALMACENAMIENTO FISICO

Data base										
PROD.										
TABLESPACE SYSTEM			USER_DATA				RBS		TEMP	
DATAFILES DISK1/SYS1.dbf			DISK2/ USER1.dbf		DISK3/ USER2.dbf		DISK1/ ROLL1.dbf		DISK1/ TEMP.dbf	
TABLAS			S_DEPT		SE_EMP					
DD Table	DD Index		S_DEPT	SE_EMP						
Data Seg	Index Seg	RB Seg	Data Seg	Data Seg						
EXTENTS										
1212121222			FREE		1122					
DATABLOCKS										

Este gráfico representa el mapa de distribución de componentes de una base de datos. Se puede apreciar desde el nivel más bajo que el conjunto de datablocks determinan los Extents.

El conjunto de Extents determina las tablas o segmentos de la base de datos.

La jerarquía es la siguiente:

{ATRIBUTOS}	= RL
{RL}	= IC
{IC}	= AC
{AC}	= SEGMENTO (TABLA)
{TABLAS}	= DATAFILE

Tipos de organización de los datafiles

- Organización secuencial
- Organización indexada
- Organización relativa

Los datos están agrupados formando los registros lógicos y, estos a su vez se organizan de tres formas distintas para fines definidos.

Las tres maneras de organizarse son:

Secuencial. - Se graban físicamente en secuencia de llegada, luego para extraer datos se realizará la búsqueda igualmente de forma secuencial. No permite acceder en forma aleatoria.

Indexada. - Se graban físicamente en secuencia de clave, luego para extraer datos se realizará la búsqueda de manera indexada, lo cual el tiempo de búsqueda será el mínimo posible.

Relativa. - Se graban físicamente en posiciones fijas del archivo.

Esta posición se conoce como la Posición Relativa del Registro (PRR).

Esta posición se indica mediante un número ordinal conocido como puntero.

3.1.2.6. Organización secuencial

Organización secuencial

Este tipo de organización es uno de los más sencillos, pero muy usado sobre todo para procesos batch.

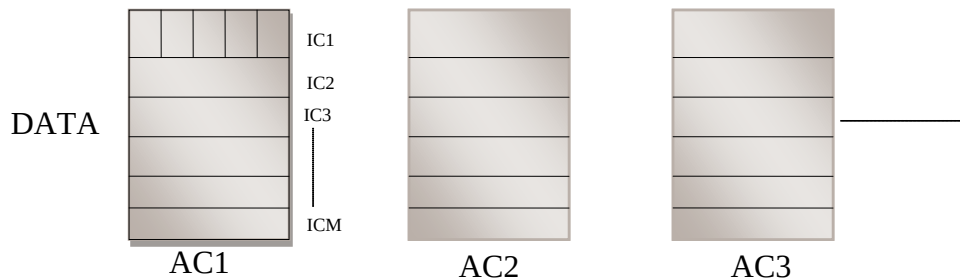
Considerando que los procesos batch, tratan grandes cantidades de registros de manera íntegra (desde el primero hasta el último), luego este tipo organización es la indicada para el proceso de grandes lotes de información.

El hecho de no poseer índices, su estructura se considera “plana”.

Para evitar muchos accesos al disco se sugiere que su IC sea grande como (4096, 8192, 16384 o, 32768).

Para que en cada operación de E/S se transfieran grandes cantidades de registros lógicos. Algunos les llaman con el esquema FIFO, de las siglas en inglés donde primero en entrar, primero en salir.

ORGANIZACIÓN SECUENCIAL



Los registros se almacenan en la misma secuencia de su grabación
La recuperación es secuencial.

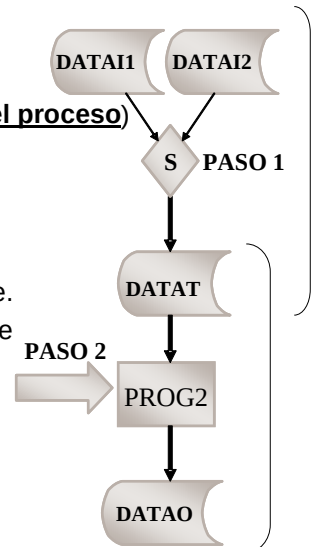
Los registros lógicos se almacenan en IC y estos en AC

Todo registro nuevo se agrega al final del archivo.

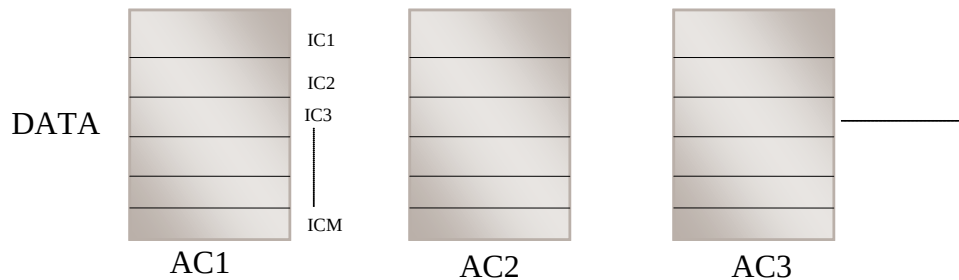
Usos: Archivos temporales, archivos logging (bitácora), tramas, etc.

ARCHIVOS TEMPORALES

- PROCESO: BATCH (GRANDES LOTES)
- DATAI=DATOS ENTRADA
- DATAT= DATOS REUNIDOS (**Existen durante el proceso**)
- Este archivo es el central donde se concentra el proceso.
- Normalmente son leídos de forma secuencial (no posee clave) e íntegramente.
- Por lo tanto su IC debe ser lo más grande posible.
- Al existir sólo durante el proceso su espacio libre es cero.
- En este modelo las entradas son lotes de los registros que provienen de otro módulo a modo interface.
- DATAO= RESULTADOS DEL PROCESO



ORGANIZACIÓN SECUENCIAL



Parámetros mínimos para la definición

NOMBRE	= PARA EL CATALOGO
ORGANIZACIÓN	= SECUENCIAL
LONG.REG	= (MAX, MODA)
IC	= m512 (Se sugiere el máximo)
CAPACIDAD	= (N , AC) N= Cantidad estimada total AC= Factor de expansión.

Indicando que la organización es Secuencial esto indicará no generar índice.

La longitud de registro puede variar por varias causas:

- 1.- La estructura del registro es variable de manera expresa.
- 2.- Pueden estar presentes campos nulos, los cuales no ocupan espacio.
- 3.- Los atributos VARCHAR permite tomar diferentes longitudes desde mínimo hasta un máximo determinado. Por tal motivo se deben especificar dos parámetros tales como la longitud máxima y la longitud más típica o moda.

Para su IC se recomienda la máxima posible, dado que este tipo de archivos son usados normalmente para procesos batch.

3.1.2.7. Organización indexada

La organización Indexada a diferencia de la Secuencial permite acceder a la tabla de manera aleatoria. Vale decir acceder a cualquier registro dependiendo de la clave de la entidad.

En este tipo de organizaciones existen dos componentes:

1.- Data

2.- Índice

El componente data posee los registros en secuencia de clave. Puede ser de mayor a menor o viceversa.

El componente índice está compuesto por las claves de las entidades. Por motivos de optimización los registros son agrupados en bloques (IC) y solo la clave mayor de este grupo se encuentra en el índice. Esta manera de organizar los índices se conoce como índice denso. El efecto de este esquema es que por cualquiera de las claves de los registros en el mismo IC, se transfiere el bloque a memoria.

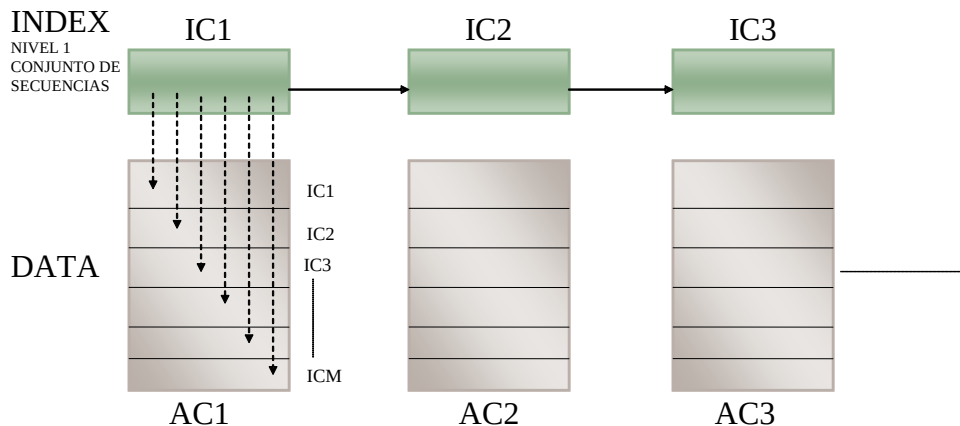
ORGANIZACIÓN INDEXADA

Los registros poseen clave

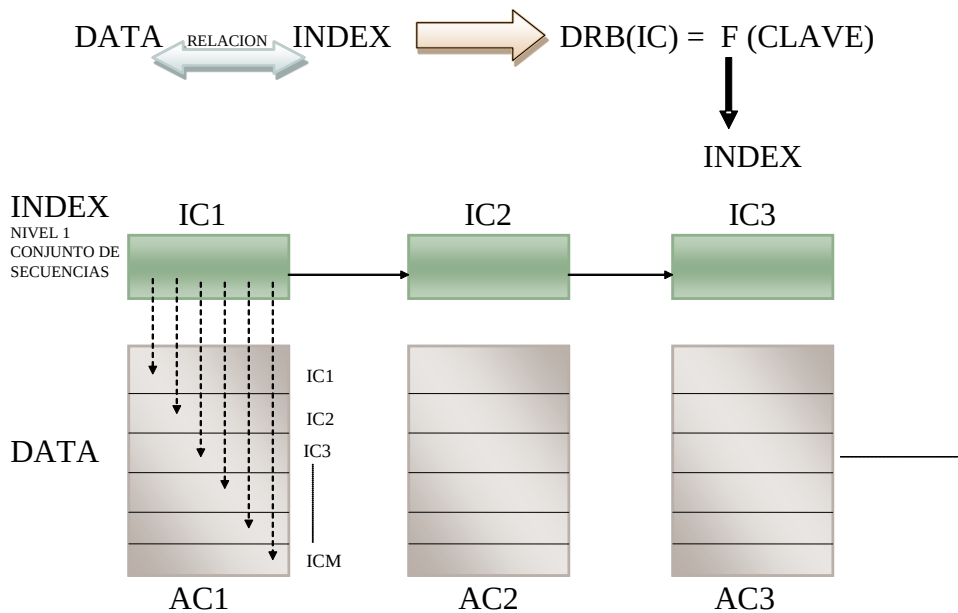
Registros lógicos organizados en secuencia de clave

Permiten recuperación acceso aleatorio.

Componentes: DATA + INDEX



ORGANIZACIÓN INDEXADA



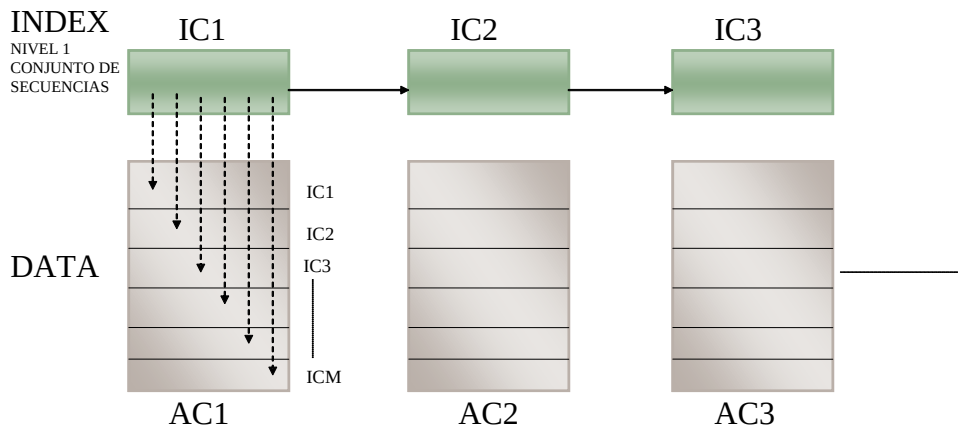
Se puede apreciar la relación entre el componente Data e Índice. Un IC de índice corresponde a un AC de la data. Cada puntero del índice apunta a un IC de la data. Los IC del índice además se relacionan vía punteros para una recuperación secuencial.

ORGANIZACIÓN INDEXADA

DATA: Similar a la Organización secuencial

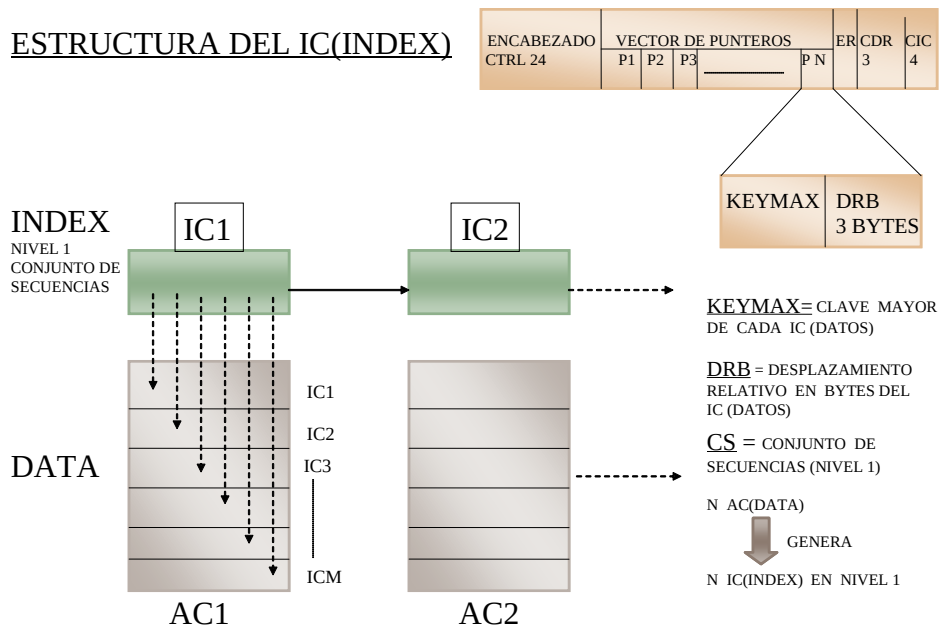
INDEX: ARCHIVO = $\{ AC = \{ IC \} \}$

PARA TODA $AC(\text{DATA}) \longrightarrow IC(\text{INDEX})$



ORGANIZACIÓN INDEXADA

ESTRUCTURA DEL IC(INDEX)



En este esquema se aprecia la estructura del componente índice:

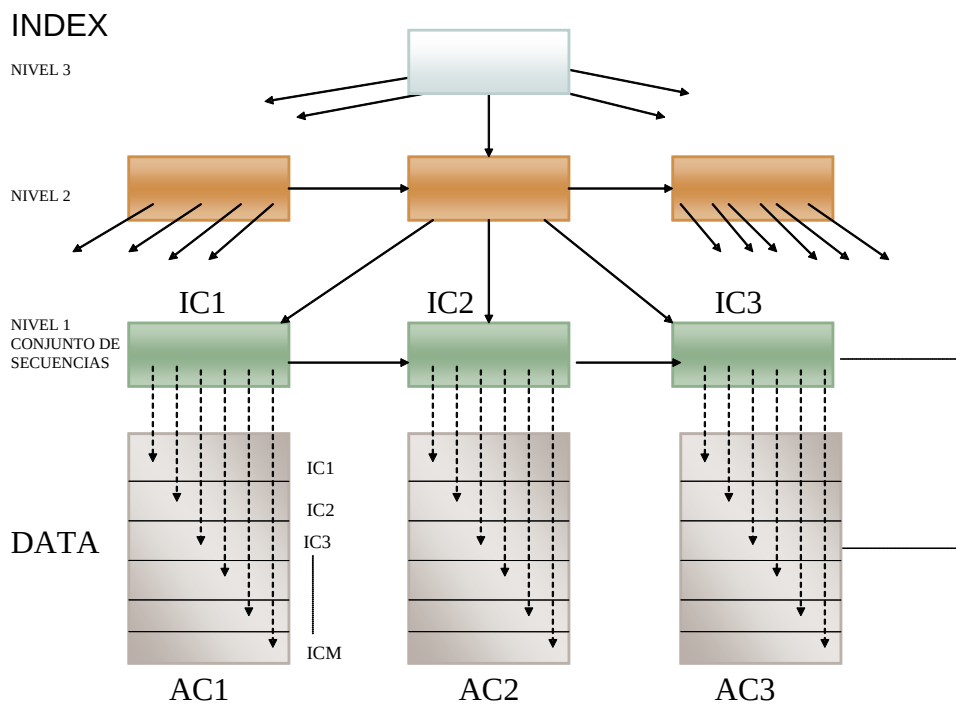
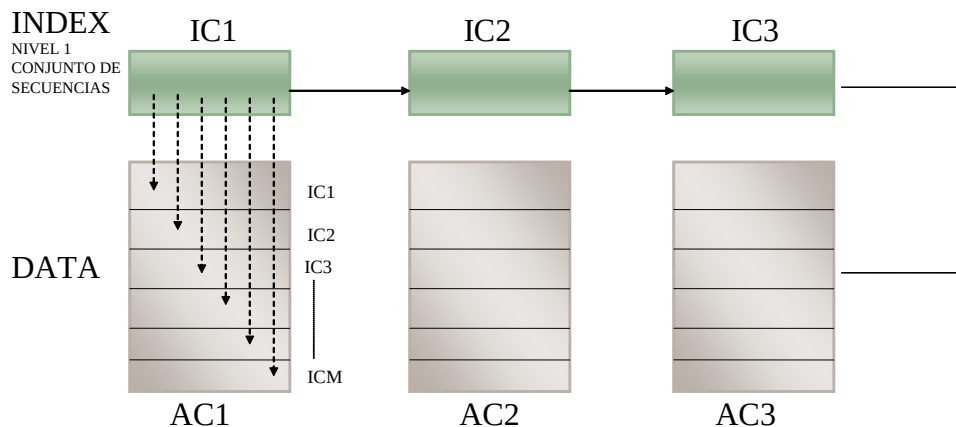
Cada puntero consta de la Clave mayor de cada IC y, su dirección física dada por el DRB. Dado se requieren un puntero por cada IC de la data, luego serán necesario por lo menos el mismo número de IC para cada AC de la data.

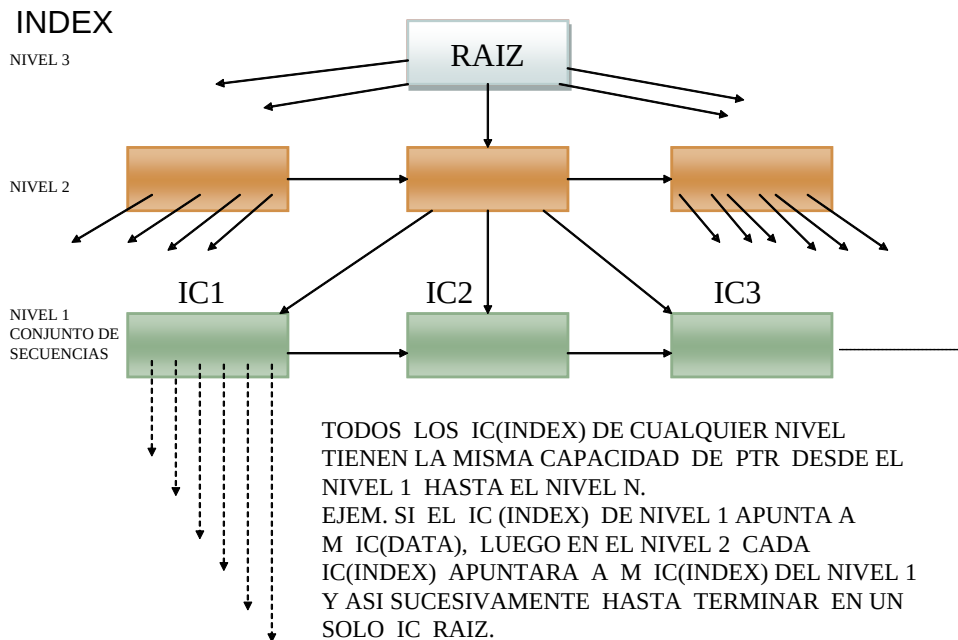
ORGANIZACIÓN INDEXADA

SI CADA AC(DATA) TIENE M IC

ENT CADA IC(INDEX) TIENE M PTR(MÍNIMO)

LUEGO TODO IC(INDEX) PUEDE DIRECCIONAR(MÍNIMO) A M IC's





Se puede observar la estructura del árbol del componente índice.
Al primer nivel del índice también se le conoce como conjunto de secuencias.

El componente índice puede ser considerado como un archivo físico más, pues está formado por AC y estas contienen IC.

Todos los IC poseen la misma estructura y, la misma cantidad de punteros sin importar el nivel del árbol.

Este esquema ha demostrado tener gran capacidad de direccionamiento siendo suficiente tres niveles para cubrir tablas que poseen muchos millones de registros.

El modelo no tiene límite para el número de niveles del árbol.

Normalmente el componente índice es muy compacto inclusive utiliza algoritmos para compactar la clave contenida en los punteros. Esta característica permite que este componente pueda ser cargado en memoria antes de hincar a operar la tabla, con lo cual asegura un tiempo de búsqueda mínimo.

Además antes de realizar la búsqueda en el disco se verifica que no se encuentre en memoria.

Cuando se trata de operaciones de actualización se requiere que el bloque que contiene al registro a ser actualizado deba ser tomado de manera exclusiva por la tarea.

PLANEAMIENTO DE LA CAPACIDAD FÍSICA DE LA BASE DE DATOS

La base de datos como cualquier otra herramienta está diseñada para funcionar de manera óptima para un periodo de tiempo, conocido como tiempo de vida útil. Al final de este periodo se debe regenerar la base de datos para volver a tenerla como en el estado original.

Este tiempo de vida útil se determina en base a las expectativas del negocio, vale decir si se planea un crecimiento agresivo la base de datos deberá estar preparada para este crecimiento de manera planificada.

Con esta planificación se evita la extensión de la tabla, que como se ha demostrado determinará la fragmentación física de la base. Por mediante la planificación se logra reservar espacio libre para uso futuro, de manera que el crecimiento se realizará controlada y ordenada, asegurando una buena performance.

El parámetro clave para este objetivo es el llamado Espacio Libre.

Este parámetro se debe definir antes del momento de la carga física.

Tiene dos valores: %IC y %AC, es decir cuánto de porcentaje en el IC habrá que dejar libre, de igual manera para el AC.

ESPACIO LIBRE

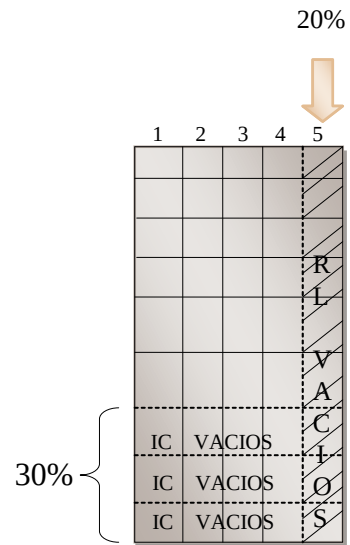
- ESTADOS DE UN ARCHIVO
 - CARGA INICIAL
 - MANTENIMIENTO
 - ALTA
 - BAJA
 - CAMBIO

SE DEBE DEJAR ESPLIB PARA FUTUROS REG.
A SER GRABADOS

EL ESPLIB SE INDICA EN LA DEFINICION DEL ARCHIVO MEDIANTE UN PORCENTAJE PARA EL IC Y AC.

EJEM. ESPLIB (20 , 30)
SIGNIFICA QUE EN CADA IC SE DEJARA EL
20% LIBRE EN EL MOMENTO DE LA CARGA,
DE IGUAL MANERA 30 % PARA EL AC

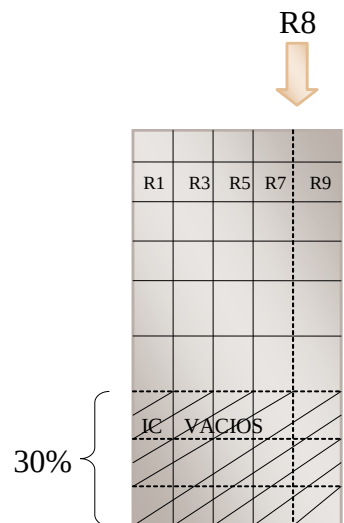
EL ESPACIO LIBRE SE USARA CUANDO EL ARCHIVO ESTE EN PRODUCCION PARA LAS INSERCCIONES.



EN EL MOMENTO DE LA CARGA
SOLO SE USARAN 24 RL.

- **RUPTURA DE AC.-**
- SE PRODUCE CUANDO EL AC LLENO SU ESPLIB Y SE DESEA ADICIONAR UN NUEVO REG.
- SE PROCEDE DE LA SGTE. MANERA
 - 1.-SE OBTIENE OTRA AC VACIA
 - 2.- SE REMUEVE LA MITAD DEL AC LLENA HACIA EL AC VACIA
 - 3.- LUEGO SE ADICIONA EL REG. NUEVO
 - 4.- SE ACTUALIZA EL INDEX

TODOS ESTOS PROCESOS SON AUTOMÁTICOS,
PERO CONSUMEN TIEMPO DE PROCESO
POR NO ESTAR NECESARIAMENTE EN EL
MISMO CILINDRO LA DEGRADACIÓN DE
LA PERFORMANCE ES CRÍTICA.
EL DISCO TIENDE A FRAGMENTARSE.



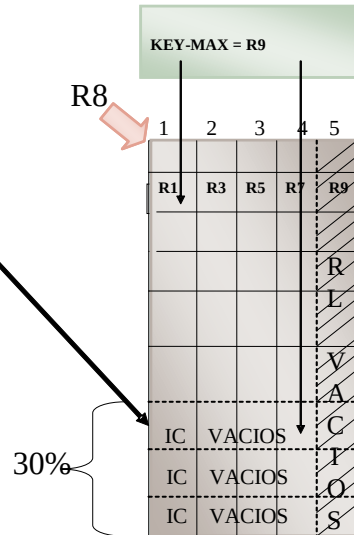
EN EL MOMENTO DE LA CARGA
SOLO SE USARAN 24 RL.

A continuación se detalla cada uno de los pasos mencionados. Describiendo los cambios físicos que se dan en el Área de Control.

ESPACIO LIBRE

- RUPTURA DE IC.-
- SE PRODUCE CUANDO EL IC HA LLENADO SU ESPLIB Y SE DESEA ADICIONAR UN NUEVO REG. LOG.
- SE PROCEDE DE LA SGTE. MANERA
 - 1.-SE OBTIENE OTRO IC VACIO
 - 2.- SE REMUEVE LA MITAD DEL IC LLENO HACIA EL IC VACIO
 - 3.- LUEGO SE ADICIONA EL REG. NUEVO
 - 4.- SE ACTUALIZA EL INDEX

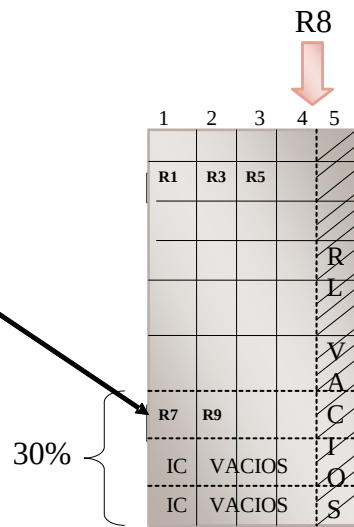
TODO ESTE PROCESO ES AUTOMATICO, PERO CONSUME TIEMPO DE PROCESO POR ESTAR EN EL MISMO CILINDRO LA DEGRADACION DE LA PERFORMANCE ES CONTROLADA.



EN EL MOMENTO DE LA CARGA SOLO SE USARAN 24 RL.

- RUPTURA DE IC.-
- SE PRODUCE CUANDO EL IC HA LLENADO SU ESPLIB Y SE DESEA ADICIONAR UN NUEVO REG. LOG.
- SE PROCEDE DE LA SGTE. MANERA
 - 1.-SE OBTIENE OTRO IC VACIO
 - 2.- SE REMUEVE LA MITAD DEL IC LLENO HACIA EL IC VACIO
 - 3.- LUEGO SE ADICIONA EL REG. NUEVO
 - 4.- SE ACTUALIZA EL INDEX

TODO ESTE PROCESO ES AUTOMATICO, PERO CONSUME TIEMPO DE PROCESO POR ESTAR EN EL MISMO CILINDRO LA DEGRADACION DE LA PERFORMANCE ES CONTROLADA.

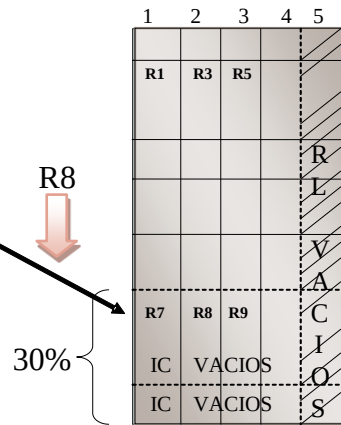


EN EL MOMENTO DE LA CARGA SOLO SE USARAN 24 RL.

ESPACIO LIBRE

- RUPTURA DE IC.-
- SE PRODUCE CUANDO EL IC HA LLENADO SU ESPLIB Y SE DESEA ADICIONAR UN NUEVO REG. LOG.
- SE PROCEDE DE LA SGTE. MANERA
 - 1.-SE OBTIENE OTRO IC VACIO
 - 2.- SE REMUEVE LA MITAD DEL IC LLENO HACIA EL IC VACIO
 - **3.- LUEGO SE ADICIONA EL REG. NUEVO**
 - 4.- SE ACTUALIZA EL INDEX

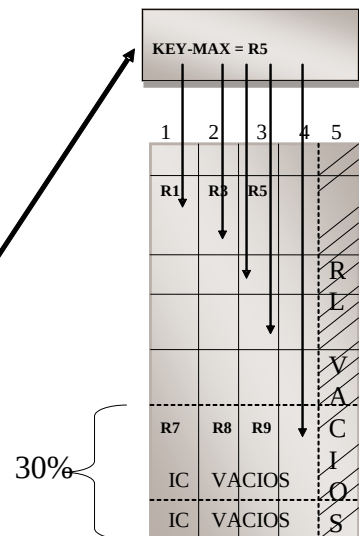
TODO ESTE PROCESO ES AUTOMATICO, PERO CONSUME TIEMPO DE PROCESO POR ESTAR EN EL MISMO CILINDRO LA DEGRADACION DE LA PERFORMANCE ES CONTROLADA.



EN EL MOMENTO DE LA CARGA SOLO SE USARAN 24 RL.

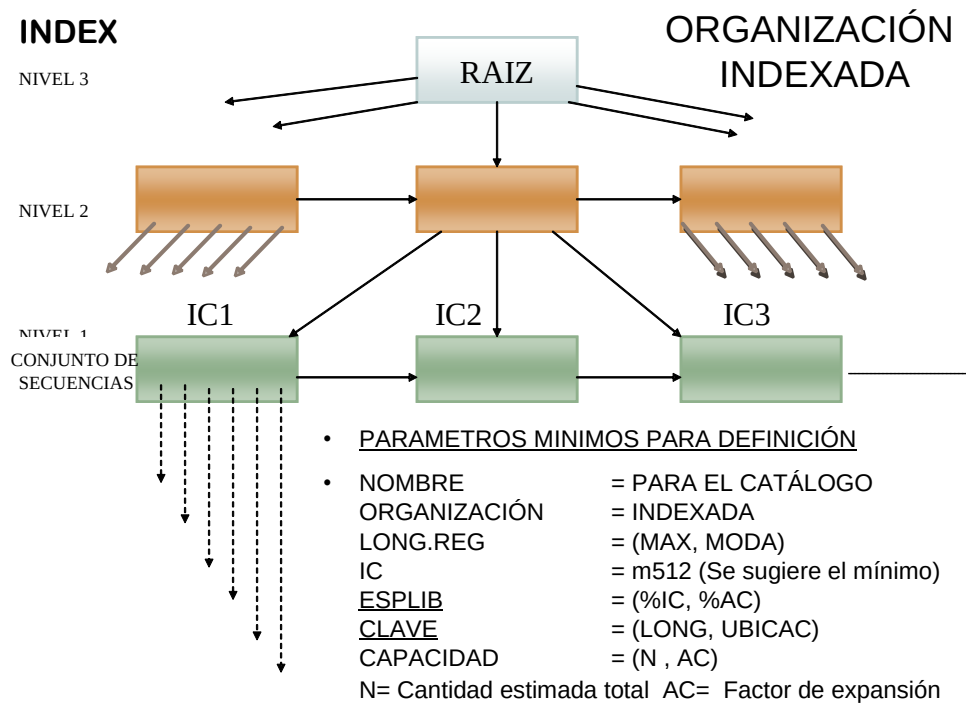
- RUPTURA DE IC.-
- SE PRODUCE CUANDO EL IC HA LLENADO SU ESPLIB Y SE DESEA ADICIONAR UN NUEVO REG. LOG.
- SE PROCEDE DE LA SGTE. MANERA
 - 1.-SE OBTIENE OTRO IC VACIO
 - 2.- SE REMUEVE LA MITAD DEL IC LLENO HACIA EL IC VACIO
 - 3.- LUEGO SE ADICIONA EL REG. NUEVO
 - **4.- SE ACTUALIZA EL INDEX**

TODO ESTE PROCESO ES AUTOMATICO, PERO CONSUME TIEMPO DE PROCESO POR ESTAR EN EL MISMO CILINDRO LA DEGRADACION DE LA PERFORMANCE ES CONTROLADA.



EN EL MOMENTO DE LA CARGA SOLO SE USARAN 24 RL.

La actualización del índice consiste en actualizar las claves para los punteros asociados a los IC. Los IC vacíos siempre tienen su puntero asociado, después que los IC almacenan registros, la clave mayor de estos se almacenará junto a su DRB.



Los parámetros para definir una tabla indexada, son similares a los de la tabla secuencial.

Los parámetros nuevos son:

- **Clave.**- Se indica mediante el o los atributos que conforman la clave. Físicamente se traduce en las coordenadas que indican la posición como:

- ubicación esta se indica mediante el desplazamiento en byte del inicio del registro.
- longitud se indica mediante cantidad de bytes.

- **Espacio Libre.**- Debe fijarse los valores tanto para el %IC como para %AC, de manera que la tabla pueda crecer sin generar fragmentación.

Para ilustrar la metodología de fijación de estos parámetros, se desarrolla como ejemplo el caso de la consulta crítica bancaria.

Desarrollo de caso: Consulta Crítica Bancaria.

CONSULTA CRÍTICA

Ejemplo: Se desea definir las características físicas de un archivo maestro de clientes de un banco con aproximadamente 500,000 de registros lógicos y desea aumentar su clientela a 1'000,000 en los próximos 6 meses (Cada semestre generado).

El diseño de registro lógico es el siguiente:

REG-CLTE:

TIPDOC	CHAR(02)
NRODOC	CHAR(08)
FECNAC	NUM(08)
NOMBRE	CHAR(35)
DIRECC	CHAR(35)
TELEF	NUM(10)
<u>NROCTAS</u>	NUM(02)
CUENTAS	Vector variable de 1 a 99 elementos
	DEPENDEN <u>NROCTAS</u>
NROCTA	CHAR(10)
TIPO	CHAR(02)
SALDO	NUM(13)

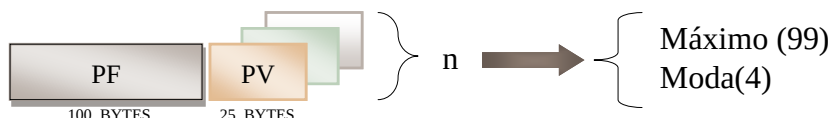
NOTA: Este es un caso de archivo de longitud variable, donde se presenta una parte fija de 100 bytes y otra variable de 25 bytes.

Solución del Caso:

ORGANIZACIÓN INDEXADA

CONSIDERACIONES SOBRE SUS PARAMETROS

LONGREG (MAX, MODA) sobre todo para archivos de longitud variable bajo el formato: $RL = PF + n \cdot PV$



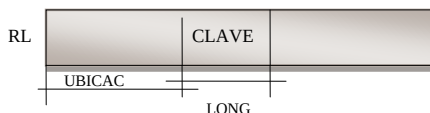
- PARAMETROS MINIMOS PARA DEFINICION
 - NOMBRE = **MAESTRO-CLIENTES**
 - ORGANIZACIÓN = **INDEXADA**
 - LONG.REG = **(2575, 200)**
 - IC = m512 (Se sugiere el mínimo) ESPLIB = (%IC, %AC)
 - CLAVE = (LONG, UBICAC)
 - CAPACIDAD = (N , AC)
- N= Cantidad estimada total AC= Factor de expansión

La moda debe calcularse en base a estadística, caso contrario de manera heurística

CONSIDERACIONES SOBRE SUS PARAMETROS

IC : PARA ACCESO ALEATORIO BUSCAR MINIMO
(TOMAR COMO REFERENCIA LA MODA DEL RL.)

CLAVE (LONG, UBICAC)
(DESPLAZMTO)



- PARAMETROS MINIMOS PARA DEFINICION
 - NOMBRE = **MAESTRO-CLIENTES**
 - ORGANIZACIÓN = **INDEXADA**
 - LONG.REG = **(2575,200)**
 - IC = **1024** (Se escoge el mínimo) ESPLIB = (%IC, %AC)
 - CLAVE = **(10, 0)**
 - CAPACIDAD = (N , AC)
- N= Cantidad estimada total AC= Factor de expansión
- | IC | RL | ER |
|------|----|--------|
| 512 | 2 | 102 |
| 1024 | 5 | 5(min) |
| 1536 | 7 | 111 |

ORGANIZACIÓN INDEXADA

CONSIDERACIONES SOBRE SUS PARAMETROS

CAPACIDAD :

N = Total registros en el siguiente periodo de vida útil 1'000,000

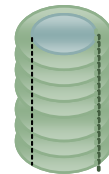
AC Bloque de almacenamiento físico y Factor de crecimiento

Para archivos voluminosos se recomienda usar un cilindro.

Para el ejemplo se usará un cilindro de $19*3*4096$ bytes esta capacidad se deberá expresar en número de registros.

Luego $1 \text{ cil} = 19*3*4096$ $1 \text{ cil} = 19*12 \text{ IC's}$ y $1 \text{ cil} = 19*12*5 \text{ RL's}$

- PARAMETROS MINIMOS PARA DEFINICION
- NOMBRE = **MAESTRO-CLIENTES**
ORGANIZACIÓN = **INDEXADA**
- LONG.REG = **(2575,200)**
IC = **1024** (Se sugiere el mínimo)
ESPLIB = **(%IC, %AC)**
- CLAVE = **(10, 0)**
- CAPACIDAD = **(N=1000000 , AC=19*12*5)**
N= Cantidad estimada total AC= Factor de expansión



AC

CALCULO DEL ESPACIO LIBRE

Toda base de datos se genera para un tiempo de vida útil después de este periodo deberá ser sometido a una reorganización periódica (mantenimiento)

En este tiempo de vida útil el archivo físico ha agotado el espacio libre que originalmente se le otorgó

Si la cantidad actual de RL es N y se planifica un crecimiento hacia N_0 ($N \rightarrow N_0$) luego el porcentaje total de espacio libre será :

$$\% \text{ LIBRE} = (N_0 - N) / N_0$$

ESTE PORCENTAJE TOTAL DEBERA SER

REPARTIDO ENTRE EL
PORCENTAJE PARA EL IC Y AC.

- PARAMETROS MINIMOS PARA DEFINICION
- NOMBRE = **MAESTRO-CLIENTES**
ORGANIZACIÓN = **INDEXADA**
LONG.REG = **(2575, 200)**
IC = **1024** (Se sugiere el mínimo)
ESPLIB = **(%IC, %AC)**
- CLAVE = **(10, 0)**
- CAPACIDAD = **(N=1000000 , AC=19*12*5)**
N= Cantidad estimada total AC= Factor de expansión

CALCULO DEL ESPACIO LIBRE

% LIBRE = $(N_0 - N) / N_0$ ESTE PORCENTAJE TOTAL DEBERA SER REPARTIDO ENTRE EL PORCENTAJE PARA EL IC Y AC.

EL %IC SE RESERVA PARA PREVEER ADICION DE REGISTRO NUEVOS EN EL INTERVALO DE CONTROL, TAMBIEN PARA QUE ALGUNOS REGISTROS PUEDAN CRECER DE LONGITUD.

EL %AC SE RESERVA IC LIBRES PARA SER USADOS CUANDO EL IC CORRESPONDIENTE ESTE LLENO Y NECESITE ADICIONAR UN RL MÁS.

$$\% IC + \% AC = (N_0 - N) / N_0$$

- PARAMETROS MINIMOS PARA DEFINICION
- NOMBRE = **MAESTRO-CLIENTES**
- ORGANIZACIÓN = **INDEXADA**
- LONG.REG = **(2575, 200)**
- IC = **1024** (Se sugiere el mínimo) ESPLIB = **(%IC, %AC)**
- CLAVE = **(10, 0)**
- CAPACIDAD = **(N=1000000 , AC=19*12*5)**
- N= Cantidad estimada total AC= Factor de expansión

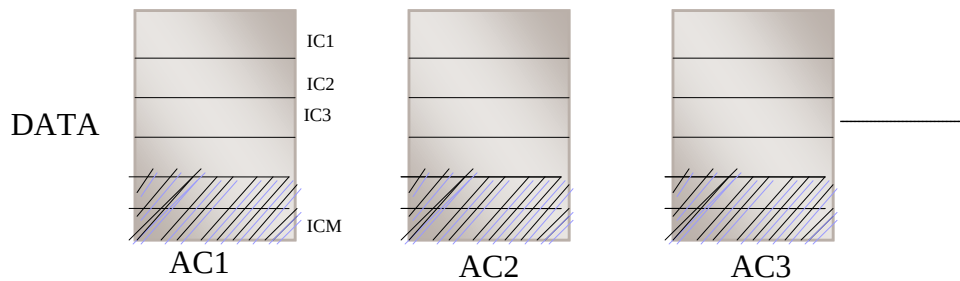
Análisis de valores posibles:

ESPLIB (IC% , AC%)

- Caso-1.- ESPLIB(0,0).- Significa que no reserva espacio futuro. El datafile no crecerá.
- Cada expansión se realiza regenerando archivos.
- Usos.-
 - Archivos de Consulta
 - Tablas Históricas

ESPLIB (IC% , AC%)

- Caso-2.- ESPLIB(0 , AC)

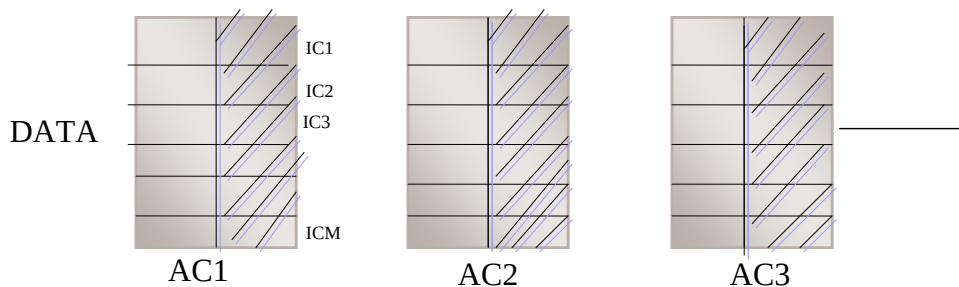


Todo el espacio libre será asignado para el Área de Control.

La consecuencia de este valor será que desde la primera necesidad de insertar un registro nuevo se producirá Ruptura de Intervalo de Control.

ESPLIB (% IC , %AC)

- Caso-3.- ESPLIB(IC , 0)



Por el contrario, en este caso todo el espacio libre será asignado para el IC.

La consecuencia de esta asignación será de los IC podrán insertar sin problema al inicio. Sin embargo cuando agoten su espacio asignado y requieran seguir insertando producirán ruptura de AC lo cual genera fragmentación en la base de datos.

ESPLIB (% IC , %AC)

- Conclusión
- El %IC asegura inserciones sin problemas hasta que este se agote.
- El %AC asegura la performance cuando el %IC se agotó (medida de emergencia).
- Luego el %IC indica cuantos RL nuevos pueden insertarse OK.
- El %AC indica para cuantos IC llenos está preparado el gestor de BD.

CÁLCULO DEL ESPACIO LIBRE

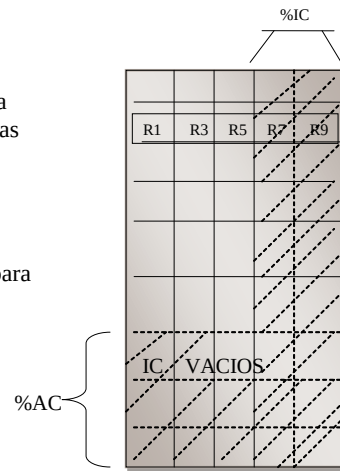
$$\%IC + \%AC = (N_o - N) / N_o$$

Dado que existen 2 variables (%IC y %AC) y una sola ecuación se deberá de fijar el valor de una de ellas para determinar la otra.

Considerando que el espacio libre a nivel de IC es el mas crítico fijaremos este porcentaje y mediante la ecuación se hallará el %AC.

El porcentaje a ser fijado típicamente es el necesario para que pueda realizar inserciones en los IC evitando RUPTURAS de IC.

- **PARAMETROS MINIMOS PARA DEFINICION**
 - NOMBRE = **MAESTRO-CLIENTES**
 - ORGANIZACIÓN = **INDEXADA**
 - LONG.REG = **(2575, 200)**
 - IC = **1024** (Se sugiere el mínimo)
 - ESPLIB = **(%IC, %AC)**
 - CLAVE = **(10, 0)**
 - CAPACIDAD = **(N=1000000, AC=19*12*5)**
- N= Cantidad estimada total AC= Factor de expansión.



CÁLCULO DEL ESPACIO LIBRE

EJEMPLO: Sea $N = 500,000$ RL $\rightarrow N_o = 1'000,000$ RL

RL = 200 bytes $\rightarrow$ IC = 1024 bytes AC = 1 Cil
IC = 5 RL

1 Cil = $(20 - 1) * 3 * 4096$ bytes

1 Cil = $19 * 3 * 2 * 4 * 512$ bytes

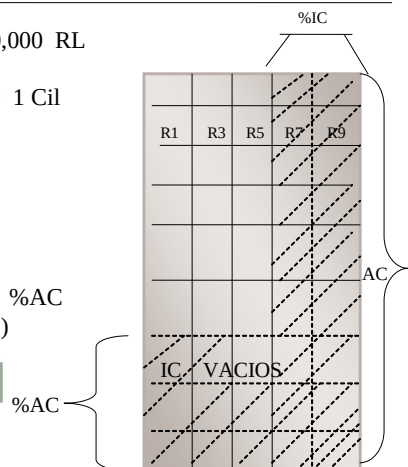
1 Cil = $19 * 3 * 4$ IC

1 Cil = $19 * 3 * 4 * 5$ RL

Luego: $(1'000,000 - 500,000) / 1'000,000 = \%IC + \%AC$

$\%IC + \%AC = 50\%$ (Se convertirá a RL)

COMO FIJAR % IC = RL LIBRES EN CADA IC)



SE REALIZA EL SGTE. ANALISIS:

COMO CADA RL EQUIVALE AL 20%

LUEGO TENEMOS LOS SGTES. VALORE:

1.- 00% SE ELIMINA POR TEORIA

2.- 20% SOLO PERMITE ADICIONAR UN RL

3.- 40% PERMITE ADICIONAR DOS RL

ENTRE LAS OPCIONES 2 Y 3 SE ESCOJERA LA OPCION 3 POR PERMITIR HASTA DOS INSERCCIONES ANTES DE PRODUCIRSE UN SPLIT DE IC.

CALCULO DEL ESPACIO LIBRE

EJEMPLO: Sea $N = 500,000$ RL $\rightarrow N_0 = 1'000,000$ RL

RL = 200 bytes $\rightarrow$ IC = 1024 bytes AC = 1 Cil
IC = 5 RL

1 Cil = $(20 - 1) * 3 * 4096$ bytes

1 Cil = $19 * 3 * 2 * 4 * 512$ bytes

1 Cil = $19 * 3 * 4$ IC

1 Cil = $19 * 3 * 4 * 5$ RL

Luego: $(1'000,000 - 500,000) / 1'000,000 = \%IC + \%AC$

$\%IC + \%AC = 50\%$ (Se convertirá a RL)

FIJAMOS $\%IC = 40\%$ (2 RL LIBRES EN CADA IC)

Luego : RL ($\%IC$ libre) + RL ($\%AC$ libre) = RL (50% AC)

así $2 * (19 * 12) + (5 - 2) AC \text{ libre} = 50\%(19 * 12 * 5)$

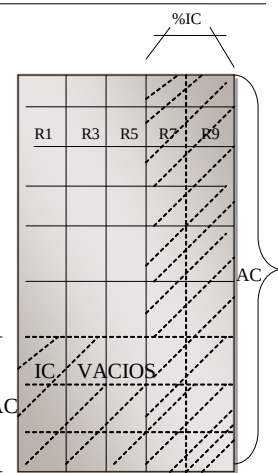
$3 * AC(\text{libres}) = (2.5 - 2) (19 * 12)$

$AC(\text{libres}) = (1/6) (19 * 12)$

$\%AC(\text{libres}) = (1/6) (19 * 12) / (19 * 12)$

$\%AC(\text{libres}) = 1/6$

$\%AC(\text{libres}) = 16.666\%$



CALCULO DEL ESPACIO LIBRE

EJEMPLO: Sea $N = 500,000$ RL $\rightarrow N_0 = 1'000,000$ RL

RL = 200 bytes $\rightarrow$ IC = 1024 bytes AC = 1 Cil
IC = 5 RL

1 Cil = $(20 - 1) * 3 * 4096$ bytes

1 Cil = $19 * 3 * 2 * 4 * 512$ bytes

1 Cil = $19 * 3 * 4$ IC

1 Cil = $19 * 3 * 4 * 5$ RL

FIJAMOS $\%IC = 40\%$

Luego : RL ($\%IC$ libre) + RL ($\%AC$ libre) = RL (50% AC)

así $RL(\%AC \text{ libre}) = RL(50\% AC) - RL(\%IC \text{ libre})$

$(5 - 2) AC \text{ libre} = 50\%(19 * 12 * 5) - 2 * (19 * 12)$

$(5 - 2) AC \text{ libre} = 50\%(19 * 12 * 5) - 40\% * (19 * 12 * 5)$

$(5 - 2) AC \text{ libre} = (50\% - 40\%) * (19 * 12 * 5)$

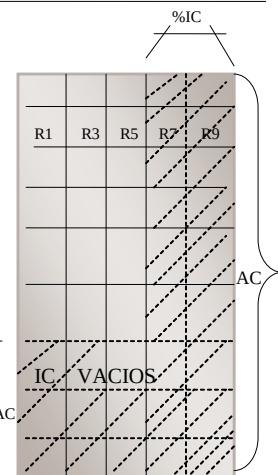
$AC \text{ libre} = (50\% - 40\%) * (19 * 12 * 5) / (5 - 2)$

$\%AC \text{ libre} = (50\% - 40\%) * (5) / (5 - 2)$

$\%AC \text{ libre} = (50\% - 40\%) / (1 - 2/5)$

$\%AC \text{ libre} = (50\% - 40\%) / (100\% - 40\%)$

$\%AC = (\%ESPLIB - \%IC) * 100 / (100 - \%IC)$



CALCULO DEL ESPACIO LIBRE

EJEMPLO: Sea $N = 500,000$ RL $\Rightarrow N_0 = 1'000,000$ RL

RL = 200 bytes $\Rightarrow$ IC = 1024 bytes AC = 1 Cil
IC = 5 RL

1 Cil = $(20 - 1) * 3 * 4096$ bytes

1 Cil = $19 * 3 * 2 * 4 * 512$ bytes

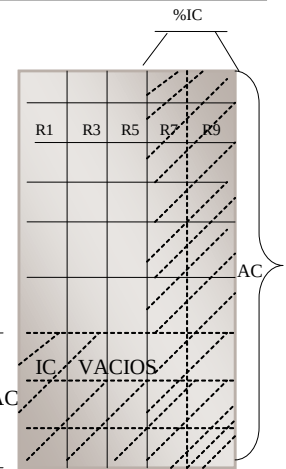
1 Cil = $19 * 3 * 4$ IC

1 Cil = $19 * 3 * 4 * 5$ RL

Luego: $(1'000,000 - 500,000) / 1'000,000 = \%IC + \%AC$
 $\%IC + \%AC = 50\%$ (Se convertirá a RL)

FIJAMOS $\%IC = 40\%$ (2 RL LIBRES EN CADA IC)

$\%AC$ (libres) = 16.666 % (38 IC LIBRES EN CADA AC)



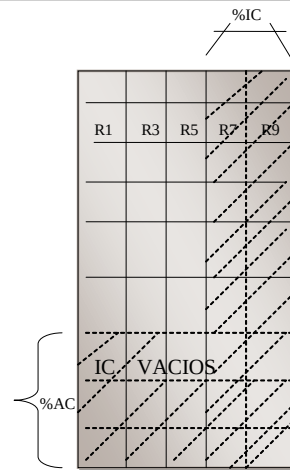
COMPROBANDO:

$$\begin{aligned} 2 * 19 * 12 + 3 * 38 &= 2 * 19 * 12 + 3 * 2 * 19 \\ &= 2 * 19 * 12 + 19 * 12 * (1/2) \\ &= (0.5) * (19 * 12 * 5) \\ &= (0.5) AC \end{aligned}$$

CALCULO DEL ESPACIO LIBRE

FINALMENTE LOS PARAMETROS
DEL ARCHIVO FISICO QUEDAN
DETERMINADOS.

- PARAMETROS MINIMOS PARA DEFINICION
- NOMBRE = **MAESTRO-CLIENTES**
- ORGANIZACIÓN = **INDEXADA**
- LONG.REG = **(2575, 200)**
- IC = **1024** (Se sugiere el mínimo)
- ESPLIB = **(40, 16.666)**
- CLAVE = **(10, 0)**
- CAPACIDAD = **(N=1000000, AC=19*12*5)**
N= Cantidad estimada total AC= Factor de expansión



CÁLCULO DEL ESPACIO EN DISCO

En función de los parámetros del archivo físico calcular el espacio en disco necesario para su data e índex.

- PARAMETROS MINIMOS PARA DEFINICION
- NOMBRE = **MAESTRO-CLIENTES**
- ORGANIZACIÓN = **INDEXADA**
- LONG.REG = **(2575, 200)**
- IC = **1024** (Se sugiere el mínimo)
- ESPLIB = **(40, 16.666)**
- CLAVE = **(10, 0)**
- CAPACIDAD = **(N=1000000 , AC=19*12*5)**
- N= Cantidad estimada total AC= Factor de expansión

CÁLCULO DEL ESPACIO EN DISCO

CÁLCULO PARA LA DATA

Se sabe que para cada AC se requiere un cilindro que puede almacenar a $19*12*5$ RL como máximo. En el momento de la carga se cuenta con 500,000 RL's , se respetará el espacio libre, luego por cada AC se almacenarán $(3 \text{ RL/IC}) * (19*12*5/6 \text{ IC/AC})$ luego 570 RL/AC para cargar 500,000 se tiene $500,000/570 \text{ AC's}$ es decir 877.19 se asignan 878 AC's.

Data = 878 cil

- PARAMETROS MINIMOS PARA DEFINICION
- NOMBRE = **MAESTRO-CLIENTES**
- ORGANIZACIÓN = **INDEXADA**
- LONG.REG = **(2575, 200)**
- IC = **1024** (Se sugiere el mínimo)
- ESPLIB = **(40, 16.666)**
- CLAVE = **(10, 0)**
- CAPACIDAD = **(N=1000000 , AC=19*12*5)**
- N= Cantidad estimada total AC= Factor de expansión

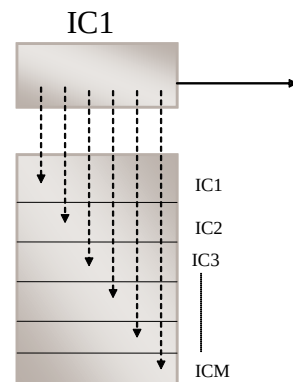
CÁLCULO DEL ESPACIO EN DISCO

CALCULO PARA EL INDEX:

PASOS:

- 1.- Calcular el mínimo numero de punteros necesario para el AC de data.
- 2.- Calcular longitud de puntero.
- 3.- Estimar longitud del IC del índice.
- 4.- Para nivel 1 el número de IC(indexe) es igual al número de AC de la data.
- 5.- Para los siguientes niveles se aplicaran al máximo los punteros para crear el árbol hasta llegar a la raíz.

CTR 24	VECTOR DE PTR			ER	C D I C	C I C 4
	1	-----	252	1	3	



Aplicación de la Metodología:

Paso 1.- Calcular el mínimo número de punteros necesarios para el AC de data.

El mínimo de punteros es igual al número de IC contenidos en el AC de la data.

$\#Punteros = \#IC (AC)$

Paso 2.- Cálculo de la longitud de cada puntero.

Cada puntero contiene Clave mas DRB.

$Longitud-Puntero = clave + DRB$

Paso 3.- Estimar longitud del IC del índice.

Como todo IC debe ser múltiplo de 512, por ser del índice se debe procurar que sea el mínimo.

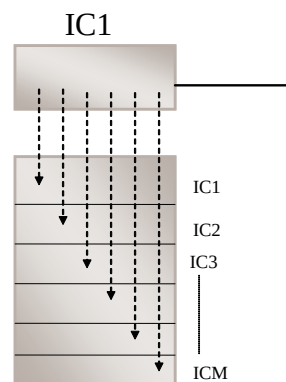
Estos cálculos se muestran a continuación:

CÁLCULO DEL ESPACIO EN DISCO

CÁLCULO PARA EL INDEX:

Cálculo de la longitud de IC(índice)
 el IC(índice) requiere por lo menos 19×12
 228 ptr's (1 para cada IC de la data).
 Cada ptr está compuesto por :
 $KEY + DRB = 10 + 3$ normalmente las
 claves se compactan para optimizar
 luego $PTR = 5 + 3$ (8 bytes por ptr.)
 Los IC siempre son m512,
 se tendrá: **$IC((índice) = 2048$** bytes
 con 252 ptr de capacidad.

CTRl 24	VECTOR DE PTR		ERR 134	C D I C 4
	1	-----	252	



Paso 4.- Determinar para el nivel uno del índice.

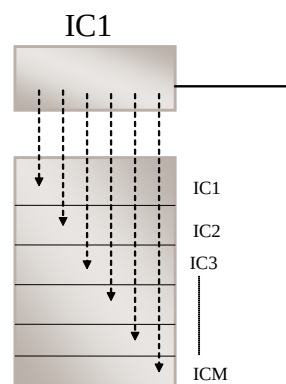
Paso 5.- Crear el árbol hasta la raíz.

CÁLCULO DEL ESPACIO EN DISCO

CÁLCULO PARA EL INDEX:

Cálculo de la longitud de IC(índice)
 el IC(índice) requiere por lo menos 19×12
 228 ptr's (1 para cada IC de la data).
 Cada ptr está compuesto por :
 $KEY + DRB = 10 + 3$ normalmente las
 claves se compactan para optimizar
 luego $PTR = 5 + 3$ (8 bytes por ptr.)
 Los IC siempre son m512,
 se tendrá: **$IC((índice) = 2048$** bytes
 con 252 ptr de capacidad.

CTRl 24	VECTOR DE PTR		ERR 134	C D I C 4
	1	-----	252	



Este es el esquema general de todo árbol de un archivo indexado.

Se ha comprobado que para archivos muy grandes en registros, los niveles no pasan de tres.

El componente índice normalmente reside en memoria, los buffers para los índices se reservan en el momento de apertura del archivo o conexión a la base de datos.

Esta característica optimiza el tiempo de búsqueda dando las coordenadas del volumen, Área de Control e Intervalo de Control.

REGENERACION FÍSICA DE BD

- Después de cumplido el tiempo de vida útil
- Se deberá proceder a su regeneración por que el espacio libre definido debe estar agotado de acuerdo a la estimación hecha y se debe prevenir las rupturas de IC y AC (fragmentación del disco).
- Los pasos a seguir son los siguientes:
 - Salvar componente DATA
 - Eliminar físicamente el archivo (Data e Índice)
 - Definir el archivo con nuevos valores de parámetros
 - ESPLI B (%IC, %AC) estos valores deberán ser estimados para el nuevo periodo de vida útil.
 - N = Cantidad de RLs esperados al final del periodo de vida útil.
 - Cargar la DATA salvada.

Este es uno de los procesos en el mantenimiento de la base de datos.

Los cálculos previos realizados son válidos para un tiempo de vida útil. Sin embargo después de este tiempo, los espacios libres asignados originalmente estarán agotados, lo que causará rupturas y fragmentación de la base de datos de manera continua, lo cual se evidenciará por una mala performance. La degradación podrá llegar hacer que colapse el gestor de la base de datos, dejando de operar.

Para evitar todo este cuadro nefasto, se realiza este proceso de regeneración.

Este proceso en sí no es complejo pero sí muy delicado.

Dado el volumen de datos que maneja la base deberán realizar cuadros de totales por diferentes dimensiones.

El otro aspecto es la continuidad del servicio, es decir mientras se de este proceso la base estará fuera de servicio, lo cual es negativo para el negocio. La estrategia adoptada para esta situación consiste en dos puntos:

Ser muy breve el tiempo del proceso y

Realizarlo en horas de más bajo uso del servicio.

ÍNDICES SECUNDARIOS (CLAVES ALTERNAS)

Este concepto ya se ha desarrollado anteriormente, por lo en este capítulo nos limitaremos a enfocar el aspecto técnico.

De manera similar a la estructura de la clave primaria para este índice se requiere contar con una data sobre la cual se construya el índice.

El componente data para este índice está compuesto por las claves primarias y secundarias bajo la estructura de un archivo de longitud variable.

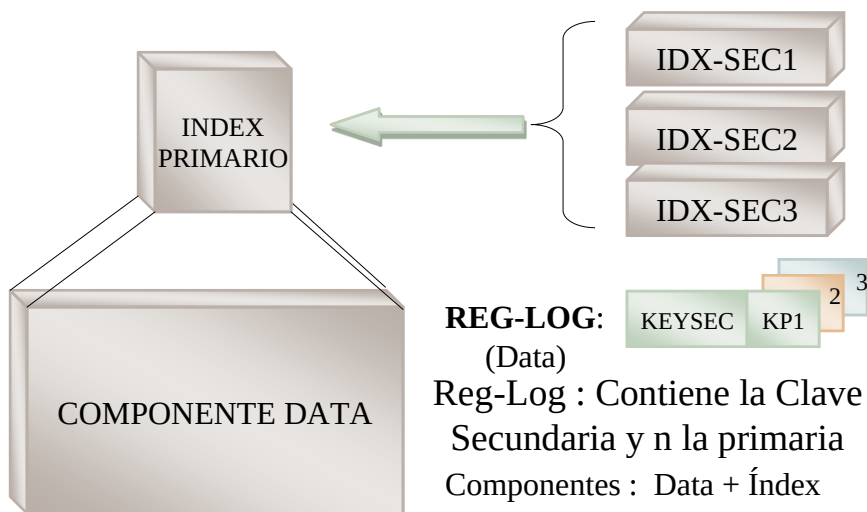
Estructura del registro data:

CLAVE-SECUNDARIA, CLAVE-PRIMARIA1, CLAVE-PRIMARIA2,.....

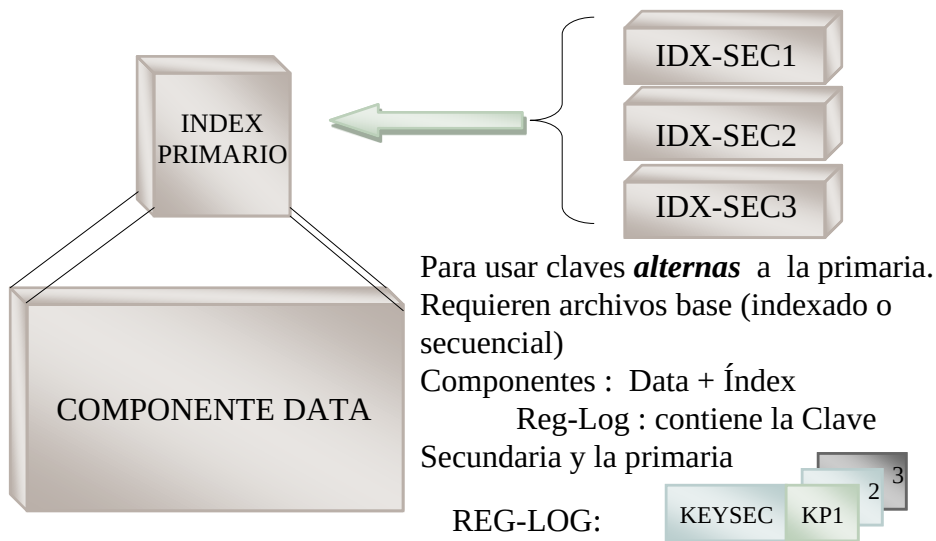
Esta estructura nos indica que para una clave secundaria podría haber más de una primaria, dado que las claves secundarias pueden repetirse no así las claves primarias.

Sobre este componente data se genera la estructura de índice ya presentada, de manera que dada una clave secundaria el índice devuelve de una a mas claves primarias, con estas se acceden al índice primario para recuperar la data definitiva.

INDICES ALTERNANTES



Para usar claves **alternas** a la primaria. Requieren archivos base (indexado o secuencial)



Ventaja: Permite accesos alternos al básico

Desventaja: En cada actualización se procesa más de un índice

Ventaja: Permite accesos alternos al básico

Desventaja: En cada actualización se procesa más de un índice

Costo de Vistas Indexadas

Para archivos voluminosos los niveles del Índice típicamente llegan a 3. Luego comenzando desde el nodo raíz se realizan 3 lecturas para llegar al nivel 1 y conseguir el DRB del IC que contiene la data.

Para conseguir la data solo será necesaria una lectura.

Total-Costo=4 accesos al disco.

A diferencia de archivos secuenciales donde su costo probable es $1/2N$.

N número total de IC (bloques, conjunto de registros).

Se recomienda acceder por medio de índices para búsquedas aleatorias.

Considerando que inclusive el componente índice se carga en la memoria real, podemos considerar que solo es necesario un solo acceso a disco, con lo cual la performance se asegura que sea óptima.

A modo de ejercicio realizaremos el siguiente ejemplo:

LONG-REG = 200 BYTES

IC = 1024, IC = 5 REG

CAPACIDAD = 1'000,000 REG.

AC = 1 CILINDRO = $19 * 3 * 4096$ BYTES

AC = $19 * 12$ IC = 228 IC

AC = $228 * 5$ REG

AC = 1,140 REG

NRO-AC = $1'000,000 / 1,140$

NRO-AC = 878 AC

Luego el archivo contiene 878 AC y cada AC contiene 228 IC

Dado que los accesos se dan a nivel del IC podrían ser necesarios hasta 200,184 accesos.

3.1.2.8. Organización relativa

ORGANIZACIÓN RELATIVA

Después de haber presentado las organizaciones de archivos como la Secuencial y la Indexada, nos toca presentar la tercera manera de organizar archivos, llamada Organización Relativa que permite realizar un acceso aleatorio a pesar que el archivo no posea componente índice.

Esta característica es posible por la estructura que presenta:

Los registros son de longitud fija. Cada registro se ubica físicamente en una posición fija, dada en referencia al inicio del archivo, de manera que se puede determinar el desplazamiento del registro mediante un simple cálculo de su desplazamiento relativo en bytes :

$$DRB = (PRR - 1) * Long-Reg.$$

*PRR = Posición Relativa del Registro

Permite acceso aleatorio / Los Reg. no poseen clave

Solo poseen componente Data.

Registro de longitud fija(L_0) y espacio libre cero.

PRR: Posición Relativa del Registro

indica su lugar físico donde está ubicado el registro.

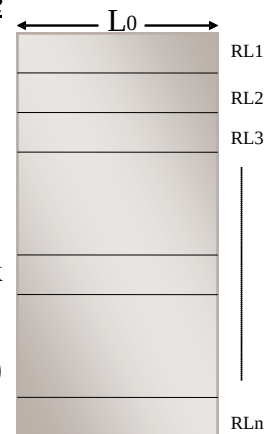
Es un número ordinal(1,2,3,..n)

$$Dir(k) = (L_0) * (PRR(k) - 1)$$

Los registros que no existen
se les reserva su espacio físico

formateado.

$$Dir(k) = (L_0) * (k-1)$$



Los registro se agrupan en IC y estos en AC.

Permite acceso aleatorio / Los Reg. no poseen clave

No poseen componente Índice

Acceso **directo** con PRR.

Registro de longitud fija(L_0) y espacio libre cero.

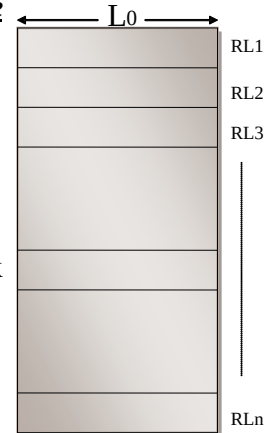
Ventajas.-

El acceso es directo no posee índice.

Mejora tiempo de respuesta

Al ser formateado no generan rupturas
de IC, AC.

$$Dir(k) = (L_0) * (k-1)$$



Desventajas.-

Solo maneja archivos de longitud fija.

Para el acceso aleatorio se debe conocer el PRR

Permite acceso aleatorio / Los Reg. no poseen clave

No poseen componente Índice

Acceso **directo** con PRR.

Registro de longitud fija(L_0) y espacio libre cero.

Usos.-

Por su gran velocidad de acceso

Por su alta seguridad al no

presentar problemas de

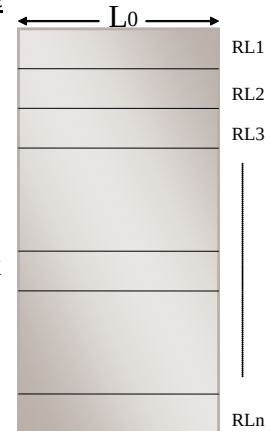
rupturas de IC ni AC

es preferido para módulos

Transaccionales en Tiempo Real.



$$\text{Dir}(k) = (L_0) * (k-1)$$

**Permite acceso aleatorio / Los Reg. no poseen clave**

No poseen componente Índice

Acceso **directo** con PRR.

Registro de longitud fija(L_0) y espacio libre cero.

Modo Uso.-

Se graba el RL con PRR = N (ubicación física)

donde N= Cantidad de RL máxima

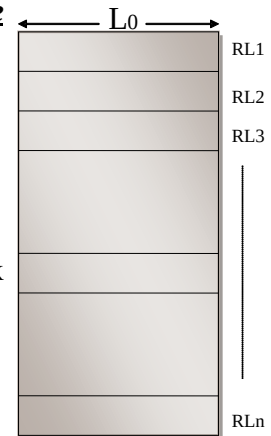
para un periodo de vida útil. Al grabarse

este registro, los N-1 RL's

previos quedarán formateados.



$$\text{Dir}(k) = (L_0) * (k-1)$$



Para conseguir el nuevo PRR se incrementará un correlativo.
(parámetro del sistema)

Para realizar búsquedas aleatorias deberá conocerse el PRR.

Permite acceso aleatorio / Los Reg. no poseen clave

No poseen componente Índice

Acceso **directo** con PRR.Registro de longitud fija(L_0) y espacio libre cero.**PARAMETROS MINIMOS PARA DEFINICION**

NOMBRE = PARA EL CATALOGO

ORGANIZACIÓN = RELATIVA

LONG.REG = (L_0 , L_0)

IC = m512 (Mínimo para transaccional)

CAPACIDAD = (N , AC) N = Cantidad estimada total AC = Factor de expansión



$$Dir(k) = (L_0) * (k-1)$$

Como se puede apreciar la definición del archivo sigue el mismo esquema que los anteriores dado que se trata de parámetros comunes.

Los software gestores de bases de datos facilitan al usuario el cálculo de estos parámetros por ejemplo:

LONG.REG : Se calcula en base a presencia de los atributos declarados. Cada atributo se define con un formato determinado pudiendo ser:

-CHAR, VARCHAR

-NUMERICO:

-DECIMAL: EMPACADO, DESEMPACADO

-BINARIO: COMA-FIJA, COMA-VARIABLE

Cada uno de estos formatos deben ir acompañados de sus longitudes ya sea de char o numéricos.

Finalmente el indicador de puede ser NULL o no.

CLAVE: El software lo determina por la posición relativa del atributo declarado.

Recordar Parámetros Críticos

Características del IC

- 1.- Número de registros Lógicos.
- 2.- Número de Sectores Físicos.
- 3.- Porcentaje libre.

Función: Almacenamiento y transferencia (Discos ↔ RAM)

Características del AC

- 1.- Número de Intervalos de Control.
- 2.- Número de Pistas (físicas).
- 3.- Porcentaje libre.

Función: Almacenamiento y expansión del archivo

Parámetros Críticos

Intervalo de control.- Agiliza o retarda el tráfico de datos entre los discos y memoria

Espacio Libre.- Permite crecimiento ordenado evitando reacomodos de páginas y cilindros.

Áreas de Control.- Para grandes volúmenes deben fijarse cilindros.

Se puede entender al IC como el puente entre la memoria real y los discos, luego en la medida que este tenga la longitud adecuada permitirá un tráfico óptimo o no.

En cuanto al espacio libre, este es determinante para el rendimiento y confiabilidad.

El rendimiento está en función del tiempo que demanda la operación, cuando el sistema dispone de espacio libre para ubicar nuevos registros, las operaciones aplicadas serán las mínimas. Por otro lado cuando la situación es la falta de espacio libre se desencadenan una serie de operaciones adicionales ya descritas anteriormente.

En cuanto a la confiabilidad está en función de cuan estable sea la estructura física de almacenamiento. La escasez de espacio libre obliga a crear nuevas AC remover data hacia otros cilindros, estos cambios físicos siempre están acompañados de un pequeño margen de falla por causa técnicas, debiéndose evitar sobre todo en ambientes en línea. Finalmente recordemos la recomendación de fijar la longitud del AC a un cilindro para volúmenes grandes de datos.

3.2. Rutinas de Implementación física de procesos

DISEÑO FÍSICO PROCESOS PROGRAMAS Y RUTINAS

La base de datos es el fundamento de todos los procesos que se puedan plantear en el sistema. Existen módulos típicos que aseguran un alto rendimiento del sistema sobre todo para enfrentar características complejas por parte de los negocios.

Las dos modalidades típicas y opuestas de proceso como On-Line y Batch que en ambos casos para grandes volúmenes se vuelven módulos críticos presentaremos modelos de base de datos así como su modelo de proceso, demostrando su eficiencia en:

- Recurso de disco

Tiempo de respuesta

Confiablez de proteger la información ante cualquier fallo .

Se presentará tanto el módulo on-line como el batch con sus diferentes variantes y matices, indicando en cada caso sus ventajas y desventajas de aplicación.

En el módulo on-line se complementará con estructuras de datos conocidas como Cadenas.

3.2.1. Módulo On Line

PROCESOS EN LÍNEA (Transaccional)

De toda la variedad de tipos de procesos existentes, el módulo Transaccional es uno de los más complejos a la luz de las características presentadas en el módulo On-line de la Arquitectura del Sistema.

El módulo transaccional se encuentra ubicado en el módulo On-Line específicamente en sub-módulo Operativo. Tiene como objetivo capturar en la base de datos las miles operaciones dadas en los negocios. Por lo tanto deben asegurarse agilidad como integridad.

Entiéndase agilidad como el tiempo mínimo requerido para realizar la operación.

Por el lado de Integridad, asegurar que la base de datos no pierda su coherencia en sus datos relacionados, vale decir por ejemplo un saldo final de verificarse en base a su saldo inicial afectado por sus movimientos.

3.2.1.1. Estructuras de encadenamiento de datos

ESTRUCTURA DE ENCADENAMIENTO DE DATOS UN ENFOQUE PRÁCTICO

Dentro de las estructuras de datos existentes la de encadenar datos es una de las más usadas para familia de registros al interior de una tabla o tablas. Esto significa que los registros de una familia pueden estar físicamente grabados de manera aleatoria al interior de la tabla, sin embargo deben estar ligados o relacionados lógicamente.

Estas ligaduras se realizan mediante apuntadores o punteros, que es una técnica para que desde un registro se conozca la dirección o ubicación de otro.

Dependiendo del tipo de organización los punteros pueden ser:

- Claves
- Dirección física
- Posición Relativa del Registro

Estos punteros se almacenan como atributos del registro llamados link (ligaduras).

3.2.1.2. Encadenamiento de datos

Aplicaciones prácticas Estructuras de Datos

- Cadena Simple
- Cadena Doble
- Cadena Compleja
- Anillos

Todas estas estructuras tienen en común que agrupan registros dispersos físicamente. Cada una de ellas presenta matices distintos, dependiendo de la cantidad de punteros presentes en la estructura pueden más o menos complejas. Se dice que en la medida que aumenta el número de punteros la complejidad crece geométricamente.

De allí los nombres de estas estructuras como cadena simple, doble y compleja. Finalmente la estructura tipo anillo también se considera una cadena donde el primer elemento está ligado al último.

A continuación se presentan los esquemas conceptuales y técnicos pero todo ello de cara a la aplicación práctica en una base de datos.

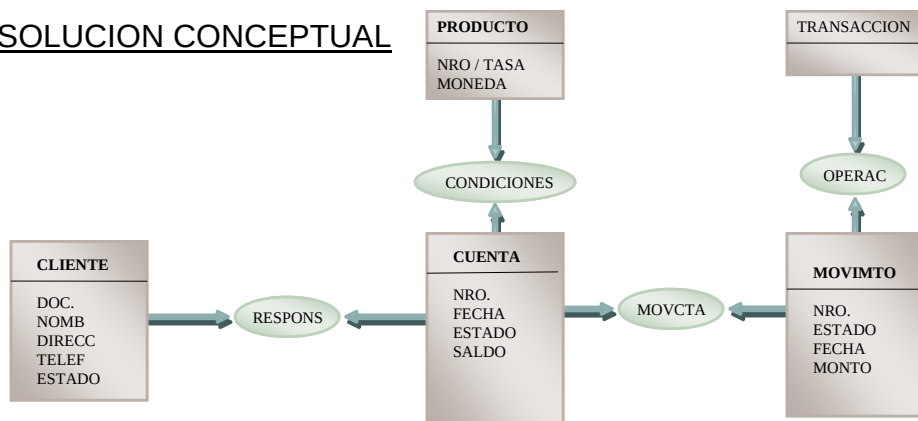
3.2.1.2.1. Cadena simple

Módulo Transaccional

- Modelo típico
 - Graba Registro de Movimiento
 - Actualiza Maestro
 - Tiempo Real :
 - Seguro ante cualquier falla
 - Tiempo respuesta mínimo
 - Arquitectura: Cliente / Servidor
 - Promedio 1'000,000 trx / día
 - Acceso Aleatorio

Módulo Transaccional

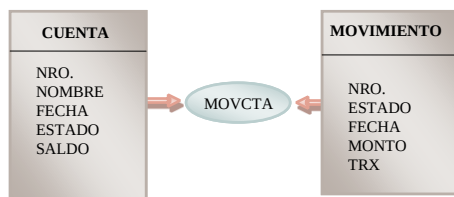
SOLUCION CONCEPTUAL



Siempre se debe partir desde un esquema conceptual, luego será transformado al esquema lógico recurriendo a los principios de optimización vale decir minimizando tiempo y recursos físicos.

Módulo Transaccional

REQUERIMIENTO PARA DISEÑO



Transacción altamente utilizada (procesar pocas tablas)

Procesa Saldos (proceso delicado)

Consultas de últimos movimientos

En tiempo Real: Tiempo de respuesta mínimo

Seguro ante cualquier falla

Base de Datos segura: (Evitar ruptura IC, AC)

Aplicación de Cadena Simple

Mov-diario:
Relativo

Mae-Ctas

Cta	nom	saldo	stat	pt
4640	JP	2000	A	001

Cta	Trx	Monto	
4640	C	100	PRR(001)
			PRR(100)
			PRR(500)
			PRR(999)

Especificaciones:

Mae-Ctas:

Indexado

Clave: Cta

Puntero : Ult-mov.

Aplicación de Cadena Simple

Mov-diario:
Relativo
Puntero:= Mov-ant

Mae-Ctas

Cta	nom	saldo	stat	pt
4640	JP	2000	A	001

Cta	Trx	Monto	Mov-ant
4640	C	100	0
			PRR(100)
			PRR(500)
			PRR(999)

Especificaciones:

Mae-Ctas:

Indexado

Clave: Cta

Puntero : Ult-mov.

Aplicación de Cadena Simple

Mae-Ctas

Cta	nom	saldo	stat	pt
4640	JP	2000	A	001

Especificaciones:

Mae-Ctas:

Indexado

Clave: Cta

Puntero : Ult-mov.

- 1.- Se valida la cuenta
- 2.- Se consigue PRR
- 3.- Se graba Movimiento.

Pasos para sgte. Trx.

Mov-diario:
Relativo
Puntero:= Mov-ant

Cta	Trx	Monto	Mov-ant	
4640	C	100	0	PRR(001)
4640	C	100	001	PRR(100)
				PRR(500)
				PRR(999)

Aplicación de Cadena Simple

Mae-Ctas

Cta	nom	saldo	stat	pt
4640	JP	1900	A	100

Especificaciones:

4.- Se actualiza Maestro

Mae-Ctas:

Indexado

Clave: Cta

Puntero : Ult-mov.

Mov-diario:
Relativo
Puntero:= Mov-ant

Cta	Trx	Monto	Mov-ant	
4640	C	100	0	PRR(001)
4640	C	100	001	PRR(100)
				PRR(500)
				PRR(999)

Aplicación de Cadena Simple

Mae-Ctas

Cta	nom	saldo	stat	pt
4640	JP	1800	A	500

Especificaciones:

Mae-Ctas:

Indexado

Clave: Cta

Puntero : Ult-mov.

Mov-diario:

Relativo

Puntero:= Mov-ant

Cta	Trx	Monto	Mov-ant	
4640	C	100	0	PRR(001)
4640	C	100	001	PRR(100)
4640	C	100	100	PRR(500)
4640	C	100	500	PRR(999)

Aplicación de Cadena Simple

Mae-Ctas

Cta	nom	saldo	stat	pt
4640	JP	1700	A	999

Especificaciones:

Mae-Ctas:

Indexado

Clave: Cta

Puntero : Ult-mov.

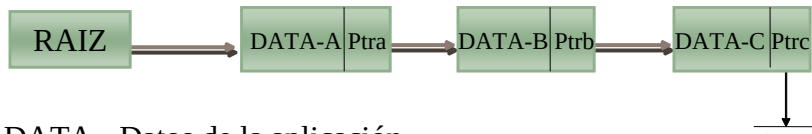
Mov-diario:

Relativo

Puntero:= Mov-ant

Cta	Trx	Monto	Mov-ant	
4640	C	100	0	PRR(001)
4640	C	100	001	PRR(100)
4640	C	100	100	PRR(500)
4640	C	100	500	PRR(999)

Aplicación de Cadena Simple



DATA.- Datos de la aplicación

PTR.- Dirección del siguiente elemento

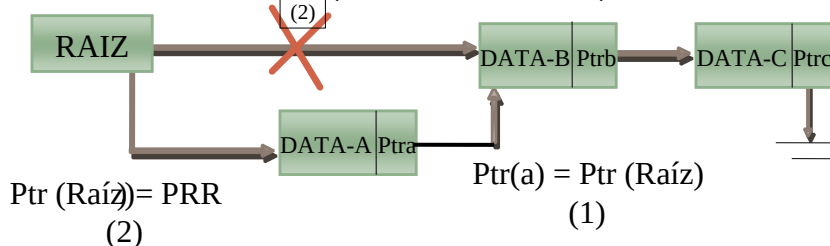
PUNTERO: { CLAVE
PRR
DRB

Toda cadena requiere de un elemento raíz, que contenga el inicio de la cadena

Los registros lógicos son administrados por el Método de Acceso Físicamente pueden estar alejados, sin embargo están unidos lógicamente.

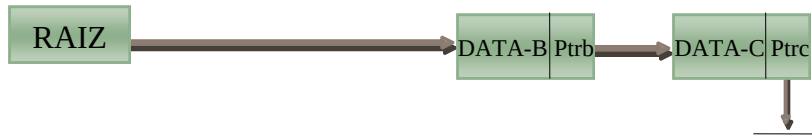
EVENTO:GRABA MOVIMIENTO

- **OPERACIÓN.-** VALIDA(CTA, TMOV, MTO)
 - METODO.- LEE-MAE(CTA)
 - METODO.- CHK-SALDO(SALDO, TMOV, MTO)
- **OPERACIÓN.-** CONSIGUE(PRR)
 - METODO.- ADD-PRR(+1 , PRR)
- **OPERACIÓN.-** GRABA-MOV(CTA, TMOV, MTO, MOV-ANT, PRR) (1)
 - METODO.- GRABA-MOV(CTA, TMOV, MTO, MOV-ANT, PRR)
- **OPERACIÓN.-** ACT-MAE(CTA, TMOV, MTO, PRR) ————— (2)
 - METODO.- ACT-MAE(CTA, TMOV, MTO, PRR)



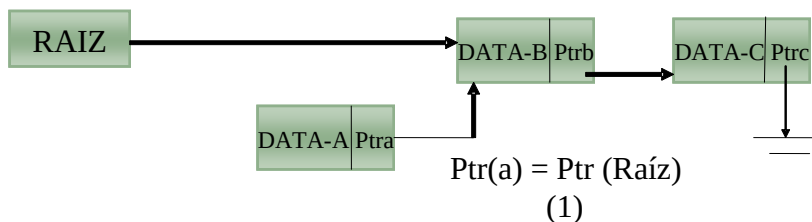
EVENTO:GRABA MOVIMIENTO

- **OPERACIÓN.-** GRABA-MOV(CTA, TMOV, MTO, MOV-ANT, PRR) (1)



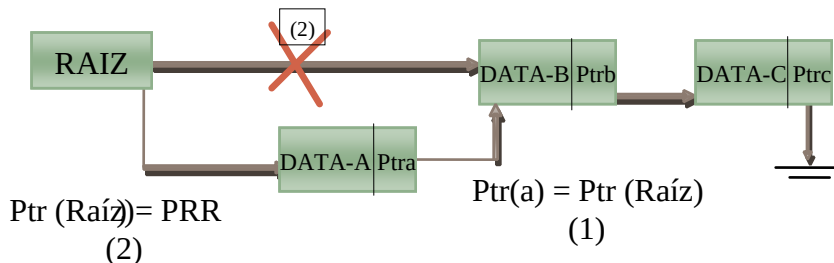
EVENTO:GRABA MOVIMIENTO

- **OPERACIÓN.-** GRABA-MOV(CTA, TMOV, MTO, MOV-ANT, PRR) (1)



EVENTO:GRABA MOVIMIENTO

- **OPERACIÓN.-** GRABA-MOV(CTA, TMOV, MTO, MOV-ANT, PRR) (1)
- **OPERACIÓN.-** ACT-MAE(CTA, TMOV, MTO, PRR) (2)

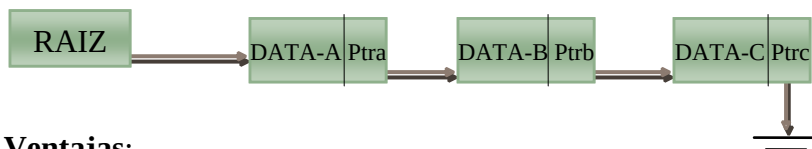


La secuencia de pasos es importante de manera que si se realizan en sentido inverso se destruirá la cadena.

La operación consiste en actualizar los punteros.

El elemento “A” puede estar físicamente distante de la cadena después de la actualización pertenecerá a ella.

APLICACIÓN DE CADENA SIMPLE



Ventajas:

- Fácil recuperación de los elementos
- Fácil mantenimiento.

Desventajas:

- Para cadenas grandes el tiempo de recuperación puede ser grande
- No tiene posibilidad de retroceder a elementos anteriores.

A partir de la solución de un caso práctico se ha presentado el marco teórico de la estructura de encadenamiento de datos.

Donde nos abre nuevas posibilidades de diseño el recurrir al uso de punteros.

Estos esquemas se enmarcan en las conocidas teorías de redes, que se fundamentan en el uso de apuntadores.

Estas posibilidades son de naturaleza distinta a las técnicas relacionales, de manera que la potencialidad radica en usar en forma combinada ambas técnicas, de acuerdo a la naturaleza del problema y requerimientos técnicos demandados.

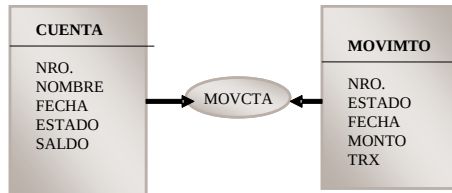
Por otro lado se ha presentado la técnica del reformato físico de tablas, que es otra alternativa para el manejo del espacio libre. Se debe evaluar en cada caso cual es la recomendable.

A partir del primer modelo de cadena simple se seguirá la presentación de las demás estructuras con todo detalle y forma de aplicación.

Módulo transaccional

Modelo x Canal

REQUERIMIENTO PARA DISEÑO:



Transacción altamente utilizada

(*Procesar pocas tablas*)

Procesa Saldos (*Proceso delicado*)

Consultas de últimos movimientos

En tiempo Real

Tiempo respuesta mínimo

Seguro ante cualquier falla

Base de Datos **segura**:

(*Evitar ruptura de IC, AC*)

Se entiende por **Canal** al tipo de vía por donde ingresa la transacción

Pueden ser Ventanilla, Cajero Automático, Teléfono, internet, punto de venta etc.

Aplicación del modelo por canal

Mov-diario:

Indexado

Clave-Prim= Canal + Corr

Clave-Sec = Cta.

Mae-Ctas

Cta	nom	saldo	stat	
4640	JP	2000	A	

Especificaciones:

Mae-Ctas:

Indexado

Clave: Cta

Se debe especificar las características del archivo de Mov. y los procesos previos a

Caso-1.-Con Formateo

Caso-2.-Sin Formateo

Cnal	Corr	Cta	Trx	Monto	
Cna1A	0001	4640	C	100	
Cana1B	0001				
Cana1C	0001				

Caso-1.- Con Formateo: Es el mismo caso de la organización Relativa, es decir los registros serán grabados con valores inicializados ceros o espacios dependiendo de su formato de cada atributo.

Caso-2.- Sin Formateo: En este caso se define un espacio libre de manera que el crecimiento planeado se ejecute sin problemas de fragmentación.

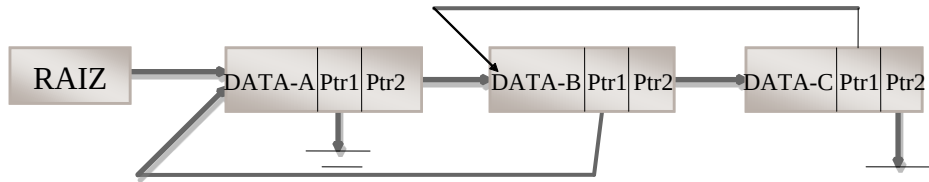
La clave debe estar formada por:

- Clave Genérica = Canal
- Clave Específica= Secuencia (1,2,3....)

De esta manera a nivel de cada AC se podrá crecer a su interior de manera controlada sin crear rupturas de AC.

3.2.1.2.2. Cadena doble

Aplicación de Cadena Doble

**Ventajas:**

Permite recorrer la cadena en ambos sentidos.

Desventajas:

El mantenimiento se hace complejo

En esta estructura al contar con doble puntero tenemos la posibilidad de un recorrido de la cadena en doble sentido, como ya se ha mencionado la aparición de un puntero adicional siempre vuelve más delicado su manejo sobre todo en la actualización.

Las aplicaciones son muchas, la más conocida es la de presentar los elementos agrupados en páginas con la posibilidad de avance y retroceso de las mismas.

3.2.1.2.3. Anillo

Aplicación de Anillo

La tarjeta de crédito es un producto bancario que permite comprar a sola firma. Toda tarjeta pertenece a un Titular. Además el Titular puede solicitar una o mas tarjetas adicionales para familiares.

Si se quisiera tener el Maestro de Tarjetas de manera que dado cualquier tarjeta titular o adicional recuperar el resto de la familia

Implementación

Mae-indexado

Cta	Est	Saldo	
Adi3	C	100	
Adi2	C	100	
Adi1	C	100	
Titul	C	100	

Esta solución tiene por objetivo ligar varios registros físicos de una misma familia.

Como se observa la solución obtenida es bastante sencilla pero muy efectiva y elegante a la vez.

Para este ejemplo planteado la solución mediante un anillo es preferible al enfoque relacional.

Siempre será posible ante cualquier caso de diseño encontrar dos soluciones bajos estos enfoques, se deberá elegir el óptimo para el caso particular. Entiéndase óptimo por no solo el que economice recursos y tiempo sino también aquel que permita un mantenimiento rápido y sencillo.

Aplicación de Anillo

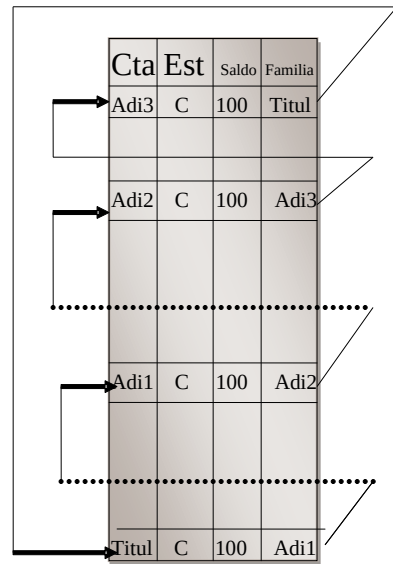
La tarjeta de crédito es un producto bancario que permite comprar a sola firma. Toda tarjeta pertenece a un Titular. Además el Titular puede solicitar una o mas tarjetas adicionales para familiares.

Si se quisiera tener el Maestro de Tarjetas de manera que dado cualquier tarjeta titular o adicional recuperar el resto de la familia

Implementación:

Mae-indexado

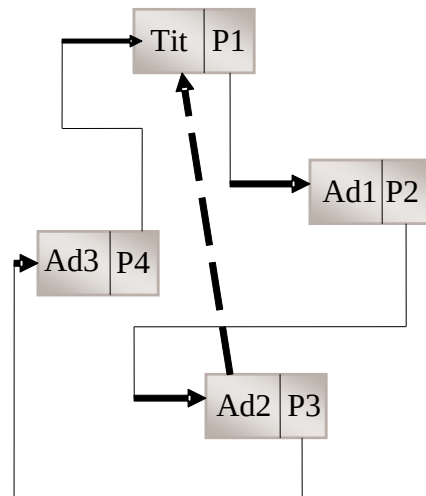
Prt = sgte-fam..



Aplicación de Anillo

Es una cadena de elementos donde el último apunta al primero. Se puede implementar con archivos indexados o relativos. Las cadenas pueden ser simples, dobles o complejas.

Una variante puede ser : que desde cualquier elemento secundario se pueda apuntar al principal.



3.2.1.2.4. Cadena compleja

Aplicación de cadena compleja

Tarjeta de Crédito.

En base al problema anterior, considerar además que cualquier tarjetahabiente, puede reemplazar su tarjeta original por otra, siendo el motivo, robo o extravío, manteniendo las anteriores en estado de canceladas como parte de su historia.

Si se quisiera tener el maestro de las tarjetas de manera que dada cualquier tarjeta, titular o adicional, activa o cancelada recuperar el resto de la familia

Implementación : Se dan dos cadenas, una principal y una secundaria, la principal tendrá dos punteros activos y la segunda sólo uno.

Mae-indexadc
Ptr1 = sgte-tarj
Ptr2 = sgte-fam

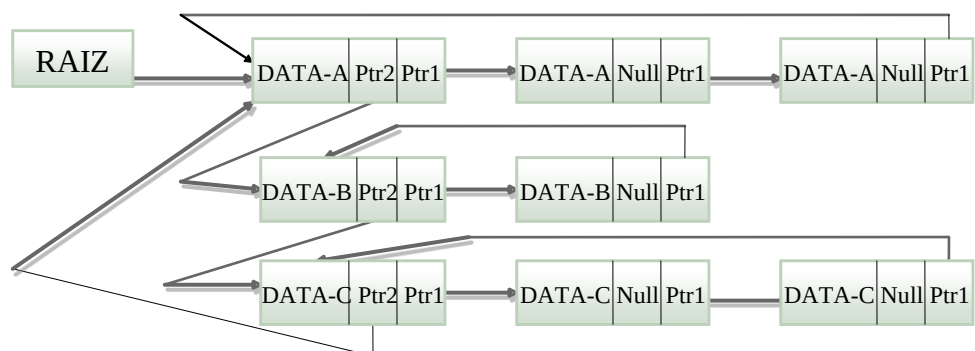
Aplicación de Cadena compleja

Tarjeta de Crédito.

En este caso se utiliza un puntero adicional para tarjeta para apuntar a su tarjeta anterior.

Implementación:

Mae-indexado
Ptr1 = sgte-tarj
Ptr2 = sgte-fam..

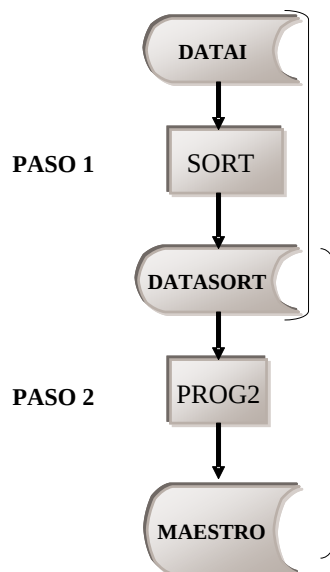


Se recomienda usarla sólo en casos muy especiales por la complejidad tanto para su desarrollo como para su mantenimiento.

3.2.2. Modulo Batch

PROCESOS EN LOTES (BATCH)

Un típico proceso batch se presenta en el siguiente esquema:



Donde se observa la presencia de dos pasos que indican la presencia de dos procesos dentro de la gran cadena de procesos pudiendo llegar a ser muy extensa.

Procesos en Lotes

BATCH: Los primeros que se usaron

ENFOQUE: Los programas o rutinas informáticas son conjunto de instrucciones que se escriben para conseguir un objetivo.

Sea se encuentre en ambiente en línea o por lotes, básicamente procesan datos contenidos en archivos según una determinada secuencia lógica.

Un mismo objetivo puede conseguirse vía en línea o batch.

La diferencia radica en la manera de tratar a los archivos en disco.

Supongamos que el proceso sea facturar servicios.

On-Line : Se deben conseguir los diferentes cargos para un solo cliente. Para conseguir estos cargos puede ser necesario acceder a “n” archivos. Resultado “n” accesos a disco.

Batch : Se deben conseguir los diferentes cargos para “n” clientes. Para conseguir estos cargos puede ser necesario acceder a “n” archivos. **Resultado “n” x “n” accesos a disco.**

Para optimizar el proceso batch se diseña en “n” pasos (procesos) de manera que en cada paso se accede una tabla para los “n” clientes.

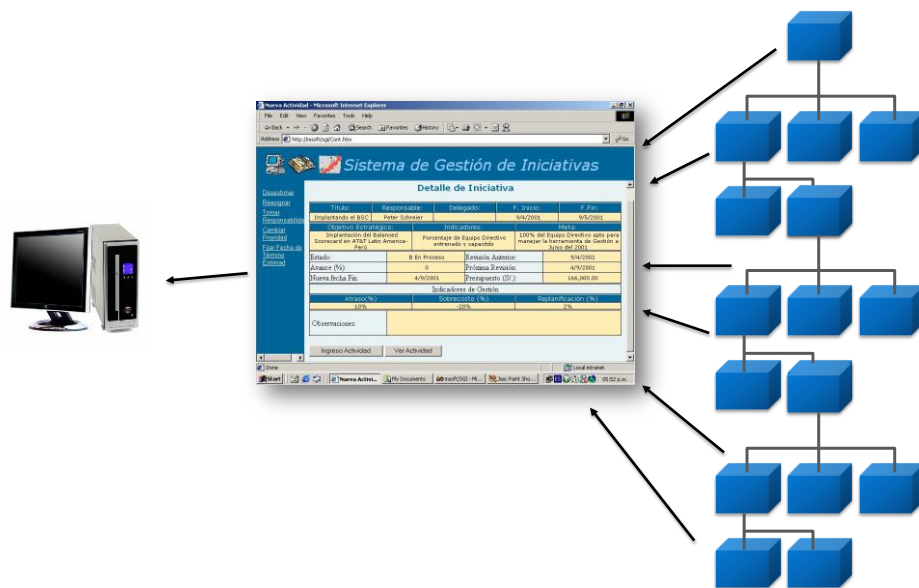
Iniciamos presentando un esquema On-Line.

Se observa que para procesar un solo cliente es necesario acceder a muchas tablas de manera aleatoria.

Un
CLIENTE

Proceso en línea

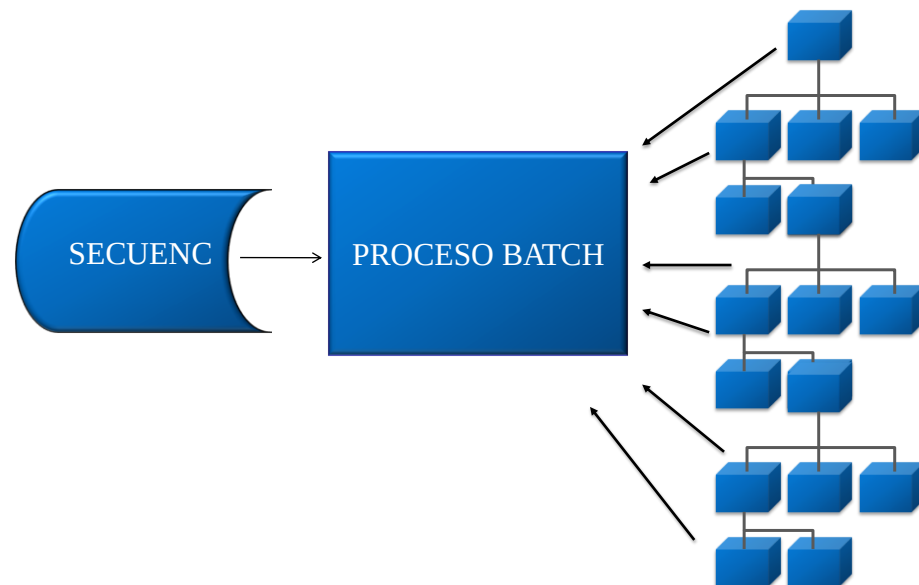
"n"
ACCESOS

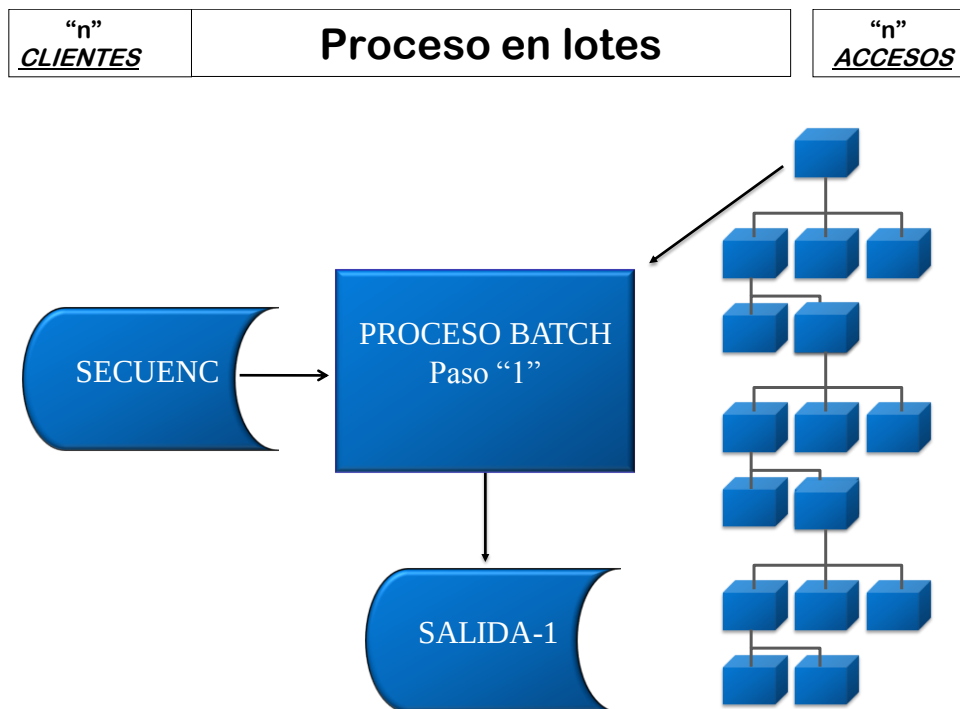


"n"
CLIENTES

Proceso en lotes

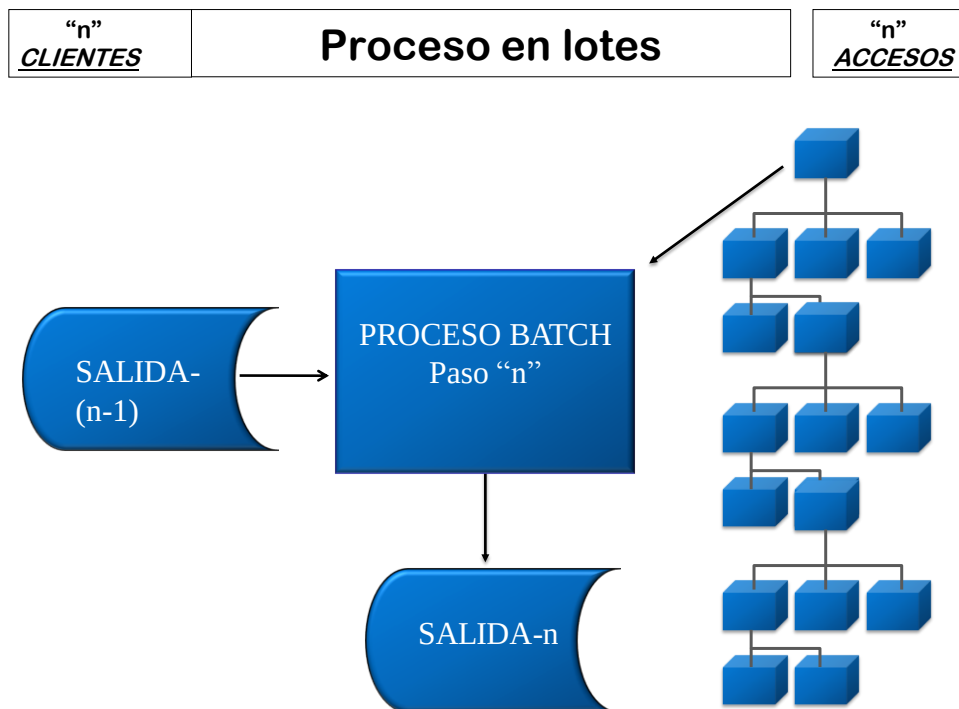
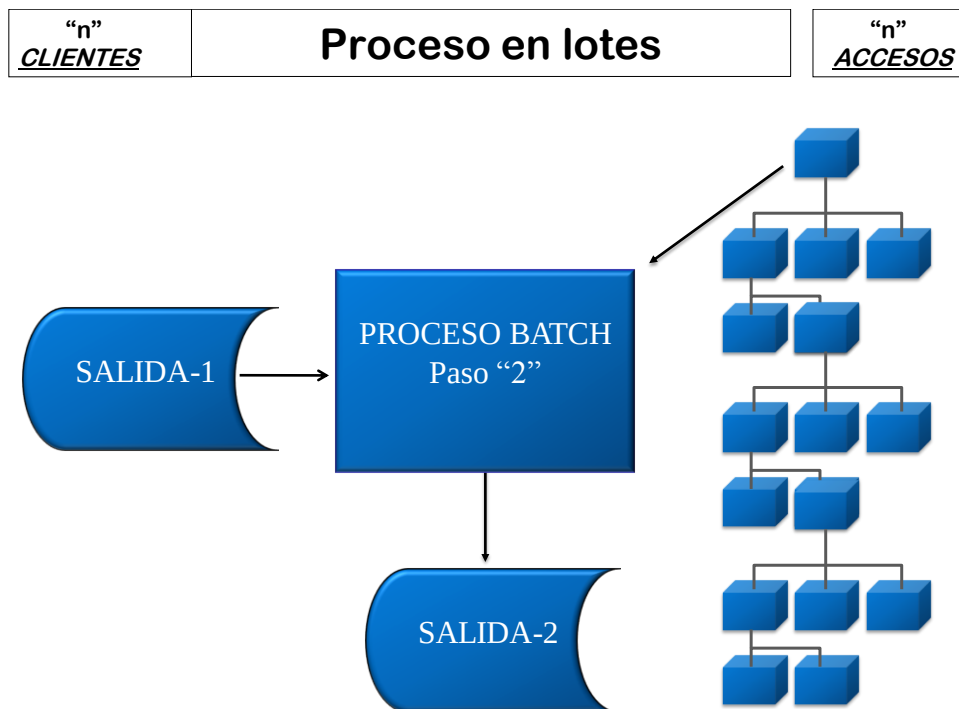
"n" x "n"
ACCESOS





Un lote de "n" clientes se encuentran en el archivo secuencial, los n serán procesados frente a una tabla. Esta modalidad mejora el tiempo de procesos porque se logra disponer de la tabla de la base de datos de manera exclusiva para el paso 1.

Al final de este paso se conseguirá un resultado parcial en un archivo intermedio que servirá de entrada para el siguiente paso.



De esta manera se llega al proceso “n”.

Demostrando la eficiencia de esta modalidad aplicando:

- 1.-Tratamiento de archivos en formato batch (bloques grandes) y
- 2.-Modalidad utilizada (barrido completo de los archivos)

De esta manera se asegura un proceso óptimo.

Procesos en Lotes

Batch : Los primeros que se usaron

Siguen vigentes por filosofía operativa empresarial

Modelo típico:

1. Carga de base de datos
2. Actualización de archivos
3. Acumulación de movimientos
4. Recuperación de maestros
5. Generación de reportes voluminosos

La ventaja de tener identificados los procesos permite que estos se diseñen de manera óptima. A continuación se detallan cada uno de estos procesos típicos sobre una base de datos.

Creación de Archivos

Proceso presente en todo módulo

Para procesos en línea. Para el archivo que esté listo a operar en tiempo real, no puede estar vacío, se requiere que exista por lo menos un registro semilla.

Este registro semilla será formateado y para el caso de archivos indexados el valor de sus claves podrán ser el mayor o menor posible

3.2.2.1. Carga de base de datos

CARGA DE BASE DE DATOS

El proceso de carga de una tabla de base de datos debe seguir cierto protocolo para asegurar su calidad.

Podemos distinguir dos grupos de archivos:

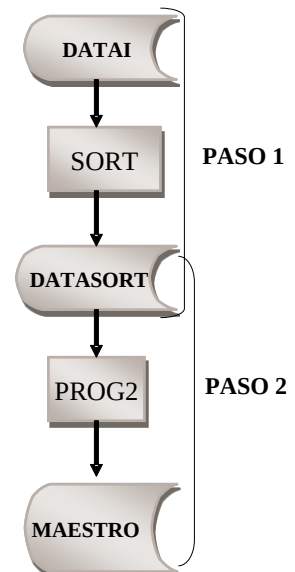
- 1.- Archivos planos
 - Secuencial
 - Indexado
 - Relativo
- 2.- Tablas de base de datos
 - Con Índice
 - Sin Índice

3.2.2.1.1. Carga de secuencial

BATCH: Procesos en Lotes

CARGA DE BASE DE DATOS.-

1.-ARCHIVO SECUENCIAL.- TEMPORAL
NO UTILIZA CLAVE
SIMILAR ANTERIOR
NO REQUIERE ORD.DATA



3.2.2.1.2. Carga indexado

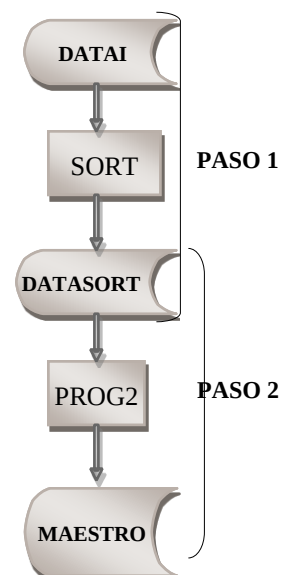
BATCH: Procesos en Lotes

CARGA DE BASE DE DATOS.-

2.-ARCHIVO INDEXADO: MAESTRO
DATA ORDENADA X CLAVE

PASOS:

- 1.- DEFINIR ARCHIVO
 - .- ORDENAR ARCHIVO ENTRADA
- 2.- CARGAR MAESTRO
 - .- ELIMINAR ARCHIVO ENTRADA



3.2.2.1.3. Carga de relativo

BATCH: Procesos en Lotes

CARGA DE BASE DE DATOS.-

3.-ARCHIVO RELATIVO.-

MOVIMIENTOS(TRX)
NO UTILIZA CLAVE
SE FORMATEA ANTES DE OPERAR
BASTA CON GRABAR EL MAYOR(PRR)

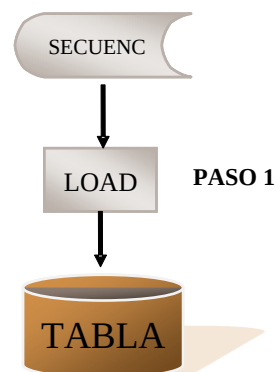
BATCH: Procesos en Lotes

CARGA DE TABLA DE BASE DE DATOS.-

A partir desde un archivo plano:

Características:

- 1.- Se da traslado de datos
- 2.- Las características físicas de la entrada es diferente a su salida.
- 3.-La entrada fue generada en ambiente Batch y la salida será usada en ambiente On-line.
- 4.- Este proceso une los dos ambientes.



Los software gestores de base de datos proveen de módulos de carga (load) de tablas físicas. Después de haber definido o inscrito la tabla nueva en la base.

DESCARGA DE BASE DE DATOS

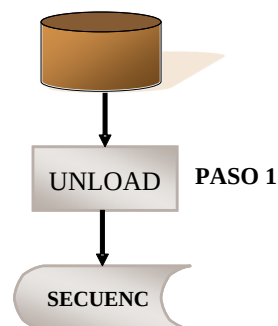
BATCH: Procesos en Lotes

DESCARGA DE BASE DE DATOS.-

A partir desde una tabla de base de datos :

Características:

- 1.- Se da traslado de datos
- 2.- Las características físicas de la entrada es diferente a su salida.
- 3.-La entrada fue generada en ambiente On-line y la salida será usada en ambiente Batch .
- 4.- Este proceso une los dos ambientes.



También puede darse el proceso inverso de descarga de base de datos, para lo cual existe el módulo utilitario (unload).

Se debe definir el archivo plano de destino. Generalmente será utilizado para procesos de grandes lotes que una vez en formato batch el proceso optimiza el tiempo de proceso.

3.2.2.2. Actualización de base de datos

ACTUALIZACIÓN DE ARCHIVOS MAESTROS

CONCEPTO.- Existen muchos casos donde la base de datos debe ser actualizada en proceso batch. Por ejemplo el pago mensual de los empleados de toda empresa, Las facturaciones de servicios públicos como luz, agua, teléfono, etc.

En todos estos casos queda claro que son procesos que comprometen un volumen gran de datos a ser procesados.

Dependiendo de las características del evento presentaremos el esquema de solución que asegure el mínimo de tiempo y utilización de recursos.

Los esquemas típicos son los siguientes:

- 1.- Actualización con índice Sin Quiebre.
- 2.- Actualización con índice Con Quiebre.
- 3.- Matching Sin Quiebre
- 4.- Matching Con Quiebre

3.2.2.2.1. Actualización indexada con y sin quiebre

BATCH: Actualización de archivos maestros

Sin quiebre

Alta, baja y cambio

BATCH: Actualización de maestro s/quiebre

SEUDO-CODIGO:

PROGRAMA.-

INICIO
PROCESO HASTA FIN-ARCH
FIN.

INICIO.-

INICIALIZA-VAR
PREPARA-ARCHIVOS
LEE-MOV

PROCESO.-

LEER-MAESTRO
ACT-MAE-MEMORIA
ACT-MAE-DISCO.
LEE-MOV.

FIN.-

ESTADISTICAS

Caso de uso.- En una tabla de maestra de direcciones de los clientes que pueden contener un millón de registros aproximadamente, se requiere modificar una cantidad de 1000 al día.

BATCH: Actualización de archivos maestros

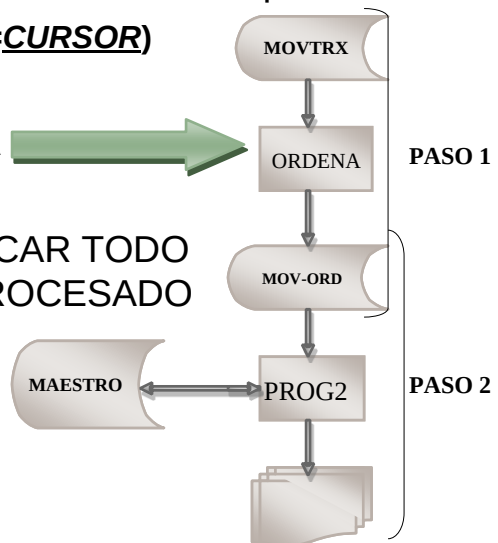
Con quiebre

Grupo de movimientos Afectan a un registro maestro
 Actualizan campos numéricos (saldos, montos, etc.)
 Una sola actualización del registro maestro por n movimientos

Procesos en Lotes

- PROCESO: Actualización de Maestro c/quiebre
- PASO1 = **`SORT(DATOS-I=CURSOR)`**
- PASO2 = **`ACT-MAE`**
- MODELO EN CASCADA 

- SE RECOMIENDA MARCAR TODO REGISTRO DE MOV. PROCESADO
- Si se intenta volver a actualizar, se detecta y se ignora



Si se aplicase el mismo esquema anterior se producirían varios accesos para actualizar la el mismo objeto de la base de datos, lo cual sería oneroso en tiempo y recursos. Luego lo óptimo es agrupar todos los movimientos que afectarán al mismo objeto, al agrupar estos movimientos conseguiremos el impacto neto y este será la única actualización para dicho objeto.

Procesos en Lotes

BATCH:

ACTUALIZACION DE MAESTRO C/QUIEBRE.

SEUDO-CODIGO:

PROGRAMA.-

INICIO
PROCESO HASTA FIN-ARCH
FIN.

INICIO.-

INICIALIZA-VAR
PREPARA-ARCHIVOS
LEE-MOV

PROCESO.-

SI CTA = CTA-ANT
ACUMULAR

SINO
QUIEBRE

FIN-SI.
LEE-MOV.

ACUMULAR.-

SI TIPMOV = A
TOTMOV = TOTMOV + MOV

SINO
SI TIPMOV = C
TOTMOV = TOTMOV - MOV
SINO MSG: ERROR TIPMOV
FIN-SI
FIN-SI

QUIEBRE.-

ACT-MAESTRO
INICIALIZA-VAR

ACT-MAESTRO.-

LEER-MAESTRO
ACT-MAE-MEMORIA
ACT-MAE-DISCO

FIN.-

ACT-MAESTRO

Caso de uso.- El caso típico para este proceso es cuando en una entidad bancaria existen miles de cheques que provienen del canje inter-bancario durante un día de operaciones. Muchos de estos cheques afectarán a una misma cuenta, por lo que se debe conseguir el monto neto y solo este resultado actualizará el saldo de la tabla maestra.

Al conseguir el monto neto para actualizar es cuando debemos cambiar de cuenta, a este cambio se le conoce como “quiebre”.

Para que pueda ser posible es necesario que el archivo de movimiento se encuentre ordenado por cuenta.

Lo que se está logrando es reducir el número de acceso a la tabla maestra para su actualización.

Este hecho es doblemente beneficioso por que más allá de reducir el número de accesos al disco, se minimiza el manipuleo de la base de datos, con lo cual baja la probabilidad de error.

Dentro de todo proceso de actualización de grandes volúmenes de datos se debe llevar un control exacto del punto exacto del flujo del proceso con un **Punto de Control**.

PUNTO DE CONTROL DE PROCESO (CHECKPOINT)

Presentaremos los componentes que hacen posible llevar el control y seguimiento de los procesos, sobre todo para tomar acciones de manera automáticamente.

MODULO CHECKPOINT.-

PERMITE CONOCER EL ESTADO DE UN PROCESO RESPECTO A SU BASE DE DATOS. PARA QUE DADO EL CASO DE FINAL ABRUPTO DEL PROCESO, RETOMAR EL PROCESO DESDE EL PUNTO DONDE SE PRODUJO EL PROBLEMA. PROCESO CONOCIDO COMO RE-ARRANQUE. PARA QUE EL PROCESO PUEDA RE-INICIAR, EL SISTEMA DEBE TENER SU ESTADO IGUAL AL ULTIMO COMMIT (ACTUALIZACION FISICA DE BD) OK.

LOS EVENTOS DEL PROCESO SON:

- A.- **INICIO**, ESTE INICIO PUEDE SER NORMAL O FORZADO
- B.- **COMMIT**, CADA VEZ QUE DEBE ACTUALIZAR FISICAMENTE LA BD. SI OCURRE UN ERROR DEBERA REALIZAR ROLLBACK.
- C.- **FIN**, PUEDE SER NORMAL O CON ERROR.

DESDE QUE INICIA UN PROCESO HASTA SU FINALIZACION SE MANTIENE EN ESTADO DE RE-ARRANQUE. DE MANERA QUE SI TERMINA DE MANERA ABRUPTA, AL RE-INICIAR EL PROCESO ESTE SERA RE-ARRANCADO.

UN PROCESO PUEDE EJECUTARSE MAS DE UNA VEZ AL DIA.
TIPO-INICIO = (NORMAL O FORZADO).

Este módulo debe ser activado cada vez que se ejecuta una cadena de procesos.

Cada vez que inicia cualquier paso de la cadena se almacena los datos que permitan re-iniciar el proceso como Job, Paso, Secuencia, etc.

ELEMENTOS.-

UTILIZA DOS TABLAS:

TABLA DE CHECKPOINT Y DE ARCHIVOS.

A) **TABLA CHECKPOINT.**- ALMACENA INFORMACIÓN DEL ESTADO DEL PROCESO HASTA LA ULTIMA INTERACCION OK QUE REALIZO EL MODULO CON EL PROCESO.

B) **TABLA ARCHIVOS**. - ESTA TABLA INDICA CUANTOS ARCHIVOS PLANOS POSEE EL PROCESO Y CUANTOS REGISTROS POR CADA ARCHIVO YA HAN SIDO GRABADOS.

AMBAS TABLAS SE ACTUALIZAN SIMULTANEAMENTE.

EVENTO:

INICIO.-

EL PROCESO AL INICIAR INVOKA AL CHECKPOINT CON

PARAMETRO = INICIO

$$\text{TIPO-INICIO} = (N, F).$$

```
ESTADO DEL CHECKPOINT:      NO EXISTE,      SE CREA CHECKPOINT
                           EXISTE EN ESTADO N, SE ACTUALIZA CHCKPNT
                           EXISTE EN ESTADO R, RE-ARRANQUE
```

(VER TABLA DE VERDAD)

Tipo de Proceso: Inicio.-

Estado del Check-Point:

Tipo: N (normal) o F (forzado)

Puede ser N (normal) luego Actualiza Check-Point

Puede ser R (re-arranque) luego Re-Arranca

Puede ser b (en blanco) luego Crea Check-Point

```

*****
* COBOL DECLARATION FOR TABLE ASDBI.ASCHKPMD
*****
01  DCLASCHKPMD.  TABLA DE CHECKPOINT
    10  CHKP-EMPRESA           PIC X(4) .
    10  CHKP-ID-JOB            PIC X(8) .
    10  CHKP-ID-PASO           PIC X(8) .
    10  CHKP-NU-SECUENCIA      PIC X(1) .
    10  CHKP-IND-REARRAN       PIC X(1) .(N , R)
    10  CHKP-TS-ULT-COMMIT     PIC X(26) .
    10  CHKP-NUM-COMMIT        PIC S9(9)  USAGE COMP.
    10  CHKP-TOT-ACTUALIZA     PIC S9(9)  USAGE COMP.
    10  CHKP-TO-ACT-UL-INT     PIC S9(9)  USAGE COMP.
    10  CHKP-FREC-COMMIT       PIC S9(9)  USAGE COMP.
    10  CHKP-TIEMPO-COMMIT     PIC S9(9)  USAGE COMP.
    10  CHKP-TIEMP-ELAPSED     PIC S9(9)  USAGE COMP.
    10  CHKP-NUM-FICH-SAL      PIC S9(4)  USAGE COMP.
    10  CHKP-IND-PARADA        PIC X(1) .
    10  CHKP-LONG-DA-REPOS     PIC S9(4)  USAGE COMP.
    10  CHKP-DATOS-REPOS.
        49  CHKP-DATOS-REPOS-LEN  PIC S9(4)  USAGE COMP.
        49  CHKP-DATOS-REPOS-TEXT PIC X(512) .

```

01 DCLASFICHMD. **TABLA DE FICHEROS**

10 FICH-EMPRESA	PIC X(4) .
10 FICH-ID-JOB	PIC X(8) .
10 FICH-ID-PASO	PIC X(8) .
10 FICH-NU-SECUENCIA	PIC X(1) .
10 FICH-NOM-FICH-SAL	PIC X(8) .
10 FICH-NUM-REGISTROS	PIC S9(9) USAGE COMP.
10 FICH-NOM-FICH-TEMP	PIC X(8) .
10 FICH-USER-ULT-MOD	PIC X(16) .
10 FICH-TS-ULT-MOD	PIC X(26) .

La funcionalidad más importante de este módulo es que después de terminar de forma abrupta, sabe que debe re-iniciar y lo más crítico es que se debe tener todo dispuesto para ejecutar a partir exactamente después del último *commit* exitoso.

De todos los movimientos para actualizar cuantos ya se procesaron y cuantos quedan pendientes.

PROCESO DE RE-ARRANQUE. -

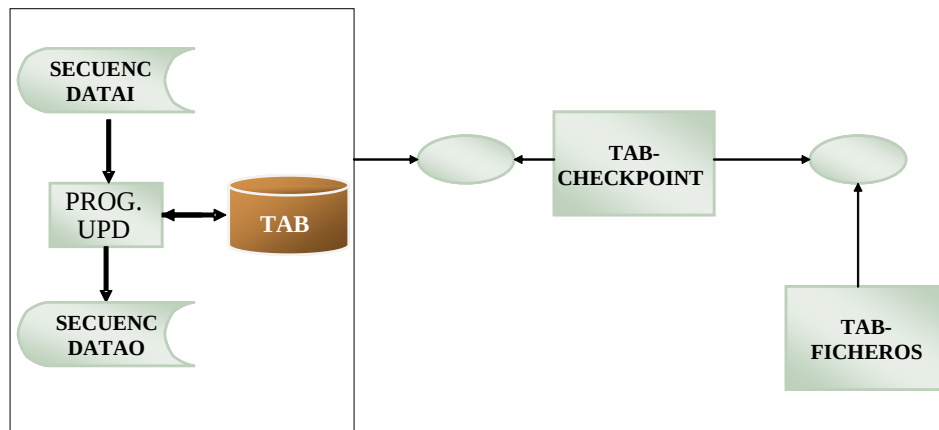
EN BASE A LA INFORMACIÓN DE

- NRO DE COMMIT SE CONOCE CUANTOS REGISTROS DE ENTRADA FUERON PROCESADOS.
- CON LA INFORMACIÓN RESPECTO A LOS ARCHIVOS PLANOS SE PODRÁ REALIZAR POR CADA UNO LA REGENERACIÓN CORRESPONDIENTE.

PARA LO CUAL EN BASE A LA CANTIDAD INDICADA EN LA TABLA PARA CADA ARCHIVO, SE REALIZA LA COPIA HACIA UN ARCHIVO TEMPORAL Y DE ALLÍ SE RETORNA NUEVAMENTE AL ARCHIVO ORIGINAL.
DE ESTA MANERA SE RESTAURA EXACTAMENTE LOS VALORES DE LOS PARAMETROS AL ULTIMO COMMIT OK DEL PROCESO.

NOTA.- PARA ESTE PROCESO SE REALIZA LA COPIA INVOCANDO AL PROGRAMA UTILITARIO CON LOS PARÁMETROS CORRESPONDIENTES.

PROCESO



Para el re-arranque :

- 1.- Se Lee de DATAI igual al de commits
- 2.- Se Graban en DATAO igual a la cantidad

FICH-NUM-REGISTROS

En este esquema queda clara la conexión del módulo del Check-Point con los demás módulos aplicativos.

El programa actualizador presenta su medio ambiente conformado por:

- 1.- Archivo de movimiento secuencial DATAI
- 2.- Tabla a ser actualizada TAB
- 3.- Archivo de salida.- Este archivo se regenera copiando del actual solo la cantidad de registros indicado en la tabla de ficheros. DATAO
- 4.- Módulo del Check-Point.- Desde el programa actualizador se realiza la llamada hacia este módulo con los parámetros correspondientes.

Cuando ocurre un fin de programa abrupto los parámetros indican esta situación por ejemplo:

Estado Check-Point	= ' R ' (Re-arranque)
Tipo-Ejecución	= ' U ' (No es la primera vez)
Tipo-Inicio	= ' N ' (Normal)

INICIO	EVENTOS				
CONDICIONES	E1	E2	E3	E4	E5
TIPO-EJECUCION:					
PRIMERA(P)	P				
NO-PRIMERA(U)		U	U	U	X
TIPO-INICIO					
FORZADO(F)		F			
NORMAL(N)			N	N	X
EST-RE-ARRANQUE					
NORMAL(N)			N		
RE-ARRANQ(R)				R	X
ALTA-CHECKPOINT	✓				
UPD-CHECKPOINT		✓	✓		
BAJA-FICHERO	✓	✓	✓		
ALTA-FICHEROS		✓	✓		
COMMIT	✓	✓	✓		
REPOSICIONAR				✓	
MSG-ERROR					✓

Que de acuerdo a la tabla de verdad adjunta se debe ejecutar la operación de **Reposicionar** que a su vez se descompone en los siguientes métodos:

- 1.-Leer-entrada **DATAI** la cantidad de registros igual a la cantidad actualizada.
- 2.-Grabar-Salida **DATAO** la cantidad indicada en la tabla de ficheros FICH-NUM-REGISTROS.

Después de lo cual se continúa con el proceso aplicativo de manera normal.

Otros eventos pueden ser:

Primera-Vez :Crear-Check-Point, Baja-Fichero, Commit

No-Primera-Vez: Actualiza-Tabla-Check-Point, Commit, Baja y Alta de ficheros.

GENERACIÓN DE CONSULTA CRÍTICA

EJEMPLO: Se desea diseñar el proceso batch para generar el archivo suplementario que resuelve la consulta crítica para una entidad bancaria. En este caso se requiere que solo sea para consulta y será generado con frecuencia diaria. El diseño de registro lógico es el siguiente:

REG-CLTE.:

TIPDOC	CHAR (02)
NRODOC	CHAR (08)
FECNAC	NUM (08)
NOMBRE	CHAR (35)
DIRECC	CHAR (35)
TELEF	NUM (10)
<u>NROCTAS</u>	NUM (02)
CUENTAS	VECTOR VARIABLE 1 A 99 ELEMENTOS
	DEPENDEN <u>NROCTAS</u>
NROCTA	CHAR (10)
TIPO	CHAR (02)
SALDO	NUM (13)

NOTA: ESTE ES UN CASO DE UN ARCHIVO DE LONGITUD VARIABLES. DONDE PRESENTA UN PARTE FIJA DE 100 BYTES Y OTRA VARIABLE DE 25 BYTES .

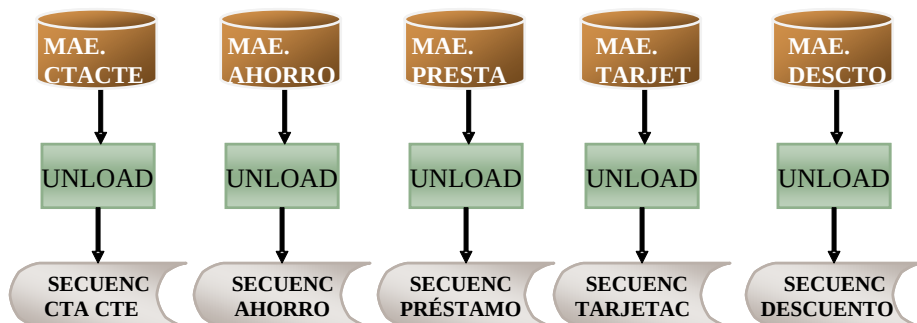
Para ilustrar esta actualización con quiebre, describiremos el proceso para la generación vía *batch* de la tabla que resuelve la consulta crítica presentada previamente.

El primer paso es conseguir los datos de entrada, los cuales se originan de las tablas maestras de las diferentes aplicaciones, como CTASCTES, AHORROS, etc. Todas creadas con el mismo formato conteniendo datos comunes.

GENERACION DE CONSULTA CRÍTICA

DESCARGA DE BASE DE DATOS.-

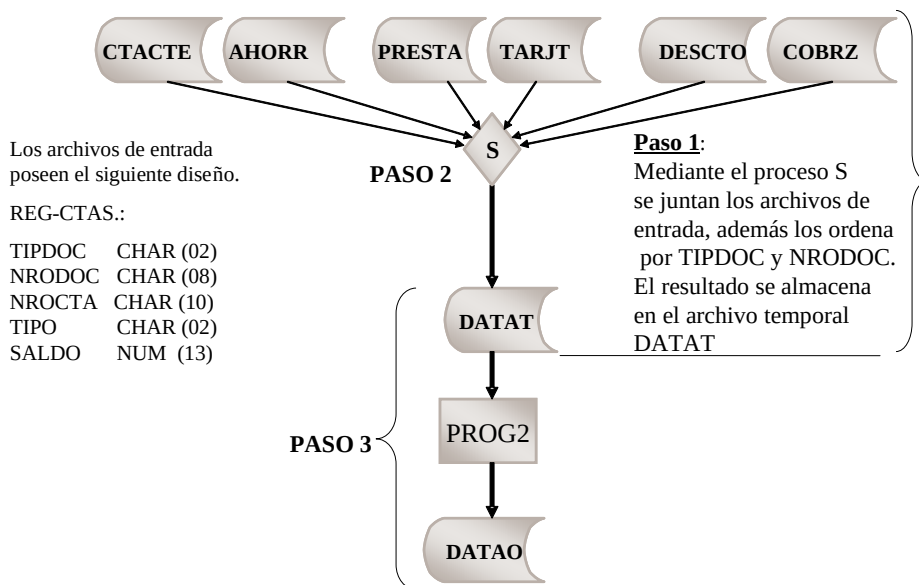
PASO 1



REG-CTAS.

TIPDOC	CHAR (02)
NRODOC	CHAR (08)
NROCTA	CHAR (10)
TIPO	CHAR (02)
SALDO	NUM (13)

DISEÑO CONSULTA CRÍTICA



Seguidamente cada de estas fuentes se juntan y se ordenan en base a la variable unificadora como la identificación del cliente y detallando cada una de sus cuentas.

El resultado será la tabla intermedia **DATAT**

Con la información así dispuesta el programa central tendrá el esquema de actualizador con quiebre, puesto que irá acumulando y al cambio de cliente se grabará la tabla.

DATA0.- El archivo de salida tendrá un diseño de registro ya indicado

REG-CLTE.:	
TIPDOC	CHAR (02)
NRODOC	CHAR (08)
FECNAC	NUM (08)
NOMBRE	CHAR (35)
DIRECC	CHAR (35)
TELEF	NUM (10)
<u>NROCTAS</u>	NUM (02)
CUENTAS	VECTOR VARIABLE 1 A 99 ELEMENTOS
	DEPENDEN <u>NROCTAS</u>
	NROCTA CHAR (10)
	TIPO CHAR (02)
	SALDO NUM (13)
REG-CLTE.:	
TIPDOC	CHAR (02)
NRODOC	CHAR (08)
FECNAC	NUM (08)
NOMBRE	CHAR (35)
DIRECC	CHAR (35)
TELEF	NUM (10)
<u>NROCTAS</u>	NUM (02)
CUENTAS	VARCHAR(2575)

Como resultado del proceso se obtiene la tabla DATA0 con el formato que se muestra. Se puede observar en el formato final que el arreglo variable se almacena en un VARCHAR.

GENERACION DE CONSULTA CRÍTICA

BATCH: PROG2

QUIEBRA CUANDO CAMBIA DE CLIENTE

SEUDO-CODIGO:

PROGRAMA.-

INICIO
PROCESO HASTA FIN-ARCH
FIN.

INICIO.-

INICIALIZA-VAR
PREPARA-ARCHIVOS
LEE-MOV

PROCESO.-

SI TIP-DOC = TIP-DOC-ANT
& NRO-DOC = NRO-DOC-ANT
 ACUMULAR
SINO
 QUIEBRE
FIN-SI.
LEE-MOV.

ACUMULAR.-

I = I + 1
W-NROCTA(I) = NRO-CTA
W-TIPCTA(I) = TIP-CTA
W-SALDO(I) = SALDO
NROCTAS = I

QUIEBRE.-

LEE-MAESTRO
GRABA-SALIDA
INICIALIZA-VAR

LEE-MAESTRO.-

LEER-PERSONAS
S-FECNAC = P-FECNAC
S-NOMBRE = P-NOMBRE
LEER-DIRECC
S-DIRECC = D-DIRECC
S-TELEF = D-TELEF

FIN.-

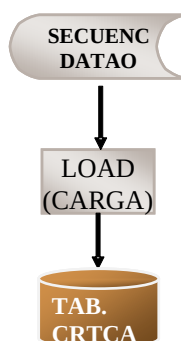
LEE-MAESTRO
GRABA-SALIDA.-
S-TIP-DOC = TIP-DOC-ANT
S-NRO-DOC = NRO-DOC-ANT

Se muestra el pseudo código aplicado para el caso de un proceso de actualización con quiebre.

GENERACION DE CONSULTA CRÍTICA

CARGA DE BASE DE DATOS

PASO 4



Finalmente se procede a realizar la carga de la tabla de la base de datos, partiendo de la tabla plana generada en el proceso *batch*.

Este proceso se realiza mediante un módulo utilitario de todo gestor de base de datos conocido como load.

Previo a esta carga se tiene definida la tabla en el gestor de la base de datos.

En base a los parámetros definidos, el utilitario inicia el proceso, determinando la creación de claves primarias, secundarias y foráneas. Además del porcentaje libre a respetar para el IC como para el AC.

Finalmente el tamaño del bloque IC y de su AC:

Existen características adicionales que se presentarán en los próximos capítulos como las restricciones o reglas de negocio de los datos y las reglas de integridad para las claves foráneas.

Todos estos detalles son tomados en cuenta por el módulo de carga, con opciones para la carga como adicionar o re-emplazar datos contenidos actualmente entre otros.

3.2.2.2.2. Matching simple y dobe

ACTUALIZACIÓN DE ARCHIVOS MAESTROS (Matching)

Después de presentar el modelo de actualización con quiebre, es el momento de plantear el caso más especial conocido como **Matching** de comparación de archivos.

Este esquema es muy útil y óptimo para eventos en los cuales nos vemos precisados para actualizar grandes volúmenes de una gran tabla, por ejemplo de una tabla que puede contener un millón de registros es necesario actualizar al 80% aproximadamente.

El modelo anterior también podría ser aplicado sin embargo no sería el más óptimo en referencia al modelo de **Matching**.

El esquema parte del hecho que si se tuviera que actualizar el 100% de una tabla no sería necesario utilizar índice puesto que todos sin excepción serían tratados.

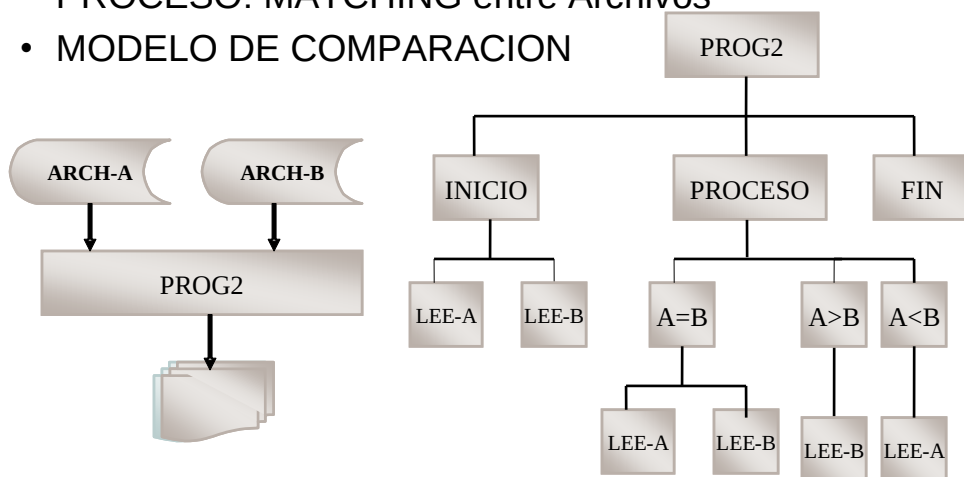
Solo sería necesario que ambas tablas fueran ordenadas en la misma secuencia.

Luego si este esquema es válido para el caso del 100%, también debe serlo para porcentajes cercanos a este vale decir 90, 80, 70 hasta el 60%.

La premisa para que así sea considerado es considerar que la lectura secuencial tendrá una alta probabilidad de encontrar el registro del match.

Procesos en Lotes

- PROCESO: MATCHING entre Archivos
- MODELO DE COMPARACION



- 1.- $A = B$ PROCESAR REG. IGUALES LEER-A LEER-B
- 2.- $A < B$ PROCESAR REG(A) LEER-B
- 3.- $A > B$ PROCESAR REG(B) LEER-A

FILE-A

NROFAC	SALDO
010	S1
020	S2
030	S3
040	S4
050	S5
060	S6
070	S7
080	S8
090	S9
100	S10
110	S11
120	S12
130	S13
140	S14
150	S15
160	S16

n

FILE-B

NROFAC	MONTO
010	M1
030	M2
050	M3
060	M4
080	M5
120	M6
140	M7
150	M8

m

Movimiento

Pendiente

Se muestra el caso típico, dos tablas secuenciales conteniendo facturas pendientes y pagos ejecutados, ambas tablas tienen el mismo parámetro de identificación (número de factura) y están ordenadas en la misma secuencia.

Luego la búsqueda se realizará a partir de la tabla de movimiento de pagos y se compara contra el registro correspondiente de la pendiente.

BATCH: MACHING DE ARCHIVOS

Caso de uso: Dado un archivo de “n” facturas pendientes debe descargar aquellas “m” que ya fueron canceladas en otras sucursales o puntos de atención.

Una solución puede ser que por cada factura pagada se acceda a su pendiente y darle de baja.

Esta propuesta se puede implementar de la siguiente manera:

1. Si ambos archivos son secuenciales. Por cada movimiento leer en la pendiente de manera secuencial hasta encontrarla
Costo: Se realizarán $(m+m*n/2)$ accesos.
2. Si la pendiente es indexada y el movimiento es secuencial. Por cada movimiento acceder por índice a la pendiente.
3. Costo: Se realizarán “m” accesos secuenciales y “m” indexadas.
4. Recordar que por cada acceso indexado se requieren 3 accesos en promedio

La elegancia del modelo de **Matching** se refuerza con lo óptimo que resulta en cuanto al número de accesos que deben realizarse al disco.

Esta característica se demuestra por el hecho de utilizar en formato **batch** (vale decir utilice un tamaño de bloque grande).

Procesos en Lotes

BATCH:

MATCHING DE ARCHIVOS.

SEUDO-CODIGO:

PROGRAMA.-

INICIO

PROCESO HASTA FIN-A V FIN-B
FIN.

INICIO.-

INICIALIZA-VAR

PREPARA-ARCHIVOS

LEE-A

LEE-B

PROCESO.-

PROCESO.

SI

$K-A = K-B$

PROCESAR-IGUALES

LEE-A

LEE-B

SINO

SI

$K-A > K-B$

LEE-B

SINO

LEE-A

FIN-SI

FIN-SI

FIN.-

SI

FIN-A & FIN-B

ESTADISTICAS

SINO

SI

FIN-A

PROCESAR-B

SINO

PROCESAR-A

FIN-SI

ESTADISTICAS

FIN-SI

Aprecia-se el pseudo código para el esquema de **Matching**.

La sencillez y elegancia del programa hace que sea de fácil desarrollo y rápido mantenimiento.

Resumen del esquema:

1.- Homologar variable utilizada para el match $KA = KB$

2.- Definir bloques grandes para los IC de las tablas.

3.- Ordenar las tablas en secuencia de KA y KB

4.- Avanzar leyendo ambos archivos de manera que coincidan sus claves de comparación.

En la rutina PROCESAR-IGUALES deber realizarse la actualización respectiva.

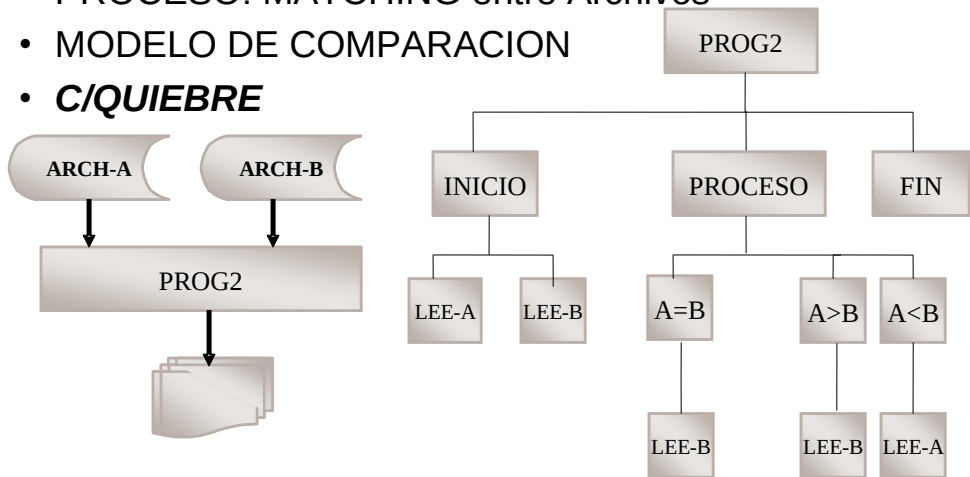
Además típicamente se emite un reporte informativo.

Las demás rutinas realizan operaciones comunes.

Para ordenar las tablas se recurren a utilitarios de soporte conocido como SORT o clasificador.

Procesos en Lotes

- PROCESO: MATCHING entre Archivos
- MODELO DE COMPARACION
- **C/QUIEBRE**



- 1.- $A = B$ PROCESAR REG. IGUALES LEER-A LEER-B
- 2.- $A < B$ PROCESAR REG(A) LEER-B
- 3.- $A > B$ PROCESAR REG(B) LEER-A

FILE-A

NROFAC	SALDO
010	S1
020	S2
030	S3
040	S4
050	S5
060	S6
070	S7
080	S8
090	S9
100	S10
110	S11
120	S12
130	S13
140	S14
150	S15
160	S16

FILE-B

NROFAC	MONTO
010	M1
030	M2
030	M3
060	M4
080	M5
120	M6
120	M7
150	M8

Una variante al esquema original, es el llamado Quiebre con **Matching**, es decir una vez que se ha dado el match (igualdad de parámetros), se sigue procesando todos los movimientos que sean iguales al registro del match.

Cuando se cambia de parámetro del match se produce el cambio o quiebre.

BATCH: MATCHING DE ARCHIVOS CON QUIEBRE

Caso de Uso: Dado un archivo de “n” facturas pendientes debe descargar aquellas “m” que ya fueron canceladas en otras sucursales o puntos de atención. Además puede haber varios pagos para una misma factura pendiente.

Se aplica el modelo de matching con una variante, cuando se realiza el match se seguirá leyendo el movimiento hasta que se cambie de factura.

Procesos en Lotes

BATCH:**MATCHING DE ARCHIVOS.****SEUDO-CODIGO:**

PROGRAMA.-	PROCESO.-	FIN.-
INICIO	PROCESO.	SI FIN-A & FIN-B
PROCESO HASTA FIN-A V FIN-B	SI K-A = K-B	ESTADISTICAS
FIN.	PROCESAR-IGUALES	SINO
	LEE-B	SI FIN-A
INICIO.-	SINO	PROCESAR-B
INICIALIZA-VAR	SI K-A > K-B	SINO PROCESAR-A
PREPARA-ARCHIVOS	LEE-B	FIN-SI
LEE-A	SINO	ESTADISTICAS
LEE-B	LEE-A	FIN-SI
	FIN-SI	
	FIN-SI	

Nuevamente se observa la sencillez y elegancia del pseudo código y a la vez la potencialidad durante el proceso.

La diferencia entre los dos esquemas es solo una línea, después de PROCESAR-IGUALES, solo se avanza en leer el movimiento y no la pendiente.

3.2.2.3. Acumulación de movimientos históricos

ACUMULACIÓN DE MOVIMIENTOS

Todo hecho de la realidad que ocurre en un sistema lo impacta de alguna manera en sus tablas maestras por que debe quedar debidamente registrado.

El conjunto de estos hechos o movimientos de cuentas configuran el historial de las cuentas y explican el comportamiento de las mismas.

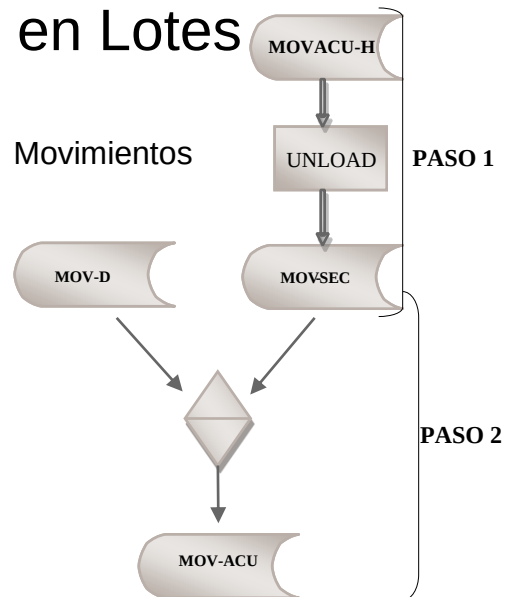
Dentro de un período de tiempo dado, por medio de estos hechos se describen como se han venido dando las operaciones del negocio, de allí su importancia de su recopilación y posterior estudio.

Todo negocio tiene la obligación estratégica y táctica de acumular estos hechos o movimientos, en primer lugar para demostrarles a sus clientes sobre los saldos que presentan sus cuentas en un momento determinado, y en segundo lugar para obtener información de patrones de comportamientos de sus clientes respecto a sus gustos y preferencias.

Presentamos las técnicas para acumular movimientos.

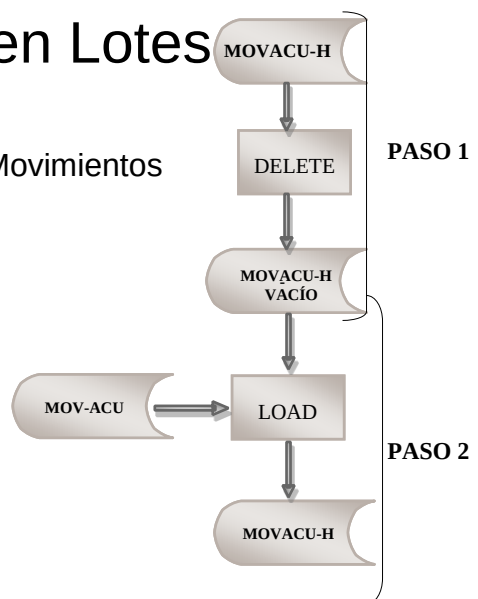
Procesos en Lotes

- MODELO EN CASCADA
- PROCESO: Acumulación de Movimientos
- Paso 1: UNLOAD=DESCARGA
- Paso 2: ADICIONAR-MOV-D



Procesos en Lotes

- MODELO EN CASCADA
- PROCESO: Acumulación de Movimientos
- Paso 3: BORRAR TAB
- Paso 2: CARGAR TAB
- (Todos los pasos son re-ejecutables)



El proceso consiste en ir sumando los movimientos diarios en un archivo acumulado del mes o algún periodo más largo de acuerdo al ciclo de vida del negocio. Luego se debe adicionar el movimiento de hoy al movimiento acumulado hasta el día de ayer. Todo este proceso se debe llevar de manera automática por medio de programas utilitarios, vale decir no amerita desarrollar programas personalizados.

3.2.2.4. Recuperación de archivos

PROCESO DE RECUPERACIÓN DE MAESTROS

El tema de recuperación de tablas maestras trata sobre la manera que se debería proceder para enfrentar la casuística de una eventual destrucción voluntaria o involuntaria de una parte de la base de datos.

Esta recuperación debe ser automática porque de lo contrario sería sumamente tedioso y arto costoso.

Considerando que los saldos de la tabla maestra son resultado de los hechos o movimientos dados, luego esta reconstrucción a pesar de ser compleja y delicada es factible y medianamente onerosa.

Se supone que solo se debe llegar a esta situación después de haber sido violadas todas las medidas de seguridad sobre el sistema.

Estos procesos forman parte del módulo de contingencia que siempre debe estar listo para actuar cuando sea requerido, de manera eficaz y segura.

A continuación presentamos los detalles de estos procesos.

BATCH: PROCESO DE RECUPERACIÓN DE MAESTROS

Caso de Uso: Los módulos transaccionales actualizan directamente a disco, en ocasiones pueden ocurrir problemas que afectan los archivos maestros, por causas de hardware ocurridos con el disco entre otros.

Para estos casos se debe activar el proceso de recuperación de archivos

Pasos a seguir:

1. Restaurar el backup del archivo maestro hasta el día (D) anterior al desastre.
2. Recuperar los movimientos correspondientes al día siguiente del backup (D+1).
3. Ejecutar un programa que procese las operaciones originadas en tiempo real, en modo batch.

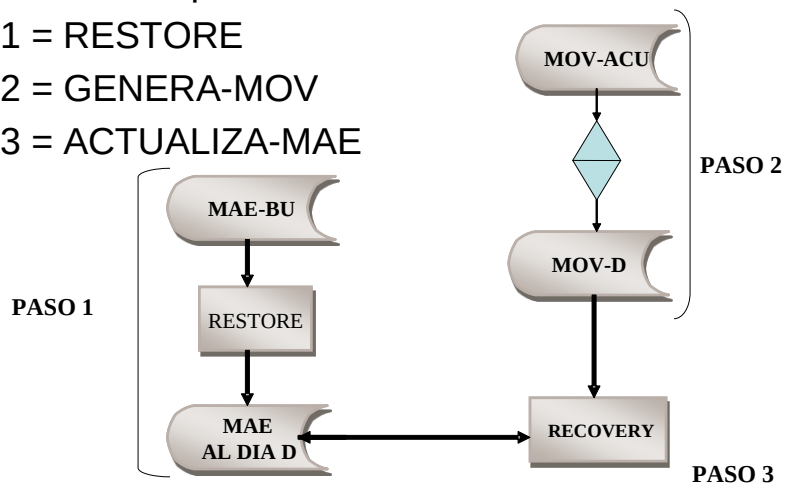
Normalmente la captura de los movimientos se realiza en tiempo real y se graban en tablas del sistema después de haber actualiza tablas maestras.

Cuando se debe reconstruir estas tablas maestras se procede de manera semejante pero en modalidad **batch**.

El programa actualizador en línea debe ser adaptado a la modalidad batch.

Procesos en Lotes

- MODELO EN CASCADA
- PROCESO: Recuperación del maestro.
- PASO1 = RESTORE
- PASO2 = GENERA-MOV
- PASO3 = ACTUALIZA-MAE



Si bien es cierto el esquema del proceso se muestra sencilla es muy delicada.

El programa actualizador en **batch** debe tener la misma funcionalidad que el de on line, adaptado a la modalidad **batch**, esto significa adecuarlo a su nuevo medio ambiente. Entiéndase como dispositivos de entrada / salida, arquitectura cliente-servidor.

Como se desarrolla en **batch**, se torna un medio ambiente centralizado y sus archivos de entrada / salida deben presentar formato **batch**.

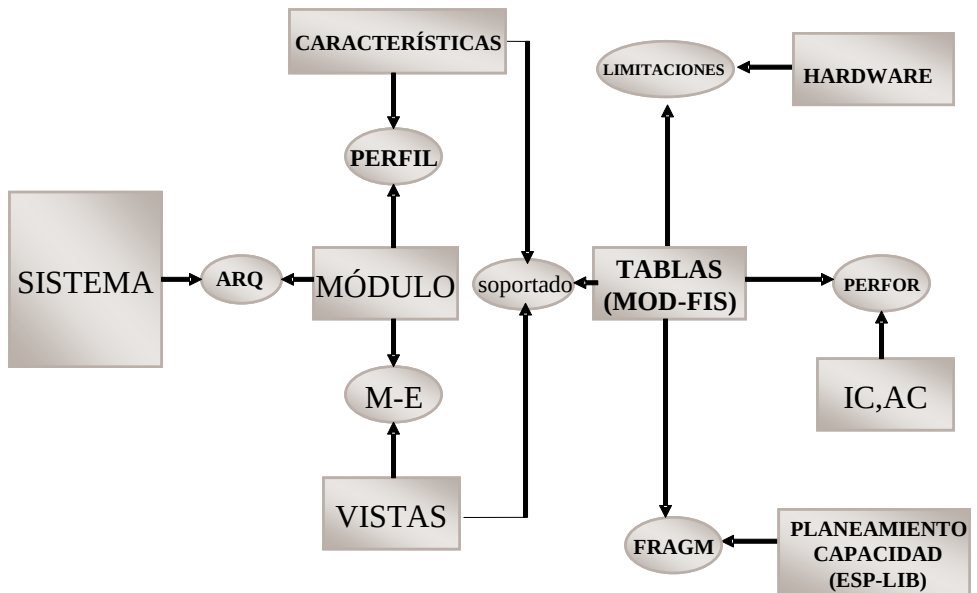
Como se trata de procesos de grandes lotes deben generarse cuadros de totales por diferentes parámetros.

En cuanto a las tablas maestras estas deberán tener los datos del último evento de actualización como fecha, a partir de la cual se deberá proceder la actualización.

Otros términos utilizados para este proceso es el de “Sincronismo” dando a entender que las tablas maestras deben estar afectadas con sus movimientos hasta el día de su último proceso.

La importancia de estos procesos a veces no es apreciada sino hasta cuando son requeridos, deben ser considerados como los Seguros de Vida del Sistema.

SÍNTESIS DEL DISEÑO



Después de presentar los diferentes temas para plantear un diseño de base de datos, que sea considerado óptimo, pasaremos a sintetizar todo lo dicho anteriormente en el presente libro, con la ayuda del gráfico precedente a modo de modelo conceptual del diseño.

1.- Arquitectura-Sistemas = { Módulos }

2.- Modelo-Externo = { Vistas }

3.- Perfil-Sistemas = { Características }

4.- Características, Vistas → Soportadas (Modelo-Físico)

5.- Modelo-Físico → Limitaciones (Hardware)

El Modelo Físico de cara a la realidad está limitado por el hardware

6.- Modelo-físico → Performance (IC,AC)

El rendimiento de la base depende fundamentalmente del IC y AC

7.- Fragmentación-Física → Minimiza (Espacio-Libre)

El parámetro de Espacio Libre minimiza las operaciones de fragmentación de la base.

3.3. Metodología para la optimización

METODOLOGÍA DE MEJORA DE RENDIMIENTO

Recomendaciones a seguir en el diseño

- 1.- Reglas de Negocio realistas.
- 2.- Crear Claves e Índices necesarios.
- 3.- Des-normalizar para casos críticos
- 4.- Diseño de procesos en base a esquemas ya probados.
- 5.- Administración física de discos para cada tipo de información.
- 6.- Uso de vistas
- 7.- Optimizar Recursos: buffer, blocks y extents.
- 8.- Escribir el SQL óptimo, cuidado con la Contención.

A continuación se presenta uno de los temas más importantes en desarrollo de sistemas, porque tiene que ver con la optimización del mismo.

En base a toda la teoría presentada en el presente material deduciremos las recomendaciones para asegurar un diseño óptimo.

Las recomendaciones cubren temas tan diversos como desde reglas de negocio hasta programación **SQL** óptimo pasando por diseño lógico de tablas.

Más que intentar recomendar un conjunto de *tips*, sintetizamos todo lo expuesto en un conjunto de ocho reglas, tratando de cubrir los ángulos típicos que atentan contra la performance.

Se requiere una persona que pueda manejar de manera integral el sistema.

Esto quiere decir conocer las reglas del negocio y conocer todo el tema informático como el análisis, diseño, desarrollo e implantación.

A continuación se detalla cada una de estas recomendaciones con su respectivo sustento.

3.3.1. Reglas de negocio realista

1.- Reglas de Negocio Realistas

R.N.: "Atención a clientes las 24 horas, los 7 días de la semana"

- Será verdad, cómo y cuándo actualizan movimientos externos, liquidaciones mensuales, ajustes y otros?
- Todos estos procesos serán ejecutados con copias temporales de los movimientos transaccionales. En cuanto a los movimientos que se dieron durante la actualización del lote transaccional, dado que son en menor cantidad serán actualizados en línea, en la maestra transaccional. El periodo de tiempo debe ser de algunos minutos y en horas de poco movimiento transaccional, para poder suspender el servicio.

3.3.2. Crear claves e índices necesarios

2.- Claves e Índices

Acceso ordenado según criterios de negocios a las tablas.

- Aseguran que no exista valor duplicado si se define como único.
- Son usados automáticamente cuando se hace referencia en las cláusulas Where. (Oracle).
- Son muy buenos para acceso por llaves alternativas. Si faltan baja la performance.
- Es importante verificar si el RDBM lo está usando. Si sobran ocupan espacio
- Generan sobrecarga de trabajo al RDBM y requieren espacio.

3.3.3. Desnormalizar para casos críticos

CONSULTAS CRÍTICAS

- ¿ Porqué alguna transacción debería ser considerada crítica ?
- Ser parte del frente de atención al cliente.
- Ser de las más usadas en el sistema de información.
- Tener un tiempo de respuesta inaceptable por el usuario.
- Ser una transacción batch para una actualización masiva de Base de datos, donde el tiempo disponible para todo el proceso es menor que el requerido.
- Identificar las transacciones críticas.



- Crear redundancia de manera consiente y controlada.
- Sugerencia.- Además de la base de datos original (normalizada) crear tablas extras de consultas con fines precisos y prácticos, con datos preparados evitando navegación a otras tablas. Generalmente estas tablas se crean diariamente.
- **Realizar un inventario de todas las transacciones que atenderá la base de datos.**

3.3.4. Diseño en base a esquemas ya probados

4.- Diseño de Procesos

- Procesos Transaccionales
 - Cadena de Simple de Transacciones
- Procesos en lotes
 - Liquidaciones / Facturaciones / Estados de cuenta
 - Múltiples Entradas / Múltiples Salidas
 - Acceso secuencial al movimiento(no usar índice)
 - Usar Puntos de control
 - Actualización de Maestras con quiebre
 - Actualización de Movimiento-Pendiente
 - Match files

3.3.5. Administración física de discos

5.- Administración física de archivos

- Información de Maestros.- Discos con niveles de seguridad apropiados su crecimiento es planificado en un mediano plazo.
- Información de Movimientos. - Gran capacidad de almacenamiento tratar de estimar físicamente para el nuevo día.
- Información de Históricos. - Deben ser generados en modalidad batch por cada pase de Movimiento -diario al histórico.
- Información de Parámetros y Tablas de preferencia en memoria residente .

3.3.6. Uso de vistas

6. - Uso de Vistas

- Se usan para:
 - Presentar las tablas a los usuarios como una sola tabla.
 - Pre empacar consultas complejas.
 - Restringir accesos.
- Notas:
 - Una vista no tiene datos propios, utiliza las tablas.
 - Una vista es como una tabla virtual.
 - Es más lento el acceso a través de las vistas. Acceden al diccionario de datos.

3.3.7. Optimizar recursos, Buffer, Blocks, Extents

7.- Optimizar Recursos

CARACTERISTICAS DEL IC

- 1.- Número de registros lógicos
- 2.- Número de sectores físicos.
- 3.- Porcentaje Libre

FUNCION: ALMACENAMIENTO Y TRANSFERENCIA (DISCOS $\Leftrightarrow$ RAM)

CARACTERISTICAS DEL AC

- 1.- Número de Intervalos de Control
- 2.- Número de pistas físicas.
- 3.- Porcentaje Libre

FUNCION: ALMACENAMIENTO Y EXPANSIÓN DEL ARCHIVO

Parámetros Críticos

- **Intervalo de Control.-** Agiliza o retarda el tráfico de datos entre discos y memoria
- **Espacio Libre.-** Permite crecimiento ordenado evitando reacomodos de páginas y cilindros
- **Áreas de Control.-** Para grandes volúmenes deben fijarse a cilindros

3.3.8. Escribir SQL óptimo cuidado con contención

8.- Escribir el SQL óptimo

```
SELECT <atributos>  
FROM <tablas>  
WHERE <condiciones>  
GROUP BY <atributos>  
HAVING <condiciones de grupo>  
ORDER BY <atributos>
```

Considerando la estructura del SQL, por lo menos dos son los parámetros a tener presente para optimizar la sentencia:

FROM indica las tablas desde donde se realizará la búsqueda. Se recomienda que el orden de las tablas sea considerando su importancia (de izquierda a derecha).

WHERE indica las condiciones que servirán de filtro. Estas condiciones típicamente consisten en comparar atributos, estas comparaciones pueden ser de =, >, <, etc.

Por ejemplo T1.A = T2.A, si el atributo "A" es clave en ambas tablas la búsqueda será óptima, en caso contrario la búsqueda se realizará en toda la tabla lo cual degradará la performance.

En este punto demostramos la manera óptima de escribir una sentencia **SQL**.

El manejador de la base de datos tiene por naturaleza procesar volumen de datos, luego en el parámetro **FROM** se espera un grupo de tablas separadas por comas.

El orden de estas tablas determina cual será considerada la tabla principal del proceso.

Para ser considerada la tabla principal debe estar en el primer lugar después del **FROM**.

Se recomienda antes de escribir el comando determinar esta característica.

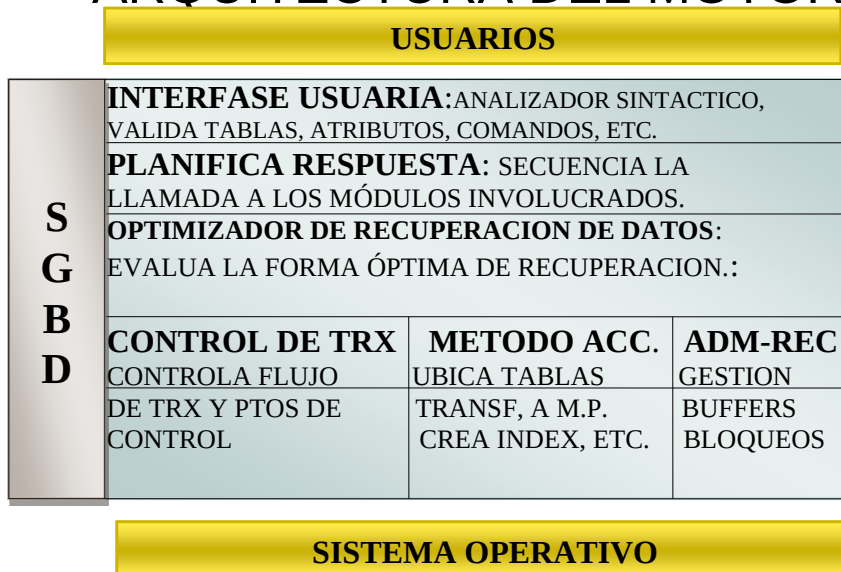
El acceder a tablas debe de preferencia ser vía clave, el atributo clave (que ha generado su índice) debe iniciar la lista en el parámetro **WHERE**

Estas pautas se amparan en el hecho de optimizar la navegación entre las tablas, buscando el camino más corto.

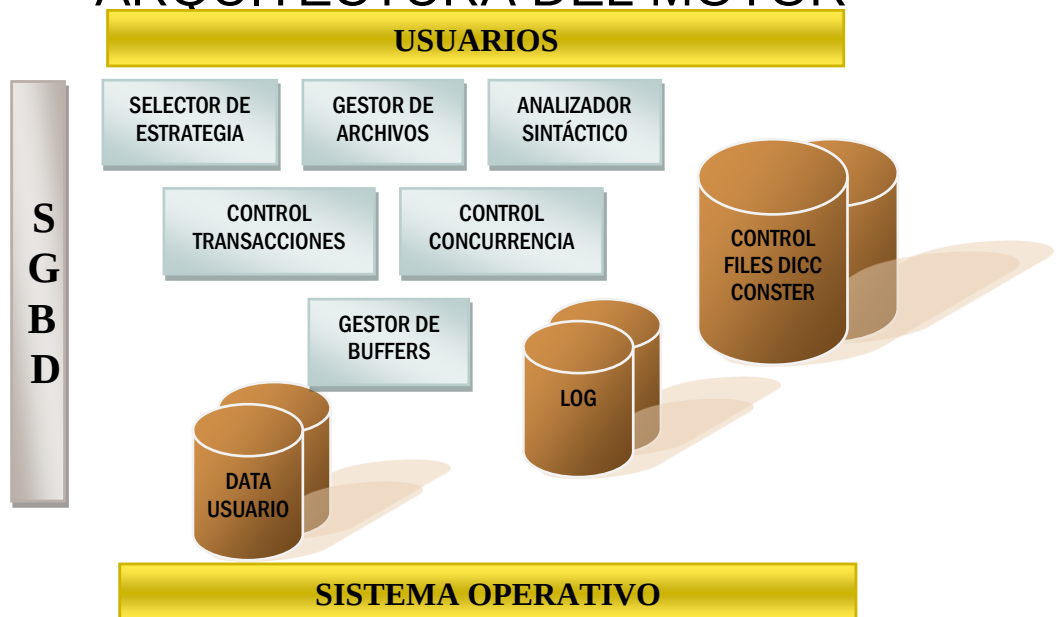
4. Arquitectura de la base de datos

ARQUITECTURA MOTOR DE BASE DE DATOS

ARQUITECTURA DEL MOTOR



ARQUITECTURA DEL MOTOR



Seguridad de BD

Control de Integridad

(Data coherente con el Negocio)

INTEGRIDAD DE DOMINIOS

NATURALEZA DE DATOS

CONTROLES

INTEGRIDAD DE CLAVE

UNICIDAD

REFERENCIAL

PROCEDIMIENTOS ALMACENADOS

DISPARADORES

Control de Disponibilidad

Gestor de Memoria Intermedia

Gestor de Transacción

Gestor de Fallas

Control de Confidencialidad

Perfil de usuario

Identificar y autenticar

4.1. Control de integridad

CONTROL DE INTEGRIDAD

- INTEGRIDAD DE DOMINIOS
 - NATURALEZA DE DATOS
 - CONTROLES
- INTEGRIDAD DE CLAVE
 - UNICIDAD
 - REFERENCIAL
- PROCEDIMIENTOS ALMACENADOS
 - DISPARADORES

4.1.1. Integridad de dominio

NATURALEZA DE LOS DATOS

- Se validan los tipos de datos válidos cuando son registrados en la base de datos, como:
- Caracter(longitud fija/variable), Numérico, Fecha, y Nulo.
- Es el primer nivel de validación antes de alterar el contenido de base de datos.
- Los valores tabulados restringen dominios de las variables.
- En algunos SGBD se definen dominios para ciertas variables es equivalente al anterior pero automático(create domain dominio1 char(8)).

- Valor NULO no almacena valor alguno.
- Caracter: Dependiendo de la plataforma puede ser implementado con código ASCII o EBCDIC. En ambos casos se utiliza un byte con 256 combinaciones permite representar letras, dígitos y caracteres especiales. Ejemplo:

	Dígitos			
• Código	0	1	2	9
• ASCII	40	41	42	49
• EBCDIC	C0	C1	C2	C9

Naturaleza de Datos

- Numéricos
 - Base 10.- Código ASCII o EBCDIC se utiliza medio byte para el signo en el último byte.
 - C = Positivo, D = Negativo, F = S/signo
 - Desempacados.- Almacena tipo caracter, signo en zona izquierda.
 - Ej.: -12345 internamente será

F1	F2	F3	F4	D5
----	----	----	----	----
 - Empacados.- Almacena sólo el dígito requiere medio byte por dígito, signo en zona derecha.
 - Ej.: -12345 internamente será

12	34	5D
----	----	----

Naturaleza de Datos

- Numéricos
 - Base 2.- Coma Flotante, se almacenan en 4 y 8 bytes dependiendo de la precisión del número.
 - Formato.- SMBE donde: S=Signo, M=Mantisa, B=Base, E=Exponente

S	E	mm	mm	mm
---	---	----	----	----

 4 bytes
 - Para un número de 4 bytes. El primer byte de la izquierda almacena signo y exponente, la base es 16. Los 3 bytes siguientes almacenan la mantisa.

S	E	mm	mm	mm
---	---	----	----	----
 - El bit signo es el primero de la izquierda,

S	E	mm	mm	mm
---	---	----	----	----

0= positivo, 1 = negativo.
 - Los 7 bits restantes(del primer bytes) almacenan exponente en exceso a 64.
 - De 1 al 64 son exponentes negativos del 65 al 127 son positivos
 - Ej.: -12345(10) equivale 3039(16) o $+0.3039 * 16^{**4}$ internamente se almacena

C	4	00	30	39
---	---	----	----	----

CONTROLES

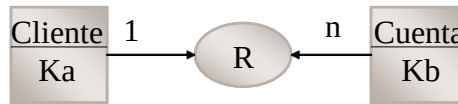
- Ejemplo:
 - create table CUENTA
 - (numero-cuenta char(10) not null,
 - fecha-apertura date,
 - saldo integer,
 - primary key(numero-cuenta)
 - check(saldo >= 0))
- La cláusula check especifica una condición que toda fila de la relación deberá respetar

4.1.2. Integridad de Clave

- Unicidad de Integridad: Clave Primaria.
- El concepto de Clave es mas semántico que físico. Atributo cuyo valor identifica a la fila y su valor es único en la relación. En el medio físico se define como key primaria como medio de acceso hacia un valor único.
- Típicamente está formada por un solo atributo.
- Está asociada a una Entidad semántica del modelo conceptual.

Integridad de Clave

- Integridad Referencial: Clave Foránea



Implementación
RESP

Key = Ka + Kb

Ka	Kb

Integridad de Clave

- Integridad Referencial: Clave Foránea
- Implementación
 - create table RESP
 - (Ka char (10) not null,
 - Kb char (10) not null,
 - primary key (Ka, Kb),
 - foreign key (Ka) references Clte,
 - foreign key (Kb) references Cta)
 - Operaciones:
 - Adición en RESP: Se valida que Ka exista en Clte y Kb exista en Cta.
 - Eliminación en Cta...:Se chequea si Kb existe en Resp.

En este caso la tabla RESP está compuesta por dos atributos que son claves primarias en otras tablas. Para que se respete la integridad de la base cuando se dé un insert en la tabla RESP deberá verificarse que ambas foráneas existan en sus respectivas tablas como claves primarias.

Integridad de Clave

- Integridad Referencial : Clave Foránea
 - Operaciones:
 - Insert: Key foránea
 - Update sobre una Key que es foránea en otra entidad.
 - Delete sobre una Key que es foránea en otra entidad.
 - Opciones
 - Restricción: No permite la acción.
 - Cascada: Elimina o modifica la key foránea según sea el caso.
 - Implementación:
 - create table RESP
 -
 - foreign key (Kb) references Cta)
 - on delete casacade
 - on update cascade, .

Una actualización en una clave foránea equivale a eliminarla e ingresar una nueva.

Eliminar una clave foránea por integridad puede ocasionar la eliminación una instancia en otra entidad y si en esta existe otra clave foránea podría generarse una eliminación en casada con lo cual se podría perder el control de los datos en algún momento.

Considerando la eliminación en cascada el peligro crece si se tiene una relación circular que podría terminar destruyendo buena parte de datos en un grupo de tablas.

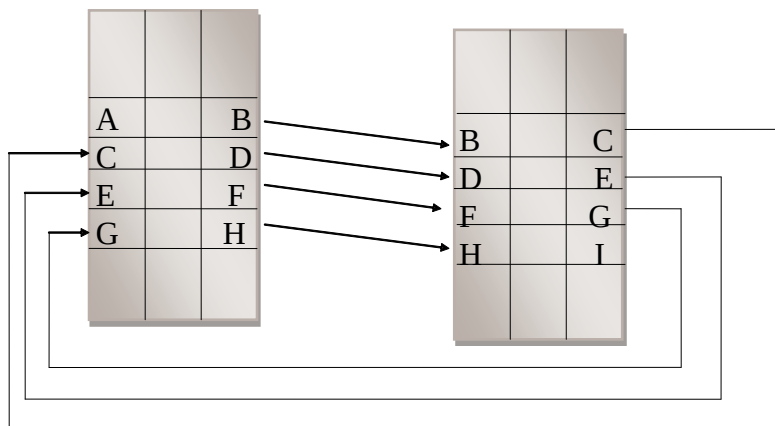
Se recomienda en un principio utilizar el parámetro de eliminación en cascada con restricción.

El diseño de claves es un punto determinante no solo asegurar buena performance en acceder a una tabla, sino para evitar claves duplicadas si se trata de claves primarias de tablas que es una relación de entidades.

El orden de las claves primarias en la clave resultante de la relación, determina el orden de las tablas a ser accedidas.

REFERENCIAS CIRCULARES

Se debe tener cuidado con las referencias circulares podrían destruirse la base de datos aplicando la eliminación en cascada



INTEGRIDAD DE DATOS

TIPO DE INTEGRIDAD. - El objetivo de este mecanismo es para proteger la calidad de datos en cuanto a su integridad semántica. Es decir, para que un conjunto de datos tenga sentido deben estar relacionados entre sí. Cuando uno de los componentes por error es borrado el conjunto perderá sentido.

Existen las siguientes restricciones:

NOT NULL: La columna debe tener valor

UNIQUE: Indica a la columna o combinación de columnas como única

PRIMARY KEY: Indica como la clave primaria de la tabla.

FOREIGN KEY: Columna de la tabla, es clave en otra.

CHECK: Indica una condición que cada fila debe respetar.

Estado de la Restricción

DISABLE NOVALIDATE: No controla data que ingresa, ni data en la BD.

DISABLE VALIDATE: No controla data que ingresa, si data en la BD.

ENABLE NOVALIDATE: Controla data que ingresa, no controla en BD.

ENABLE VALIDATE: Controla data que ingresa y data en BD.

En el caso de validar data en la base de datos, este proceso puede tomar un tiempo considerable, por lo que se recomienda su planificación.

4.1.3. Procedimientos almacenados

Procedimientos Almacenados

- Son rutinas escritas en SQL , almacenándose tanto el fuente como el ejecutable.
- Típicamente son procedimientos de uso común en el sistema. Insert, Update, Delete
- Se recomienda sólo almacenar los procedimientos cuya modificación difícilmente se dé.
- Otro uso frecuente es para controles de integridad y disparadores.

Procedimientos Almacenados

Se pueden utilizar para implementar las reglas del Negocio mediante trigger.

```
CREATE PROCEDURE upd_items_p1()
    DEFINE GLOBAL old_qly INT DEFAULT 0;
    LET old_qty = (SELEC SUM(quantity)FROM items);
END PROCEDURE
CREATE PROCEDURE upd_items_p2()
    DEFINE GLOBAL old_qly INT DEFAULT 0;
    DEFINE new_qty INT;
    LET new_qty = (SELEC SUM(quantity)FROM items);
    if new_qty > old_qty*1.50 then
        RAISE EXCEPTION-746.0 "Not allowed-rule
        violation;";
    END IF
END PROCEDURE
CREATE TRIGGER up_items
UPDATE OF quantity On items
BEFORE(EXECUTE PROCEDURE upd_items_p1())
AFTER(EXECUTE PROCEDURE upd_items_p2())
```

¿Por qué procedimientos Almacenados?

El lenguaje SQL

Select a,b,c,d from Tabla A using (b)

- Tiempo para interpretar el lenguaje.
- Tiempo para verificar la existencia de Tablas.
- Tiempo para verificar la correspondencia de atributos.
- Tiempo para verificar la navegación e índices
- Tiempo de navegación

Nota: Sólo deberíamos usar tiempo para el acceso a la base de datos.

Disparadores

Un trigger SQL es un mecanismo que activa automáticamente un conjunto de sentencias SQL cuando un conjunto de condiciones se dan en la tabla.

También denominadas reglas activas

Básicamente, un trigger consiste en un evento trigger (conjunto de condiciones) y la resultante es una acción de trigger.

Normalmente un Disparador de alertas es creado en función de Procedimientos almacenados.

Disparadores

Se pueden utilizar triggers para implementar las reglas del Negocio.

```
CREATE PROCEDURE upd_items_p1()
  DEFINE GLOBAL old_qly INT DEFAULT 0;
  LET old_qty = (SELECT SUM(quantity)FROM items);
END PROCEDURE
CREATE PROCEDURE upd_items_p2()
  DEFINE GLOBAL old_qly INT DEFAULT 0;
  DEFINE new_qty INT;
  LET new_qty = (SELECT SUM(quantity)FROM items);
  if new_qty > old_qty*1.50 then
    RAISE EXCEPTION-746.0 "Not allowed-rule violation;"
  END IF
END PROCEDURE
CREATE TRIGGER up_items
UPDATE OF quantity On items
BEFORE(EXECUTE PROCEDURE upd_items_p1())
AFTER(EXECUTE PROCEDURE upd_items_p2())
```

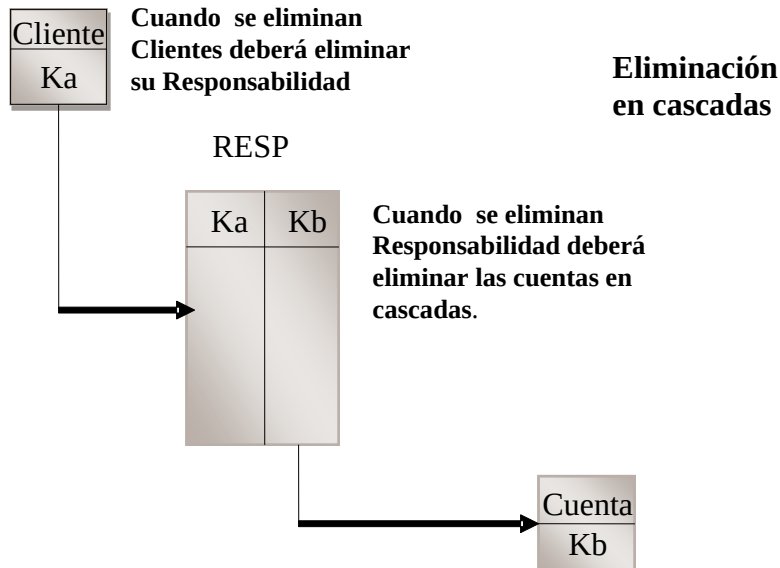
Disparadores

Se pueden utilizar triggers para lograr la integridad referencial en la base de datos.

```
CREATE TRIGGER del_cliente
DELETE ON cliente
REFERENCING OLD AS antes_del
FOR EACH ROW (DELETE FROM responsabilidad WHERE
  cliente_num = antes_del.cliente_num)

CREATE TRIGGER del_responsabilidad
DELETE ON responsabilidad
REFERENCING OLD AS antes_del
FOR EACH ROW (DELETE FROM cuentas WHERE
  cuenta_num= antes_del.cuenta_num);
```

Disparadores



Disparadores

1. No olvidar que el SQL es una operación en conjunto
 - Optimizar el tamaño de la página a transmitir
2. No olvidar que el SQL es un intérprete en donde:
 - La CPU del servidor debe interpretar y validar sintaxis.
 - El SQL valida que existan tablas, atributos y camino de búsqueda
 - No use SQL en el cliente. Debe invocar store procedures. Store procedures son SQL compilado
3. Hacer las aplicaciones transaccionales (pseudo - conversacionales y no conversacionales)

Planificación de eventos

Tabla de Eventos

Por cada Evento un trigger

CONDICIONES	E1	E2	E3	E4	E5
CONDICIÓN 1	☒				☒
CONDICIÓN2		☒			
CONDICIÓN3			☒		
CONDICIÓN4	☒			☒	
ACCIONES					
ACCIÓN1	☒				
ACCIÓN2		☒	☒		☒
ACCIÓN3				☒	
ACCIÓN4	☒				

Se recomienda planificar las operaciones en función de los eventos que se definen.
Para cada uno de estos eventos deben cumplirse las condiciones de las reglas de negocio.

4.2. Control de disponibilidad

CONTROL DE DISPONIBILIDAD DE DATOS

Control de Disponibilidad

- .Memoria Intermedia
- .Transacción
- .Concurrencia
- .Concurrencia Fallas

4.2.1. Gestión de memoria intermedia

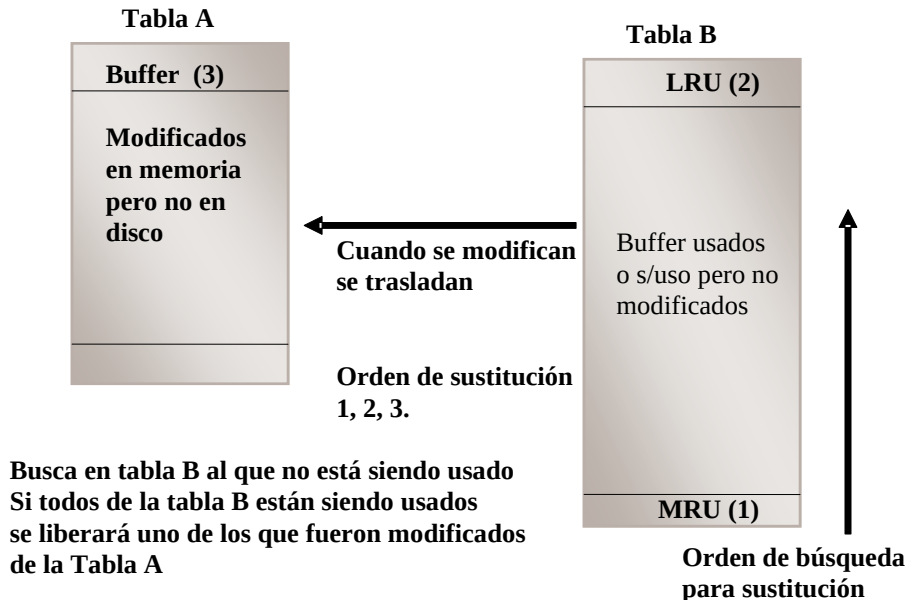
GESTIÓN DE MEMORIA INTERMEDIA

- Objetivo: Minimizar transferencia de bloques
- Lo ideal: Sería tener la base de datos en memoria
- Cuando transfiere: Sólo si el dato pedido no está
- Como transferir un nuevo bloque:
 - Debe liberar espacio (si no lo hubiera)
 - Al liberar lleva de la memoria al disco si fue modificado
 - Consigue el nuevo bloque y lo graba en el buffer.

Estrategia de Sustitución

- LRU(menos recientemente usado) de todos los bloque por liberar se escoge la que menos se ha utilizado recientemente.
- MRU(más recientemente usado) es el caso inverso al anterior, pero más efectivo.
- Se genera una tabla con LRU al inicio y MRU al final. Estos buffer contienen datos usados pero no modificados, estos últimos se encuentran en otra tabla.
- Cuando se desea liberar se toma el último es decir el MRU(lista LIFO).

Gestión de Memoria Intermedia



Cuando se va a liberar el área de buffer, previamente se debe realizar el **commit** de manera que quede actualizado de manera definitiva en disco.

El algoritmo asegura con alta probabilidad que el bloque seleccionado no será requerido.

4.2.2. Gestión de transacción

GESTIÓN DE TRANSACCIÓN

Transacción

- Conjunto de operaciones que forman una unidad lógica de trabajo.
- Características:
 - Atomicidad.- Por ser unidad, o se ejecuta todo o nada.
 - Consistencia.- Después de terminada la TRX la base de datos sigue consistente.
 - Aislamiento.- Para varias TRX concurrentes sólo una será ejecuta en un instante dado.
 - Durabilidad.- Después de la TRX el resultado se mantiene definitivo en la base de datos.
- Terminan en commit / rollback

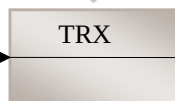
Los cambios se van grabando en Log , sigue en (I).

E = Si ocurre un problema que obliga a abortar la Trx, el log queda marcado con 'E'.



I = La Trx inicia los cambios

Rollback aplica los registros de log para des-hacer todo lo modificado en la BD.



O = Cuando los cambios terminan el Log se marca con 'O'.

A= Si la actualización es diferida se graba los cambios en disco cuando se complete un bloque.

Igualmente en una subida en caliente del sistema se procesaran todos los registros Log con estado 'O'. Los demás estados se ignoran.

Para una Trx se pueden grabar varios registros Log , uno por cada tabla modificada. Al final se grabará un registro adicional para relacionar a todos los registros comprometidos con la Trx.

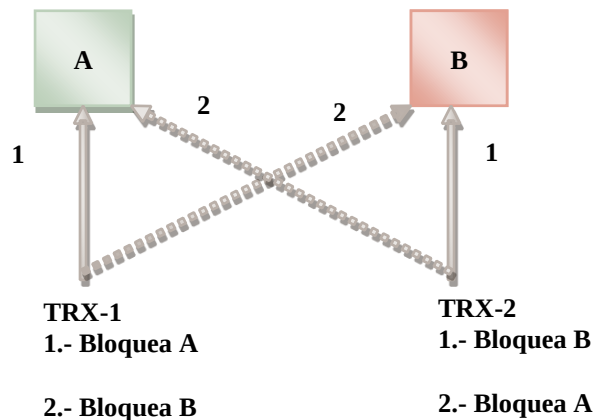
4.2.3. Gestión de concurrencia

GESTIÓN DE CONCURRENCIA

Gestión de Concurrencia

- BLOQUEOS
 - Sobre un bloque de datos se tiene un control exclusivo ante otras trx que requiera el mismo bloque de datos(en algunas SGBD el bloqueo se realiza a nivel de fila).
- INTER-BLOQUEOS
 - Cuando trx tiene control exclusivo sobre un dato A y requiere bloquear otro bloque B que ha sido tomado por otra trx. Y esta a su vez requiere el dato A.

Inter-Bloqueos



Solución: 1.- Dar Tiempo máximo de proceso luego volver a iniciar.

2.- Re-iniciar una de las trx. La que signifique menor costo

Cálculo de costo.- Cantidad de elementos, trx involucradas, operaciones realizadas, operaciones faltantes, etc.

Este esquema tiene un manejo más complejo cuando se trata de un proceso *batch*, debido a que el lote de registros a procesar, se puede estar paralizando cuando compiten por recursos con otros procesos.

En estos casos el software gestor de la base de datos al detectar que el procesos está siendo interrumpido muchas veces, toma la acción de bloquear de manera exclusiva la tabla en primera instancia de manera parcial, si a pesar de este bloqueo el proceso sigue siendo interrumpido, tomará la acción de bloquearla toda para que el proceso pueda seguir sin interrupciones.

Para evitar esta situación a nivel de la planificación de procesos, se debe contemplar estas posibles colisiones, en función de las tablas que utiliza un programa.

Solo es relevante cuando la tabla sea actualizada. Para el caso de solo consulta no habrá ningún problema, la tabla será compartida por varios procesos a la vez.

La planificación de procesos se basa en priorizar determinados procesos cuando compiten por recursos.

Cuando se trata de módulos independientes pueden ser procesados simultáneamente.

4.2.4. Gestión de fallas

GESTIÓN DE FALLAS DEL SISTEMA

Recuperación ante Fallas

- | | |
|--|---|
| <ul style="list-style-type: none">• Tipos de Fallas<ul style="list-style-type: none">– A nivel Transacción<ul style="list-style-type: none">• Lógico(Datos)• Sistemas(Contención)– Caída del sistema<ul style="list-style-type: none">• Sistema Operativo, Hardware– A nivel Disco<ul style="list-style-type: none">• Cabezal, pistas | <ul style="list-style-type: none">• Tipos de Almacenamiento:• Temporal.-<ul style="list-style-type: none">– Buffers• Permanente.-<ul style="list-style-type: none">– Discos<ul style="list-style-type: none">• Simples• RAID(Arreglo con Redundancia)– Computadores alternos– Cintas, CD |
|--|---|

Registro Bitácora (logging)

- Graba cronológicamente las transacciones
- Permite conocer el registro modificado, el atributo cambiado tanto su valor anterior y el nuevo.
- Estados del registro log: Indica estados de la TRX:
 - Iniciado(I)
 - 'OK' Listo para modificar BD (O)/ Abortado(E)
 - Actualizada BD (A)
- Mientras dure la actualización se mantiene control exclusivo sobre el bloque de registros.

Actualización de la Base de datos

- En base al archivo log se grabarán en el disco las transacciones que terminaron en estado "OK"
- Después de grabada la base de datos, el registro log, queda marcado como (A). No se será tomado en cuenta para las próximas actualizaciones.
- Generalmente para estos procesos, se acumulan TRX "OK" para procesar en bloque.
- La técnica de memoria virtual optimiza el proceso.

4.3. Control de confidencialidad

En este módulo se trata de validar que el usuario está autorizado y, además las limitaciones que tiene en cuanto el acceso a los datos.

4.3.1. Perfil de usuario

Se considera perfil de usuario como el conjunto de tipos de autorizaciones que se le asignan a un grupo de usuarios por tener las mismas necesidades de información.

Por cada perfil se determina:

- 1.- Tablas que puede acceder
- 2.- Tipo de autorización para cada tabla autorizada

- Alta, cambio, eliminación, consulta, etc.

En base a estos conceptos la base de datos deberá contener tabla de:

- **Operaciones:** Ingresar, Modificar, Eliminar, Consultar, Etc.

- Tablas

- **Perfil:** Características de autorizaciones

- Relación (Perfil-Tablas-Operaciones)

- **Usuarios:** Datos de la cuenta usuario

- Relación (usuario-Perfil)

4.3.2. Identificar y autenticar

A nivel de identificación no basta con ingresar un determinado código, deberá ser verificado mediante claves, o alguna característica personal física o intelectual. En el campo de lo físico se han implementado la huella digital, iris del ojo, etc.

Las claves normalmente son encriptados.

Otras medidas de Seguridad

- RAID, redundancia en arreglo de discos permiten replicar información de manera que si un disco del arreglo falla el sistema puede continuar sin ser afectado.
- Back up diarios, se recomienda salvar sobre todo las tablas críticas.
- Sistemas espejo remotos, en sistemas de servicio intensivo 24x7, se usan más de un servidor donde uno replica al principal.

Cuando se aplica redundancia de servidores, se recomienda que se ubiquen físicamente en lugares distintos, de esta manera en caso de atentados o siniestros naturales lo más probable es que por lo menos uno quede a salvo.

En esquemas de servidores paralelos es recomendable intercambiarlos de manera diaria donde uno de ellos sea el principal y el segundo el respaldo y viceversa

5. Implementación física de la base de datos

IMPLEMENTACIÓN FÍSICA DE LA BASE DE DATOS

Con todo lo expuesto nos toca materializar la base de datos en términos físicos, para lo cual es ineludible hacerlo mediante algún software gestor de base datos. Por suerte todos los software presentan esquemas semejantes en su arquitectura.

Detallamos a continuación los componentes principales de un motor de base para ilustrar la manera como se implementa los temas presentados anteriormente.

Se toma como referencia al motor del gestor de base de datos ORACLE.

5.1. Table space (área de disco para tablas)

TABLESPACE

Oracle almacena data en tablespace, pero físicamente en datafiles.

Desde el punto de vista lógico se tiene:

- Una base de datos está compuesta por uno o más tablespaces.

- Una tablespace esta formada por uno o más segmentos(tablas).

- Un segmento a su vez está formado por uno o más extents.

- Un extent está formado de uno o más bloques.

Desde el enfoque físico los bloques están compuestos por un número entero de sectores cada uno de estos contienen 512 bytes.

Una tablespace es creada usando el comando:

```
CREATE TABLESPACE userdata
Datafile '/u01/oradata/userdata01.dbf' SIZE 100M
AUTOEXTEND ON NEXT 5M MAXSIZE 200M;
```

TABLESPACE

CREATE TABLESPACE tablespace (espacio para tablas a ser creada)

DATAFILE clausula (archivo físico que contendrá la tabla)

MINIMUM EXTENT integer [K|M] (cada vez que crece físicamente lo hará en integer K | M bytes).

BLOCKSIZE integer [K](longitud del IC)

LOGGING|NOLOGGING (se grabarán todos los cambios en redo log file).

DEFAULT storage_clausula (se toman por defecto desde el archivo de parámetros).

ONLINE|OFFLINE (Después de su creación la tabla estará activa/inactiva).

PERMANENT | TEMPORARY

Extent_management_clausula

Segment_management_clausula

TABLESPACE

Gestión de espacio en Tablespace

La Tablespace está compuesta por segmentos.

La Tablespace puede ser administrada por el Dictionary. En el data dictionary los segmentos se crean o eliminan.

Cada segmento(tabla) almacenado en la tablespace puede tener una diferente cláusula de almacenamiento.

```
DATAFILE 'u01/oradat/userdata01.dbf'  
SIZE 500M EXTENT MANAGEMENT DICTIONARY  
DEFAULT STORAGE  
CREATE TABLESPACE userdata  
(initial 1M NEXT 1M PCTINCREASE 0);
```

TABLESPACE DE SOLO LECTURA.-

```
ALTER TABLESPACE userdata READ ONLY;
```

Genera Checkpoint

Data disponible sólo para lectura.

Los objetos pueden ser eliminados de la tablespace.

TABLESPACE

CREATE TABLESPACE tablespace (tabla a ser creada)

DATAFILE clausula (archivo físico que contendrá la tabla)

Data_file_clausula:= filename [SIZE integer [K|M] [REUSE]
[autoextend_clausula] (activa o desactiva el crecimiento automático del datafile.

SIZE: Tamaño del archivo

REUSE: Permite reusar el archivo.

NEXT: Indica el tamaño en bytes, del siguiente crecimiento automático del datafile cuando lo requiera.

MAXSIZE: Indica el máximo espacio de disco permitido para un crecimiento automático del datafile.

UNLIMITED: Indica que el espacio en disco que puede tener el datafile no tiene límite.

TABLESPACE

Cambiando almacenamiento pre-determinado

```
ALTER TABLESPACE userdata MINIMUM EXTENT 2M;  
ALTER TABLESPACE userdata DEFAULT STORAGE (initial 2M  
NEXT 2M MAXEXTENTS 999);
```

Redimensionando un Datafile .-

```
ALTER DATABASE  
DATAFILE '/u03/oradata /userdata02. dbf'  
RESIZE 200M;
```

Adicionando datafile a una tablespace .-

```
ALTER TABLESPACE user_data  
ADD DATAFILE '/u01/oradata /userdata03. dbf'  
SIZE 200M;
```

TABLESPACE

Método para mover Datafiles.-

```
ALTER TABLESPACE userdata RENAME  
DATAFILE '/u01/oradata /userdata01. dbf'  
TO '/u02/oradata /userdata01. dbf';
```

Destruyendo Tablaspace .-

```
DROP TABLESPACE userdata  
INCLUDING CONTENTS AND DATAFILES;
```

5.1.1. Estructura de almacenamiento físico

Database										
PROD.										
TABLESPACE SYSTEM			USER_DATA				RBS		TEMP	
DATAFILES DISK1/SYS1.dbf			DISK2/ USER1.dbf		DISK3/ USER2.dbf		DISK1/ ROLL1.dbf		DISK1/ TEMP.dbf	
SEGMENTS			S_DEPT		SE_EMP					
DD Table	DD Index		S_DEPT	SE_EMP						
Data Seg	Index Seg	RB Seg	Data Seg	Data Seg						
EXTENTS			FREE		1122					
1212121222										
DATABLOCKS										

5.1.2. Segmentos de table space

SEGMENTOS

Los segmentos son objetos de la base de datos:

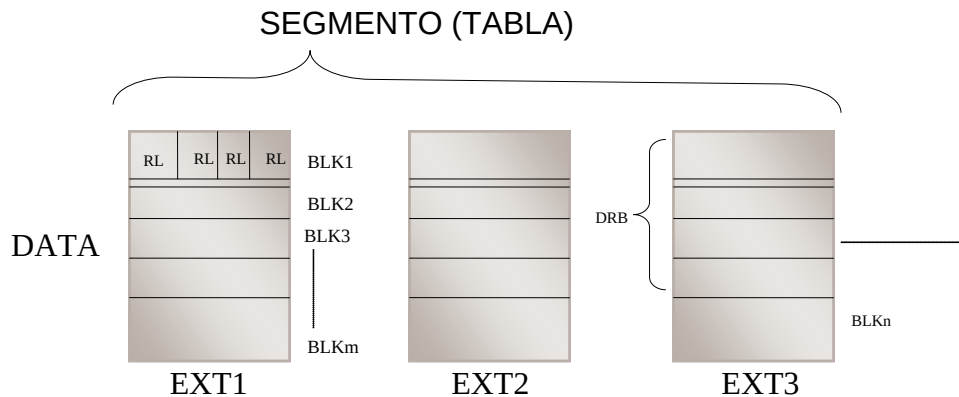
Table: Un segmento almacena la data para una tabla que no está particionada ni se trata de cluster. Los datos se almacenan no necesariamente bajo algún orden específico.

Partición de tabla: Una tabla puede estar almacenada en varias particiones, y cada una de las ellas reside en diferentes tablespaces. Normalmente estas tablas están segmentadas por rango de claves.

Cluster: Así como la tabla es tipo de segmento de datos. Un cluster puede tener una o muchas tablas. Las tablas en un cluster están en un mismo segmento y comparten las mismas características de almacenamiento. Las filas en una tabla cluster pueden ser accedidas por un índice.

Index: Todas las entradas para un índice, pueden ser almacenadas en un segmento índice.

SEGMENTOS (TABLAS)



SEGMENTOS

Un extent es un pedazo de espacio usado por un segmento dentro de un tablespace.

Un extent es generado cuando el segmento es:

- Creado
- Extentendido
- Alterado

Un extent es retirado cuando el segmento es:

- Destruído
- Truncado
- Alterado

5.1.2.1. Extents extensión de table space

Extents

Cuando una Tablespace es creada, los datafiles contienen un encabezado, el cual será el primer bloque en el archivo.

Cuando los segmentos son creados, estos son ubicados en extents libres en la tablespace. Luego estos extents son marcados como extents usados. En el caso que los extents sean liberados estos volverán al pool de extents libres.

Conjunto de extents forman un segmento.

Cuando el archivo tiene que crecer , lo hace en un número entero de extents

5.1.3. Blocks

BLOCK

Los bloques de la base de datos, es la mínima unidad de I/O.

Consisten de uno o más sectores (512 bytes)

Se definen en la creación de la tablespace.

DB_BLOCK_SIZE es el tamaño por defecto del bloque.

La base de datos puede ser creada con un bloque estándar y hasta cuatro longitudes no estándar

Los tamaños pueden ser desde 2 kb hasta 32 kb (potencia de 2).

En el caso de uso de bloques no estándar también obligará a definir buffers cachés para los bloques no estándar.

Después de creada la base de datos y definido el DB_BLOCK_SIZE no podrá ser cambiado sin recrear la base de datos.

DB_CACHE_SIZE = Valor x defecto del buffer caché , toma el valor de 4mb si SGA < 128 mb en caso contrario 16 mb.

Tamaño de bloque no estándar

Se deben de configurar buffers cachés no estándar:

.DB_2K_CACHE_SIZE para bloques de 2 Kb

.DB_4K_CACHE_SIZE para bloques de 4 Kb

.DB_8K_CACHE_SIZE para bloques de 8 Kb

.DB_16K_CACHE_SIZE para bloques de 16 Kb

.DB_32K_CACHE_SIZE para bloques de 32 Kb

Este parámetro no se requiere cuando el bloque estándar es igual a su buffer.

```
CREATE TABLESPACE tbs_1 DATAFILE 'tbs_1.dbf' SIZE 10M  
BLOCKSIZE 4K.
```

Contenido

Todas las particiones de los objetos deben residir en tablespaces con el mismo tamaño de bloque.

Todos los tablespaces temporales deben tener bloques estándar

Todos los bloques poseen:

Encabezado: Contiene dirección del bloque, directorio de la tabla, fila y transacciones (son usados cuando las transacciones hacen actualización de filas.

Data: Contiene las filas.

Espacio libre: Está entre el encabezado y la data.

Parámetros:

Para el manejo de concurrencia de transacciones en el mismo bloque se definen dos parámetros INITRANS y MAXTRANS. Estas transacciones tienen como objetivo actualizar data del bloque. Para cada transacción se dedica una porción de área para su ejecución.

INITRANS: Garantiza un mínimo de nivel de concurrencia. El valor por defecto es 1 para un segmento de datos y 2 para un segmento de índice.
MAXTRANS: Por defecto el número de concurrencia máxima es 255.

PCTFREE: Especifica para un segmento de datos, el porcentaje de espacio en el bloque reservado para crecimiento de filas como resultado de una actualización, el valor por defecto es 10%.

PCTUSED: Para un segmento de datos, este valor es el mínimo porcentaje de espacio usado, que Oracle debe mantener para cada bloque de la tabla.

Tanto para PCTFREE como para PCTUSED se debe deducir el espacio del encabezado.

Mapa de Uso

Para saber si un bloque está o no libre se activa o desactiva un bit, a este conjunto de bits se le conoce como BITMAP.

Estos mapas de bit están contenidos en bloques especializados (BMP). Cuando se va a insertar una nueva fila, se busca en el mapa un bloque con el suficiente espacio.

En la medida que el bloque cambia de estado, el mapa también.

Configuración automática:

```
CREATE TABLESPACE data02  
DATAFILE '/U01/ORADATA/DATA02.dbf' SIZE 5M  
EXTENT MANAGEMENT LOCAL UNIFORM SIZE 64K  
SEGMENT SPACE MANAGEMENT AUTO;
```

Después de creada una tablespace, las especificaciones se aplican a todos los segmentos del tablespace.

Ejemplo

PCTFREE: (%P1) Se reserva para cuando la longitud de la fila se incremente por actualización de datos. Se podrá insertar mientras el espacio libre sea mayor a P1. Se podrá utilizar (100-P1) del espacio total.

PCTUSED: (P2) Cuando se eliminan filas, se recupera espacio sin uso. El espacio usado debe ser como mínimo P2, en caso sea menor a este parámetro se permitirán inserciones.

Tablas que permiten seguimiento operativo:

DBA_EXTENTS:	Cada fila representa un extent
DBA_SEGMENTS:	Cada fila representa un segmento
DBA_TABLESPACE:	Cada fila representa una tabla
DBA_DATA_FILES:	Cada fila representa un datafile
DBA_FREE_SPACE:	Cada fila representa un extent.

5.2. Tablas

Existen Los siguientes tipos de tablas:

Tablas Regulares.- Usadas para almacenar datos, es la más común en ser usada. Las filas se almacenan físicamente no necesariamente en orden lógico.

Tablas Particionadas.- La data está dividida en varias partes por el rango de claves. Físicamente cada partición es un segmento que puede estar en tablespaces diferentes. Se usan cuando se quiere manejar el problema de la recurrencia.

Tablas Organizadas como Índices.- Se crea un árbol "B" almacenando el atributo clave y además los otros atributos, en caso de generar filas muy grandes se generan segmentos overflow.

Tablas Mixtas.- Pueden contener más de una tabla que comparten atributos y generalmente se unen en conjunto. Semejante a una tabla extendida. Mejora la performance

5.2.1. Tipos de atributos

Los tipos de atributos son los siguientes:

Escalar:

CHAR(N),NCHAR(N):Char fijo de hasta 2000 bytes por fila.
 VARCHAR2(N),NVARCHAR2(N).Char variable de hasta 4000bytes.
 NUMBER(P,S)(Dígitos de precisión, Escala de decimales)
 Numérico de hasta 38 dígitos significativos.
 DATE: De 7 bytes de longitud almacena fecha, hora, siglo, etc.
 TIMESTAMP: aaaa-mm-dd-hh-mm-ss-msmsms.
 RAW(N): Almacena una fila de hasta 2000bytes.
 BLOB, CLOB,NCLOB,BFILE: Grandes objetos binarios.
 LONG, LONG RAW: Para grandes filas de data.
 ROWID, UROWID: Permite ubicar rápidamente una fila.

Colección:

VARRAY: Arreglos.

TABLE: Nested tabla que contiene otras tablas. Realmente almacena una dirección hacia la tabla secundaria.

Relacional:

Referencia: Desde la tabla ítem se puede asociar la tabla producto.

Rowid: Es un tipo de dato que puede ser consultado con otras columnas en la tabla, presenta las siguientes características:

- Es un único identificador para cada fila de la base de datos.
- No es almacenada explícitamente como una columna.
- Se usa para un rápido acceso a la fila (no es dirección física).
- Se almacena en índices para especificar las filas con un grupo de claves.

Formato:

- Data Object number: Indica en cual tablespace se encuentra la tabla.
- Relative file number: Indica el archivo dentro del tablespace.
- Block number: Indica el bloque dentro del archivo.
- Row number: Indica la fila dentro del bloque

000000

OBJECT

FFF

FILE

BBBBBB

BLOQUE

RRR

FILA

Se almacena en un campo de 10 bytes

5.2.2. Estructura de una tabla

Estructura de una fila

Las filas son almacenadas en la base de datos como registros de longitud variable. Las columnas en las filas se almacenan en el orden como fueron declaradas. Las columnas nulas no son almacenadas. Sólo un byte se requiere por columna nula para almacenar la longitud del atributo cero.

Cada fila tiene:

Encabezado.- Usado para almacenar número de columnas en la fila y estado de bloqueo de la fila.

Data..- Para cada columna se almacena un campo de longitud, este campo puede tener de uno a tres bytes. El valor de la columna se almacena inmediatamente a este campo. Desde el directorio de la fila se almacena el pointer donde inicia la fila.

L	FLD1	L	FLD2	L	FLD3		ER	DIR
---	------	---	------	---	------	--	----	-----

CREANDO TABLAS

```
CREATE TABLE      hr.empleados
(cod_empleado      NUMBER(6),
primer_nombre      VARCHAR2(20),
apellido VARCHAR2 (25),
fecha_ingreso      DATE DEFAULT SYSDATE,
puesto VARCHAR2 (10),
salario NUMBER (8,2),
comision  NUMBER (2,2),
cod_depart  NUMBER(4) );

CREATE TABLE      hr.depart
(cod_depart  NUMBER (4),
nombre_depart  VARCHAR2 (30),
Ubicación NUMBER (4) )
STORAGE(INITIAL      200K NEXT 200K
PCTINCREASE 0 MINEXTENTS 1 MAXEXTENT 5)
TABLESPACE data;
```

En este ejemplo se crea la tabla hr.depart en el espacio de tabla data.

```
create table AT_CCCUTDMT
(CUT_COD_CLI_CD      NUMBER not null,
CUT_COD_CTA_CD      NUMBER not null,
CUT_TIP_DOC_CD      CHAR(3) not null,
CUT_FEC_INI_VIG_FF CHAR(10) not null,
CUT_NUM_DOC_NU      CHAR(20),
CUT_DIG_CTL_DOC_CD CHAR(2),
CUT_FEC_FIN_VIG_FF CHAR(10),
CUT_USR_ULT_MOD_NO CHAR(16),
CUT_TIM_ULT_MOD_TS CHAR(26),
CUT_ITE_ALT_SIS_TS CHAR(26),
CUT_ITE_BAJ_SIS_TS CHAR(26))
tablespace TBSP_DATA
pctfree 5
pctused 40
initrans 1
maxtrans 255
storage
(initial 128K
next 8200K
minextents 1
maxextents unlimited
pctincrease 0);
```

```

-- Create table DB2
CREATE TABLESPACE GCCSCTTE
  IN GCCONT
  USING STOGROUP GCSTO1T PRIQTY 4 SECQTY 0
  ERASE NO FREEPAGE 0 PCTFREE 0 DEFINE YES TRACKMOD NO
  SEGSIZE 8
  BUFFERPOOL BP1 LOCKSIZE ROW
  CLOSE NO
  COMPRESS NO
;
CREATE TABLE ATISOWN.GCCSCTAT (
  CSC_COD_COT_CD INTEGER NOT NULL GENERATED ALWAYS
  AS IDENTITY(
    START WITH 1,
    INCREMENT BY 1,
    CACHE 20,
    NO CYCLE ,
    MINVALUE 1,
    MAXVALUE 999999999),
  CSC_TIM_ULT_MOD_TS    TIMESTAMP NOT NULL
)
  IN GCCONT.GCCSCTTE
;

```

Para el motor de base de datos DB2 , es muy semejante al de Oracle, en este ejemplo se crea el espacio de tabla GCCSCTTE en la base GCCONT. Luego se crea la tabla ATISOWN.GCCSCTAT

```

-- Create/Recreate indexes
create index AT_CCCUTDMT_IDX1 on AT_CCCUTDMT
(CUT_COD_CLI_CD,
CUT_COD_CTA_CD,
CUT_TIP_DOC_CD,
CUT_FEC_INI_VIG_FF)
    tablespace TBSP_INDX
    pctfree 10
    initrans 2
    maxtrans 255
    storage
    (initial 504K
     next 1M
     minextents 1
     maxextents unlimited
     pctincrease 0);

-- Grant/Revoke object privileges
grant select on AT_CCCUTDMT to DB2ADMIN;

```

```

CREATE TABLESPACE
PEPSPPTE
    IN PEPETI
    USING STOGROUP PESTO1T
    PRIQTY 482 SECQTY 48
    ERASE NO FREEPAGE 10
    PCTFREE 5 DEFINE YES
    TRACKMOD NO
    SEGSIZE 64
    BUFFERPOOL BP1
    LOCKSIZE ANY
    CLOSE NO
    COMPRESS YES
;

```

```

CREATE TABLE ATISOWN.PEPSPPAT (
    PPP_COD_CNL_VEN_CD    INTEGER NOT NULL,
    PPP_COD_PAQ_CD        INTEGER NOT NULL,
    PPP_COD_PRO_SER_CD    INTEGER NOT NULL,
    PPP_COD_OPR_CMR_CD    INTEGER NOT NULL,
    PPP_PRO_SER_PAQ_CD    INTEGER NOT NULL,
    PPP_CNC_FTB_PED_IM    DECIMAL(18, 4) NOT
NULL,
    PPP_CNC_FTB_APR_IM    DECIMAL(18, 4) NOT
NULL,
    PPP_CAN_MAX_PRO_NU    INTEGER NOT NULL,
    PPP_CAN_MIN_PRO_NU    INTEGER NOT NULL,
    PPP_USR_ULT_MOD_NO    CHAR(16) NOT NULL,
    PPP_TIM_ULT_MOD_TS    TIMESTAMP NOT NULL
WITH DEFAULT,

    PPP_ITE_ALT_SIS_TS    TIMESTAMP NOT NULL
WITH DEFAULT,
    PPP_ITE_BAJ_SIS_TS    TIMESTAMP
)
IN PEPETI.PEPSPPTE
;

```

```

CREATE UNIQUE INDEX ATISOWN.PEPSPP10 ON
ATISOWN.PEPSPPAT
(
    PPP_COD_CNL_VEN_CD    ASC,
    PPP_COD_PAQ_CD        ASC,
    PPP_COD_PRO_SER_CD    ASC,
    PPP_COD_OPR_CMR_CD    ASC
)
USING STOGROUP PEST01I
    PRIQTY 232 SECQTY 24
ERASE NO FREEPAGE 10 PCTFREE 5
    DEFINE YES
CLUSTER
BUFFERPOOL BP2    CLOSE NO
DEFER NO

COPY NO
PIECESIZE 8 M
;

```

```

CREATE VIEW ATISOWN.PEPSPPAV
AS SELECT
    PPP_COD_CNL_VEN_CD,
    PPP_COD_PAQ_CD,
    PPP_COD_PRO_SER_CD,
    PPP_COD_OPR_CMR_CD,
    PPP_PRO_SER_PAQ_CD,
    PPP_CNC_FTB_PED_IM,
    PPP_CNC_FTB_APR_IM,
    PPP_CAN_MAX_PRO_NU,
    PPP_CAN_MIN_PRO_NU,
    PPP_USR_ULT_MOD_NO,
    PPP_TIM_ULT_MOD_TS,
    PPP_ITE_ALT_SIS_TS,
    PPP_ITE_BAJ_SIS_TS

FROM ATISOWN.PEPSPPAT
;

```

```

GRANT DELETE, INSERT, SELECT, UPDATE ON TABLE ATISOWN.PEPSPPAT TO BMA;
GRANT DELETE, INSERT, SELECT, UPDATE ON TABLE ATISOWN.PEPSPPAT TO BAA;
GRANT DELETE, INSERT, SELECT, UPDATE ON TABLE ATISOWN.PEPSPPAT TO BIA;
GRANT DELETE, INSERT, SELECT, UPDATE ON TABLE ATISOWN.PEPSPPAT TO BMD;
GRANT DELETE, INSERT, SELECT, UPDATE ON TABLE ATISOWN.PEPSPPAT TO BAD;
GRANT DELETE, INSERT, SELECT, UPDATE ON TABLE ATISOWN.PEPSPPAT TO BID;
GRANT DELETE, INSERT, SELECT, UPDATE ON TABLE ATISOWN.PEPSPPAT TO BATCBK;
GRANT ALL ON TABLE ATISOWN.PEPSPPAV TO BMA;
GRANT ALL ON TABLE ATISOWN.PEPSPPAV TO BMT;
GRANT ALL ON TABLE ATISOWN.PEPSPPAV TO BAA;
GRANT ALL ON TABLE ATISOWN.PEPSPPAV TO BAT;
GRANT ALL ON TABLE ATISOWN.PEPSPPAV TO BIA;
GRANT ALL ON TABLE ATISOWN.PEPSPPAV TO BIT;
GRANT DELETE, INSERT, SELECT, UPDATE ON TABLE ATISOWN.PEPSPPAV TO BATCBK;
GRANT DELETE, INSERT, SELECT, UPDATE ON TABLE ATISOWN.PEPSPPAV TO BMD;
GRANT DELETE, INSERT, SELECT, UPDATE ON TABLE ATISOWN.PEPSPPAV TO BAD;
GRANT DELETE, INSERT, SELECT, UPDATE ON TABLE ATISOWN.PEPSPPAV TO BID;
GRANT SELECT ON TABLE ATISOWN.PEPSPPAV TO PUBLIC
;
COMMIT;

```

Ejemplo

```
CREATE TABLE hr.departamento  
(codigo_dep number(4),  
Nombre_dep varchar2(30),  
Ubicación_dep number(4))  
STORAGE(INITIAL 200K NEXT 200K  
PCTINCREASE 0 MINEXTENTS 1 MAXEXTENTS 5)  
TABLESPACE data;
```

Claúsula **STORAGE** : Especifica características de almacenamiento. Para la tabla. En el ejemplo se indica que se usará 200k al inicio y se Incrementará de 200 en 200k, porque PCTINCREASE = 0. El máximo número de extents(crecimientos) es 5 y un mínimo de 1.

CREANDO TABLAS

MINEXTENTS: Mínimo número de extents a ser generados.

MAXEXTENTS: Máximo número de extents a ser generados.

PCTINCREASE: es el porcentaje de incremento del tamaño del extent después del next.

PCTFREE: Porcentaje de espacio libre en cada bloque, debe tomar valores entre 0 y 99. El valor por defecto es 10.

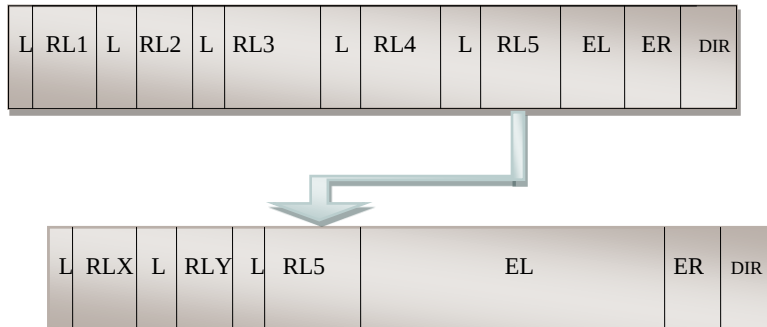
PCTUSED: Espacio mínimo usado puede tomar valores entre 0 y 99. Cuando está por debajo de este valor el bloque aceptará filas nuevas.

MAXTRANS: Máximo número de transacciones sobre el bloque.

TABLESPACE: Indica el lugar donde se creará la tabla. Si no es especificada se tomará la tablespace por defecto que es la asociada al propietario del esquema.

MIGRACION DE FILAS

Cuando se fija el valor del PCTFREE muy bajo, el espacio disponible para que una fila pueda crecer de longitud puede ser insuficiente, lo que ocasionaría su traslado o migración hacia otro bloque creado, un pointer desde el primer bloque hacia el nuevo, donde se almacenaría la fila. Esta situación obviamente degrada la performance de la base de datos, por lo cual se debe evitar.

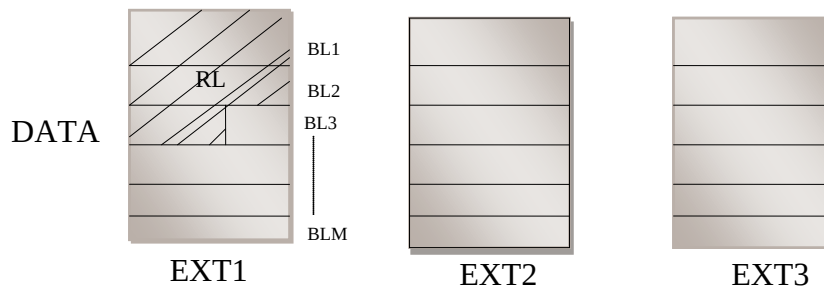


5.2.3. Filas esparcidas

FILAS ESPARCIDAS

FILAS ESPARCIDAS

Puede ocurrir que una fila pueda crecer atípicamente en una longitud extraordinaria. En este caso se divide la fila en fragmentos de fila que pueda ser almacenados en varios bloques.



ALTERAR VALORES DE PARAMETROS

ALTER TABLE hr.empleados

PCTFREE 30

PCTUSED 50

STORAGE (NEXT 500K

MINEXTENTS 2

MAXEXTENTS 100);

NEXT : Indica que cada vez que se requiera incrementar un bloque a la tabla, este tendrá un tamaño de 500k. Si adicionalmente el PTINCREASE=100, se incrementaría el tamaño del nuevo bloque en 500k. Se redondea en caso no sea exacto.

MAXEXTENT: Puede tomar valor ilimitado.

INITIAL : No podrá ser cambiado.

MINEXTENT: si se modifica su impacto se notará en caso de truncar la tabla.

Truncate , Dropped

TRUNCATE TABLE hr.empleados.

Este comando elimina las filas de una tabla y libera el espacio utilizado. Los índices correspondientes también son truncados.

No es posible revertir la operación porque no se generará en el bloque UNDO del archivo redo.log.

Por respeto a la integridad, no se trunca una tabla que referencia a otra. Los triggers asociados a la tabla no se activan.

DROP TABLE hr.department

CASCADE CONSTRAINTS;

Cuando se dropea una tabla, los extents usados por la tabla serán liberados

DROPEANDO UNA COLUMNA

```
ALTER TABLE hr.empleados  
DROP COLUMN comentario  
CASCADE CONSTRAINTS CHECKPOINT 1000;
```

Con este comando se remueve tanto la data como la longitud de la columna, liberando el espacio correspondiente.

Esta operación en una tabla larga consume considerable tiempo. Dropeando una columna consume tiempo y espacio. Sin embargo con el uso de los checkpoint cada cierta cantidad de filas se minimiza el efecto de la operación drop.

Si la instancia falla antes de concluir la operación, la tabla queda marcada como inválida y termina la operación cuando se levante la instancia.

5.3. Índices

INDICES

CLASES DE INDICES

Desde el punto de vista de su uso podemos denominar índices lógicos

Desde su implementación en su almacenamiento, índice físico.

LÓGICA

Clave

Simple: Formada por un solo atributo.

Compuesta: Formada por más de un atributo. No necesariamente contiguos ni ordenados de acuerdo a su clave.

El máximo número de atributos es 32.

Valor

Único: No se repite el valor de la clave

Replicado: El valor clave puede ser repetido.

Basada en función:

Dominio.- Se crean rutinas para manejar ciertos dominios de datos.

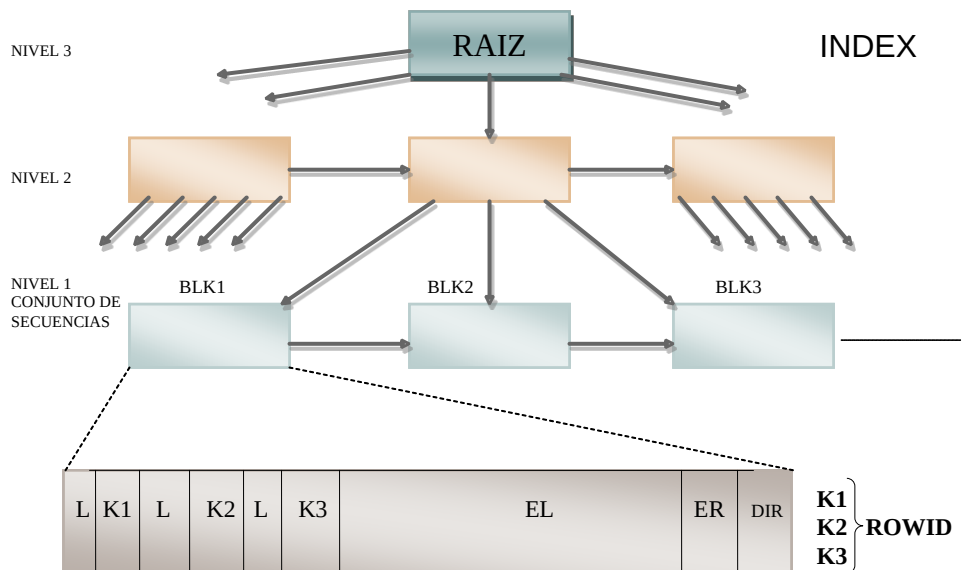
FISICA.

Sobre tablas particionadas o no

Estructura de Arbol-B normal o reverso

Bitmap

ARBOL "B"



CREANDO UN INDEX

```
CREATE INDEX hr.empleado_nombre_idx
ON hr.empleado(nombre) PCTFREE 30
STORAGE(INITIAL 200K NEXT 200K PCTINCREASE 0 MAXEXTENTS 50)
TABLESPACE indx;
```

Tabla usada empleado, columna nombre.

OPCIONES

- UNIQUE(NONUNIQUE)
- SCHEMA: Propietario del índice/tabla
- INDEX: Nombre del índice
- TABLE: Nombre de la tabla
- ASC/DESC
- TABLESPACE
- PCTFREE
- INITRANS, MAXTRANS
- STORAGE: Número de extent a ser generados
- LOGGING, NOLOGGING: Genera o no redo log file.
- NOSORT: No ordena la data, debe llegar en orden.

Actualizando Parámetros

```
ALTER INDEX empleado_nombre_idx  
STORAGE(NEXT 400K  
MAXEXTENTS 100);
```

Solo se está incrementando el almacenamiento para el índice.
Los parámetros del bloque pueden ser actualizados para
garantizar alto nivel de concurrencia en un índice del bloque.

```
ALTER INDEX empleado_nombre_idx REBUILD  
TABLESPACE indx02;
```

Se genera el índice en otra tablespace. Es como mover el
índice hacia otra tablespace

```
ALTER INDEX empleado_nombre_idx REBUILD ONLINE;
```

Los índices pueden ser reconstruidos con un mínimo de bloqueo

5.4. Integridad de datos

INTEGRIDAD DE DATOS

Restricciones Inmediatas y diferidas

Existen restricciones que deben aplicarse en el momento de la operación y otras que pueden ser diferidas hasta antes del **commit**.

Si en el momento del **commit** se viola alguna restricción entonces el sistema hará **roll back**, esta operación es muy usada sobre todo para los casos donde tablas padre e hijo poseen una relación de clave foránea y deben ser ingresadas en el mismo instante.

ALTER SESION

SET CONSTRAINTS = {IMMEDIATE | DEFERRED | DEFAULT}

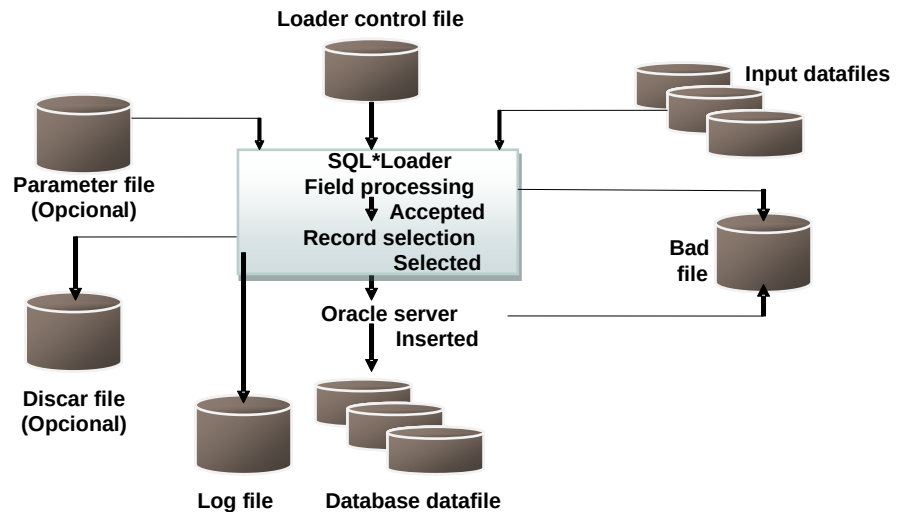
APENDICE A.-

A.1.- UTILITARIOS

UTILITARIOS DE LA BASE DE DATOS

UTILITARIOS

SQL*Loader.- Carga archivos planos a tablas relacionales.



UTILITARIOS : SQL*Loader

Control file

Contiene toda la información necesaria para poder realizar la carga de la data desde archivos planos. Como dónde encontrar la data, el formato que posee (DDL)

Sintáxis.

Formato libre

Los comentarios comienzan con “—”

La palabra clave “constant” es reservada.

UTILITARIOS : SQL*Loader

Control file . Contiene toda la información necesaria para poder realizar la carga de la data desde archivos planos. Como donde encontrar la data, el formato que posee(DDL),

Está compuesta de tres partes:

- A.- Contiene información general como: opciones, filas, archivo de entrada de donde se cargará la información y como se realizará la selección antes de su carga.
Load data,
Infile 'xxx'; Badfile 'yyy'; Discarfile 'zzz';
Append, Insert, Replace.
- B.- Especifica el comando INTO TABLE 'TTT', tablas a ser cargadas. Los DDL correspondientes(formatos).
- C.- La tercera parte es opcional, si está presente contiene los datos de entrada.

UTILITARIOS : SQL*Loader

Input data file.

SQL*Loader lee desde uno o más archivos indicados en control file.

Para SQL*Loader la data en el input está organizada como registros.

Un datafile puede ser:

Formato fijo.

Formato variable

Formato continuo.

Ejemplo: Formato Fijo.

Load data

Infile 'EJEMPLO.dat' "fix 10"

Into table ejemplo

Fields terminate by ','

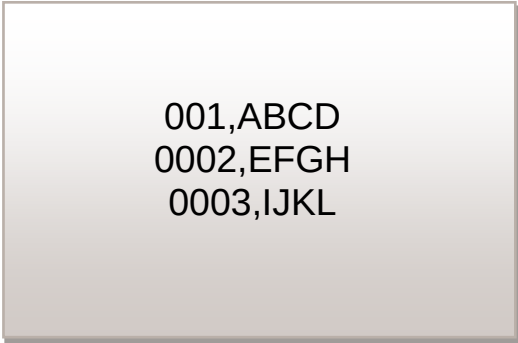
(coll1, coll2)

Ejemplo.dat

0001,abcd

0002,fghi

0003,jklm



001,ABCD
0002,EFGH
0003,IJKL

Log File

Contiene lo siguiente:

- 1.- Encabezado de información
- 2.- Información general.
- 3.- Información sobre tablas.
- 4.- Información sobre el input.
- 5.- Información sobre la carga.
- 6.- Sumario de variables.

UTILITARIOS : SQL*Loader

SQL*Loader: Release 8.1.6.3.0 - Production on Mar Ago 24 11:25:42 2004

(c) Copyright 1999 Oracle Corporation. All rights reserved.

Fichero de control: EJEMPLO.ctl
 Fichero de datos: EJEMPLO.dat
 Fichero defectuoso: EJEMPLO.bad
 Desechar fichero: ninguno especificado

(Permitir todos los registros desechados)
 Número a cargar: ALL
 Número a saltar: 0
 Errores permitidos: 50
 Vector de transferencia: 64 filas, máximo de 65536 bytes
 Continuación: ninguno especificado
 Ruta utilizada: Convencional

Tabla EJEMPLO, cargada de cada registro lógico.
 Opción INSERT activa para esta tabla: APPEND

Nombre Columna	Posición	Long	Term	Entorno	Tipo de dato
EJM_COL1	FIRST	4	,		CHARACTER
EJM_COL2	NEXT	4	,		CHARACTER

Tabla EJEMPLO:
 3 Filas se ha cargado con éxito.
 0 Filas no cargada debido a errores de datos.
 0 Filas no cargada porque todas las cláusulas WHEN han fallado.
 0 Filas no cargada porque todos los campos son nulos.

UTILITARIOS : SQL*Loader

Control file:

```

LOAD DATA
APPEND
INTO TABLE T_SN_TRAMAS_TMP FIELDS TERMINATED BY '|'
(
  TTMP_ID                "SQ_SN_TRAMAS_TMP.NEXTVAL",
  TTMP_GRP_PRC           CHAR(2),
  TTMP_FEC_HOR           DATE(16) "YYYYMMDDHH24MISS",
  TTMP_COD_OPC           CHAR(5),
  TTMP_COD_AGE           CHAR(5),
  TTMP_COD_WKS           CHAR(5),
  TTMP_NUM_OPR           DECIMAL EXTERNAL,
  TTMP_COD_CPB           DECIMAL EXTERNAL,
  TTMP_TRM_IN            CHAR(4000),
  TTMP_FEC_HOR_REG       "SYSDATE",
  TTMP_EDO               "'P'"
)

```

UTILITARIOS : SQL*Loader

Input Data y datafile.

SQL*Loader lee data desde uno a n archivos especificados en el control file.

Desde la perspectiva de SQL* Loader la data en el data file esta organizada como registros.

Un datafile puede ser de formato:

```

Fijo:      INFILE <datafile> "fix n"
Variable  INFILE <datafile> "var n"
Continuo  INFILE <datafile> "str cadena-fin".
          \n nueva línea
          \r control de carro

```

UTILITARIOS : SQL*Loader

Registros lógicos

SQL*Loader puede aplicar una de dos estrategias para formatear registros lógicos.

Combinar un conjunto de registros físicos para uno lógico.

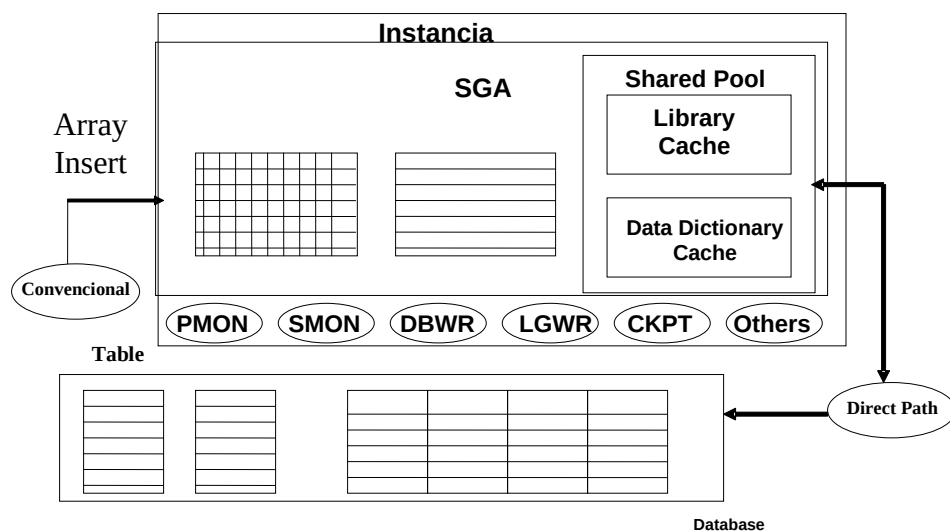
CONCATENATE “n”: Relaciona “n” registros físicos por cada lógico.

Generar los registros lógicos de acuerdo a que cumplan ciertas condiciones en los datos.

CONTINUEIF “condición: Se continuará leyendo el siguiente registro físico mientras no se cumpla la condición dada. Por ejemplo una condición podría ser que el registro contenga en la columna 80 el carácter “#”.

ARQUITECTURA

Métodos de carga:



UTILITARIOS : SQL*Loader

Métodos de carga .- SQL*Loader, provee de dos métodos para la carga de datos:

Ruta convencional.- Construye un arreglo de filas a ser insertadas y usa la sentencia SQL INSERT para la carga. Durante la ruta convencional los registros son cargados de acuerdo a las especificaciones de los atributos. Se insertan en la tabla especificada por el archivo de control.

Ruta Directa.- Por esta ruta se cargan bloques de datos contruidos en memoria hacia extents de las tablas. Se utilizan las especificaciones de los atributos para construir bloques. Esta ruta carga los datos haciendo un bypass a los buffers caché de memoria y acceden al SGA sólo para conocer los extents de grabación

UTILITARIOS : SQL*Loader

Comparación de métodos

CONVENCIONAL	DIRECTA
Usa COMMITS	Usa data save
Siempre se genera Redo Log	No siempre se generan Redo log.
Respetan todos los Controles	Solo respetan Clave primaria, no nula
Se activan los triggers	No se activan los triggers
Puedan cargar cluster	No pueden carga cluster
Otros usuarios puedes realizar cambios	Otros usuarios no pueden realizar cambios

UTILITARIOS : SQL*Loader**Conversión de datos**

Durante la carga vía convencional, los atributos se convierten en dos pasos:

- 1.- Los atributos especificados en el archivo de control son usados para interpretar el formato del input y convertidos para que la sentencia INSERT lo use como data.
- 2.- La base de datos acepta la data ejecutada por el Insert y lo almacena en la base de datos.

UTILITARIOS : SQL*Loader**Bad File**

Contiene lo siguiente:

- 1.- Registros con formatos inválidos
- 2.- Si la base de datos no lo acepta también será excluido

Discard file

- 1.- Son registros que cumplen con el formato y restricciones de integridad pero no con el criterio de selección en el control file.